MLREML: modelos lineales mixtos por REML con información promedio para el mejoramiento genético vegetal y animal

Guía de usuario, fundamentos teóricos y ejemplos con datos reales — versión 1.0.0

Osval Antonio Montesinos López, Abelardo Montesinos-López, José Crossa, J. Cricelio Montesinos-López

Octubre de 2026

Índice

1 Introducción 5
1.1 Qué es MLREML 5
1.2 Para quién es esta guía 5
1.3 Instalación 6
1.4 Capacidades y su correspondencia con ASReml 6
1.5 Un primer ejemplo 8
2 Fundamentos teóricos 9
2.1 El modelo lineal mixto 9
2.2 Estructuras como productos directos 9
2.3 Verosimilitud restringida 10
2.4 El algoritmo de información promedio 10
2.5 Estimación de efectos fijos y predicción de efectos aleatorios 12
2.6 Medias predichas y errores estándar de diferencias 13
2.7 Pruebas de hipótesis 13
2.8 Funciones de los parámetros de varianza 13
2.9 Matrices de relación 14
2.10 Costo computacional 14
2.11 Efectos correlacionados de varios factores: modelos directo–materno 14
2.12 El motor disperso: REML con las ecuaciones del modelo mixto 15
2.13 Grados de libertad de Kenward–Roger 17
3 Sintaxis de los modelos 18
3.1 La función `mlreml()` 18
3.2 Términos aleatorios 19
3.3 Estructuras residuales 21
3.4 Modelos multivariados 21
3.5 Nombres de parámetros, valores iniciales y restricciones 21
3.6 El objeto ajustado 21

4 Ejemplo 1: un ensayo de avena en diseño alfa-látice 22
4.1 Los datos 22
4.2 Variedades como efecto fijo: medias ajustadas 23
4.3 Variedades como efecto aleatorio: BLUP y heredabilidad 24
4.4 Comparación de medias ajustadas y BLUP 25
4.5 ¿Vale la pena el diseño de bloques incompletos? 26
4.6 Diagnóstico 26
4.7 Verificación con lme4 27
5 Ejemplo 2: análisis espacial de un ensayo de trigo 28
5.1 Los datos 28
5.2 Modelo de bloques completos 29
5.3 Modelo espacial AR1 × AR1 29
5.4 Tendencia global 29
5.5 Variograma muestral 30
5.6 Ganancia en precisión 31
5.7 Tendencia espacial estimada 32
5.8 Datos espaciales irregulares 32
6 Ejemplo 3: ensayos multiambientales y modelos factor analíticos 33
6.1 Los datos 33
6.2 La sucesión de modelos 34
6.3 Interpretación del modelo FA(1) 35
6.4 Predicciones por ambiente y desempeño general 36
6.5 Recomendaciones para análisis MET 38
7 Ejemplo 4: predicción genómica (GBLUP) con 599 líneas de trigo 38
7.1 Los datos 38
7.2 Control de calidad y matriz genómica 38
7.3 GBLUP en un ambiente 40
7.4 Predicción de líneas no fenotipadas 40
7.5 Validación cruzada: pedigree, genómica y ambos 41
7.6 GBLUP multiambiental 41
7.7 Notas prácticas 43
8 Ejemplo 5: modelo animal con pedigree en ganado lechero 43
8.1 Los datos 43
8.2 Preparación del pedigree 43
8.3 Modelo animal para la primera lactancia 44
8.4 Modelo de repetibilidad con todas las lactancias 46
8.5 Efecto materno 47
8.6 Análisis bivariado de leche y grasa 47

8.7 Efectos genéticos directos y maternos 49
8.8 Conjuntos de datos grandes: el motor disperso 52
9 Ejemplo 6: single-step GBLUP con individuos genotipados y no genotipados 54
9.1 Planteamiento 54
9.2 Alineación de G con el pedigree y construcción de H 54
9.3 Comparación por validación 55
9.4 Single-step con el motor general 55
9.5 Recomendaciones 56
10 Ejemplo 7: medidas repetidas y regresión aleatoria 56
10.1 Correlación residual: crecimiento de niños 56
10.2 Regresión aleatoria lineal: privación de sueño 57
10.3 Regresión aleatoria con polinomios de Legendre: curvas de crecimiento 59
11 Ejemplo 8: análisis dialélico 60
11.1 Los datos 60
11.2 El modelo 61
11.3 Interpretación 62
11.4 Aptitudes combinatorias predichas 62
11.5 Comparación de modelos 63
12 Ejemplo 9: ensayo no replicado de generación temprana 63
12.1 Los datos 63
12.2 Modelos 64
12.3 Consecuencias para la selección 65
12.4 Tendencia espacial y testigos 67
13 Uso avanzado y solución de problemas 67
13.1 Historia de iteraciones 67
13.2 Valores iniciales y reinicio 68
13.3 Parámetros fijos y verosimilitud perfil 69
13.4 Componentes de varianza conocidos 70
13.5 Efectos fijos confundidos 70
13.6 Parámetros en la frontera 71
13.7 Matrices de relación mal condicionadas 71
13.8 Escala de las variables 72
13.9 Memoria y tiempo 72
13.10 Problemas frecuentes 72
14 Estudio de simulación: propiedades de los estimadores 73
14.1 Diseño de bloques completos con genotipos aleatorios 73
14.2 Tamaño de la prueba de razón de verosimilitudes en la frontera 75
14.3 Modelo espacial 76

15 Validación numérica, rendimiento y limitaciones 77
15.1 Comparación con otros programas 77
15.2 Tiempo de cómputo y memoria 80
15.3 Estrategias para conjuntos de datos grandes 82
15.4 Limitaciones de la versión 1.0.0 83
16 Resumen de funciones 83
16.1 Equivalencias de sintaxis con ASReml-R 84
16.2 Guía rápida de modelos 85
16.3 Funciones posteriores al ajuste 86
Referencias 87
Apéndice: manual de referencia de las funciones 90
16.4 `MLREML-package`: MLREML: linear mixed models by Average-Information REML .. 90
16.5 `mlreml`: Fit a linear mixed model by Average-Information REML 90
16.6 `summary.mlreml`: Print and summary methods for mlreml fits 93
16.7 `coef.mlreml`: Extract fixed and random effects 94
16.8 `wald`: Wald tests for fixed effects 95
16.9 `predict.mlreml`: Predicted means with standard errors and standard errors of differences 96
16.10 `vpredict`: Functions of variance parameters with delta-method standard errors 96
16.11 `lrt`: Likelihood ratio test between nested models 97
16.12 `vc_matrix`: Estimated covariance and correlation matrix of a structured component 97
16.13 `variogram`: Sample variogram of residuals of a field trial 98
16.14 `plot.mlreml`: Diagnostic plots 98
16.15 `pedigree_prepare`: Prepare a pedigree 99
16.16 `inbreeding_coef`: Inbreeding coefficients (Meuwissen and Luo, 1992) 99
16.17 `Amatrix`: Numerator (additive) relationship matrix 99
16.18 `Ainverse`: Sparse inverse of the additive relationship matrix 100
16.19 `snp_qc`: Quality control of a SNP matrix 100
16.20 `Gmatrix`: Genomic relationship matrix 101
16.21 `align_G`: Align a genomic matrix with the pedigree matrix 101
16.22 `Hmatrix`: Single-step relationship matrix H and its inverse 101
16.23 `bend`: Make a symmetric matrix positive definite (bending) 102
16.24 `gblup`: Fast GBLUP by spectral decomposition 102
16.25 `cv_gblup`: K-fold cross-validation of GBLUP 103

1 Introducción

1.1 Qué es MLREML

MLREML es un paquete de R para ajustar modelos lineales mixtos por máxima verosimilitud restringida (REML) con el algoritmo de información promedio (*Average Information*, AI). Está pensado para los problemas típicos del mejoramiento genético vegetal y animal: estimación de componentes de varianza y covarianza, heredabilidades y correlaciones genéticas, predicción de valores genéticos (BLUP, EBV y GEBV), análisis de ensayos multiambientales, análisis espacial de experimentos de campo, modelos con pedigree y con información genómica, modelos multivariados y modelos longitudinales.

El nombre combina las dos ideas que organizan el paquete: modelos lineales mixtos (*Mixed Linear models*) y su estimación por REML. Su sintaxis está inspirada en la de ASReml (Gilmour et al., 2015; Butler et al., 2017), el programa de referencia en este campo, de modo que un usuario de ASReml puede leer y escribir modelos de MLREML casi sin aprendizaje adicional. MLREML es, sin embargo, software independiente y de código abierto: no contiene código de ASReml, no depende de él y no requiere licencia.

La idea central del paquete es la misma que hace poderoso a ASReml. Un modelo lineal mixto no se define solo por los efectos que contiene, sino por la forma en que esos efectos y los residuos se distribuyen y se correlacionan. Por eso cada término aleatorio de MLREML se escribe como un producto directo de estructuras de varianza: por ejemplo, `us(env):vm(gen, G)` declara efectos genéticos de cada genotipo en cada ambiente, correlacionados entre genotipos según la matriz genómica $\mathbf{G}$ y entre ambientes según una matriz de covarianza no estructurada que se estima a partir de los datos.

MLREML incluye además la covarianza entre los efectos genéticos directos y maternos (y, en general, entre efectos aleatorios de varios factores que comparten una matriz de relación) mediante el término `str()`; un segundo motor de cálculo basado en las ecuaciones del modelo mixto con matrices dispersas, para conjuntos de datos con cientos de miles de registros; y grados de libertad del denominador de Kenward–Roger para las pruebas F de los efectos fijos.

1.2 Para quién es esta guía

Esta guía está escrita para investigadores, profesores y estudiantes de posgrado en genética cuantitativa, mejoramiento genético, estadística experimental y bioestadística. Supone conocimientos básicos de R y de modelos lineales. La teoría de los modelos mixtos se presenta de forma completa en el Capítulo 2, con la notación que se usa en el resto del documento, de modo que la guía puede usarse también como material de curso.

El documento tiene tres partes. La primera (Capítulos 1 a 3) describe el modelo, el algoritmo de estimación y la sintaxis. La segunda (Capítulos 4 a 12) presenta ejemplos completos con datos reales de dominio público: un diseño alfa-látice de avena, un ensayo de trigo con análisis espacial, un ensayo multiambiental de maíz, 599 líneas de trigo del CIMMYT genotipadas con marcadores, registros de producción de leche de vacas Holstein con su pedigree, curvas de crecimiento de pollos, tiempos de reacción de personas en un estudio de privación de sueño, un dialelo completo de tabaco y un ensayo no replicado de generación temprana de trigo. La tercera parte (Capítulos 13 a 16) trata el uso avanzado, un estudio de simulación, la validación numérica del paquete frente a otros programas, sus límites y un resumen de todas las funciones; la referencia completa de las funciones está en el apéndice.

1.3 Instalación

El paquete se distribuye como un archivo fuente `MLREML_1.0.0.tar.gz` disponible en https://github.com/osval78/MLREML. Depende de los paquetes `Matrix` (versión 1.6-0 o posterior, que se distribuye con R) y `Rcpp`, y de `methods`, `stats`, `graphics` y `utils`. El paquete contiene código C++ (la inversa dispersa que usa el motor disperso y el cálculo de los coeficientes de consanguinidad), por lo que se instala desde el código fuente con un compilador: Rtools en Windows, las herramientas de línea de comandos de Xcode en macOS (`xcode-select --install`) y las herramientas de desarrollo habituales en Linux (por ejemplo, el paquete `r-base-dev` de Debian y Ubuntu):

```
install.packages("Rcpp")
install.packages("MLREML_1.0.0.tar.gz", repos = NULL, type = "source")
library(MLREML)
```

Los ejemplos de esta guía usan datos de los paquetes `agridat` (Wright, 2024), `BGLR` (Pérez y de los Campos, 2014), `pedigreemm` (Vazquez et al., 2010) y `lme4` (Bates et al., 2015), y comparan resultados con `lme4`, `nlme` (Pinheiro y Bates, 2000) y `pbkrtest` (Halekoh y Højsgaard, 2014). Ninguno de ellos es necesario para usar MLREML:

```
install.packages(c("agridat", "BGLR", "pedigreemm", "lme4", "nlme",
"pbkrtest"))
```

Para que el álgebra lineal sea rápida se recomienda usar R con una biblioteca BLAS optimizada (OpenBLAS, Intel MKL o Accelerate en macOS). El tiempo de un ajuste con el motor denso depende sobre todo de la descomposición de Cholesky de matrices $n \times n$, y una BLAS optimizada la acelera entre 10 y 30 veces respecto a la BLAS de referencia. El motor disperso también se beneficia, porque los bloques densos de su factorización dispersa y de su inversa dispersa se calculan con BLAS y LAPACK.

1.4 Capacidades y su correspondencia con ASReml

La tabla siguiente resume las capacidades de ASReml que se mencionan habitualmente en plant y animal breeding y la forma de obtenerlas en MLREML. Se indica con franqueza qué está implementado por completo, qué de forma parcial y qué no está disponible en esta versión.

Capacidades de ASReml y su equivalente en MLREML.

Capacidad (ASReml)	En MLREML	Estado
Modelos lineales mixtos, datos desbalanceados	`mlreml()`	Completo
REML con información promedio (AI)	algoritmo AI con amortiguamiento	Completo
Componentes de varianza con errores estándar	`summary()`, inversa de la matriz AI	Completo
Heredabilidad, correlaciones, funciones de componentes	`vpredict()` (método delta)	Completo
Modelo animal con pedigree, EBV	`vm(animal, A)`, `Amatrix()`, `Ainverse()`	Completo
Coeficientes de consanguinidad	`inbreeding_coef()` (Meuwissen y Luo)	Completo

Capacidad (ASReml)	En MLREML	Estado
GBLUP	`vm(gen, G)`, `Gmatrix()`, `gblup()`	Completo
Single-step GBLUP (matriz H)	`Hmatrix()`, `align_G()`, `bend()`	Completo
Control de calidad de SNP	`snp_qc()`	Básico
Modelos multivariados, correlación genética	`cbind(y1, y2) ~ trait`, `us(trait)`	Completo
Ensayos multiambientales (MET), G×E	`diag()`, `us()`, `fa(k)`	Completo
Varianzas residuales heterogéneas	`residual = ~ diag(env):units`	Completo
Análisis espacial fila × columna	`residual = ~ ar1(row):ar1(col)`	Completo
Espacial irregular	`expc(x)`, `sexp(x, y)`	Completo
Medidas repetidas, ambiente permanente	`id(animal)` + `vm(animal, A)`, `ar1()`	Completo
Regresión aleatoria	`us(leg(x, k)):id(animal)`, `pol()`	Completo
Efectos maternos	términos adicionales `id(dam)`, `vm(dam, A)`	Completo
Dialelos (ACG, ACE, recíprocos)	`mm(p1, p2)`, `id(cruza)`	Completo
BLUP, PEV, confiabilidad	`coef(fit, "random")`	Completo
Medias predichas y SED	`predict(fit, classify = ...)`	Completo
Pruebas de Wald incrementales y condicionales	`wald()`	Completo (las pruebas condicionales respetan la marginalidad)
Comparación de modelos (LRT, AIC, BIC)	`lrt()`, `AIC`, `BIC`	Completo
Variograma muestral	`variogram()`	Completo
Covarianza genética directa-materna	`str(animal, dam, K = A)`	Completo
Métodos de matrices dispersas (millones de registros)	`engine = "sparse"` (ecuaciones del modelo mixto)	Completo para términos `id`, `vm`, `diag`, `us` y `str()`
Grados de libertad de Kenward–Roger	`wald(fit, ddf = "KR")`	Completo (motor denso)
Modelos lineales generalizados mixtos	—	No disponible

MLREML tiene dos motores de cálculo. El motor denso usa el método basado en **V**: es exacto, simple y fácil de verificar, y permite estructuras muy generales sin código especial, pero limita su uso cómodo a unos 5,000–6,000 registros en una computadora personal con 8 GB de memoria. El motor disperso resuelve las ecuaciones del modelo mixto con una factorización

de Cholesky dispersa y la inversa dispersa de la matriz de coeficientes, como lo hacen los programas usados en las evaluaciones genéticas nacionales; admite las estructuras más usadas con conjuntos de datos grandes y ajusta modelos animales con cientos de miles de registros en la misma computadora (Secciones 2.12 y 15.2). Para GBLUP con un solo efecto genético, la función `gblup()` ofrece una alternativa exacta basada en la descomposición espectral de **G**. El Capítulo 15 documenta los tiempos de cómputo y la memoria.

1.5 Un primer ejemplo

Para mostrar la forma general de un análisis, consideremos un experimento simulado con 30 genotipos evaluados en 4 bloques completos. El modelo es

$$y_{ij} = \mu + \beta_j + g_i + e_{ij}, \qquad g_i \sim N\left(0, \sigma_g^2\right), \qquad e_{ij} \sim N(0, \sigma_e^2),$$

con bloques fijos y genotipos aleatorios.

```
set.seed(1)
d <- data.frame(block = factor(rep(1:4, each = 30)), gen = factor(rep(1:30,
4)))
d$y <- 10 + rnorm(4)[d$block] + rnorm(30, sd = 1.5)[d$gen] + rnorm(120)
fit <- mlreml(y ~ block, random = ~ id(gen), data = d, trace = FALSE)
summary(fit)

## MLREML fit (AI-REML)
## Observations: 120   Fixed-effect columns: 4   Iterations: 6 (converged)
## REML log-likelihood: -192.9292   AIC: 389.86   BIC: 395.37
##
## Variance parameters:
##                 estimate std.error z.ratio bound
## id(gen)!sigma2   1.9621    0.5686    3.45     P
## R!units!sigma2   0.7992    0.1212    6.60     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

La salida muestra el número de observaciones, las columnas de efectos fijos, las iteraciones, el logaritmo de la verosimilitud restringida y los criterios de información. Luego da cada parámetro de varianza con su error estándar aproximado (obtenido de la inversa de la matriz de información promedio), el cociente *z* y un código de frontera. La heredabilidad en sentido amplio a nivel de media de genotipo y su error estándar se obtienen con `vpredict()`:

```
vpredict(fit, H2 ~ V1 / (V1 + V2 / 4))

##    estimate std.error
## H2   0.9076   0.02802

head(coef(fit, "random", "id(gen)"))

##   level     blup    pev     se reliability
## 1     1  0.14700 0.2407 0.4906      0.8773
## 2     2 -1.03768 0.2407 0.4906      0.8773
## 3     3 -0.29958 0.2407 0.4906      0.8773
## 4     4  1.02779 0.2407 0.4906      0.8773
## 5     5  0.47789 0.2407 0.4906      0.8773
## 6     6 -0.08843 0.2407 0.4906      0.8773
```

La segunda instrucción devuelve los BLUP de los genotipos con su varianza del error de predicción (PEV), su error estándar y su confiabilidad $1 - \text{PEV}/\sigma_g^2$.

2 Fundamentos teóricos

Este capítulo presenta el modelo que ajusta MLREML, la verosimilitud restringida, el algoritmo de información promedio tal como está implementado, y las cantidades que el paquete calcula después del ajuste: estimadores de efectos fijos, predictores de efectos aleatorios, varianzas del error de predicción, medias predichas, pruebas de hipótesis y funciones de los parámetros de varianza.

2.1 El modelo lineal mixto

Sea $\mathbf{y}$ el vector de n observaciones. El modelo lineal mixto general es

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \sum_{k=1}^{K} \mathbf{Z}_k \, \mathbf{u}_k + \mathbf{e}, \qquad (2.1)$$

donde $\boldsymbol{\beta}$ es el vector de p efectos fijos con matriz de diseño $\mathbf{X}$ ($n \times p$, de rango completo después de eliminar columnas redundantes), $\mathbf{u}_k$ es el vector de q_k efectos aleatorios del término k con matriz de diseño $\mathbf{Z}_k$, y $\mathbf{e}$ es el vector de residuos. Se supone que

$$\mathbf{u}_k \sim N\big(\mathbf{0}, \mathbf{G}_k(\boldsymbol{\theta})\big), \qquad \mathbf{e} \sim N\big(\mathbf{0}, \mathbf{R}(\boldsymbol{\theta})\big), \qquad \mathrm{Cov}(\mathbf{u}_k, \mathbf{u}_l) = \mathbf{0}, \ \mathrm{Cov}(\mathbf{u}_k, \mathbf{e}) = \mathbf{0},$$

y que las matrices $\mathbf{G}_k$ y $\mathbf{R}$ dependen de un vector de parámetros de varianza $\boldsymbol{\theta}$ (varianzas, covarianzas, correlaciones, cargas factoriales). La distribución marginal de las observaciones es

$$\mathbf{y} \sim N(\mathbf{X}\boldsymbol{\beta}, \mathbf{V}), \qquad \mathbf{V}(\boldsymbol{\theta}) = \sum_{k=1}^{K} \mathbf{Z}_k \, \mathbf{G}_k \mathbf{Z}_k{}' + \mathbf{R}. \qquad (2.2)$$

Toda la flexibilidad del modelo está en la forma de $\mathbf{G}_k$ y $\mathbf{R}$. En un análisis tradicional de varianza, cada $\mathbf{G}_k$ es $\sigma_k^2 \mathbf{I}$ y $\mathbf{R} = \sigma_e^2 \mathbf{I}$. En genética cuantitativa, en cambio, $\mathbf{G}$ puede ser $\sigma_a^2 \mathbf{A}$ (pedigree), $\sigma_g^2 \mathbf{G}_{\text{genómica}}$, o $\boldsymbol{\Sigma}_E \otimes \mathbf{K}$ en un ensayo multiambiental; y $\mathbf{R}$ puede ser diagonal con varianzas distintas por ambiente, o tener la estructura autorregresiva separable de un ensayo de campo.

2.2 Estructuras como productos directos

MLREML construye cada $\mathbf{G}_k$ como un producto directo (de Kronecker) de matrices pequeñas, una por cada factor que define el término:

$$\mathbf{G}_k = \sigma_k^2 \, \boldsymbol{\Sigma}_{k1} \otimes \boldsymbol{\Sigma}_{k2} \otimes \cdots \otimes \boldsymbol{\Sigma}_{kC}. \qquad (2.3)$$

Por ejemplo, para efectos de genotipo dentro de ambiente con E ambientes y N genotipos, $\mathbf{u}$ tiene EN elementos ordenados con el genotipo variando más rápido, y $\mathbf{G} = \boldsymbol{\Sigma}_E \otimes \mathbf{K}$. La matriz de diseño del término es el producto *face-splitting* (producto de Khatri–Rao por filas) de las matrices de diseño de cada factor: si $\mathbf{z}_{c,i}{}'$ es la fila i de la matriz de diseño $\mathbf{Z}_c$ del factor c, la fila i de $\mathbf{Z}_k$ es $\mathbf{z}_{1,i}{}' \otimes \mathbf{z}_{2,i}{}' \otimes \cdots$.

Una propiedad algebraica sencilla permite calcular la contribución de cada término a $\mathbf{V}$ sin formar nunca $\mathbf{Z}_k$ ni $\mathbf{G}_k$ completos. Para el producto face-splitting se cumple

$$\mathbf{Z}_k(\boldsymbol{\Sigma}_1 \otimes \boldsymbol{\Sigma}_2)\mathbf{Z}_k{}' = (\mathbf{Z}_1 \boldsymbol{\Sigma}_1 \mathbf{Z}_1{}') \circ (\mathbf{Z}_2 \boldsymbol{\Sigma}_2 \mathbf{Z}_2{}'), \qquad (2.4)$$

donde $\circ$ es el producto de Hadamard (elemento a elemento). Cuando el factor c es una variable de clasificación, $\mathbf{Z}_c$ tiene un solo uno por fila y $\mathbf{Z}_c\boldsymbol{\Sigma}_c\mathbf{Z}_c'$ es simplemente la matriz $\boldsymbol{\Sigma}_c$ indexada por los niveles de cada observación: su elemento (i,j) es $\Sigma_c[\ell_c(i),\ell_c(j)]$, donde $\ell_c(i)$ es el nivel del factor c en la observación i. Así, una matriz genómica de 6,547 animales se usa solo en las filas y columnas de los animales con registros, y el término `us(env):vm(gen, G)` contribuye a $\mathbf{V}$ con los elementos $\Sigma_E[e_i,e_j]\,K[g_i,g_j]$. La identidad (2.4) es la que permite a MLREML combinar libremente cualquier estructura con cualquier otra.

Los residuos se tratan del mismo modo. La estructura por defecto, `units`, es la identidad sobre las observaciones. La estructura `diag(env):units` produce $R_{ij}=\sigma^2_{e(i)}\delta_{ij}$, varianzas residuales distintas por ambiente. La estructura `ar1(row):ar1(col)` produce

$$R_{ij}=\sigma^2\,\rho_r^{|r_i-r_j|}\,\rho_c^{|c_i-c_j|}, \qquad (2.5)$$

el modelo espacial separable de Gilmour et al. (1997), donde r_i y c_i son la fila y la columna de la parcela i. Como la matriz se construye a partir de las coordenadas de cada observación, no es necesario que el ensayo sea una cuadrícula completa: las parcelas faltantes simplemente no aparecen.

2.3 Verosimilitud restringida

Los efectos fijos se eliminan de la verosimilitud mediante contrastes de error $\mathbf{K}'\mathbf{y}$, con $\mathbf{K}'\mathbf{X}=\mathbf{0}$, cuya distribución no depende de $\boldsymbol{\beta}$ (Patterson y Thompson, 1971). Salvo una constante, el logaritmo de la verosimilitud restringida es

$$\ell_R(\boldsymbol{\theta})=-\frac{1}{2}\left[\log|\mathbf{V}|+\log|\mathbf{X}'\mathbf{V}^{-1}\mathbf{X}|+\mathbf{y}'\mathbf{P}\mathbf{y}+(n-p)\log(2\pi)\right], \qquad (2.6)$$

con la matriz de proyección

$$\mathbf{P}=\mathbf{V}^{-1}-\mathbf{V}^{-1}\mathbf{X}(\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}. \qquad (2.7)$$

REML corrige el sesgo de máxima verosimilitud debido a la estimación de los efectos fijos; en un diseño balanceado sus estimadores coinciden con los del análisis de varianza cuando estos son positivos. El valor que reporta MLREML incluye la constante $(n-p)\log(2\pi)$ y es igual al de `lme4` y `nlme`, lo que facilita comparar programas (Capítulo 15). Dos verosimilitudes REML solo son comparables si los modelos tienen exactamente los mismos efectos fijos.

2.4 El algoritmo de información promedio

Sea $\mathbf{V}_i=\partial\mathbf{V}/\partial\theta_i$. Las derivadas primeras de (2.6), que forman el vector *score*, son

$$s_i=\frac{\partial\ell_R}{\partial\theta_i}=-\frac{1}{2}\left[\operatorname{tr}(\mathbf{P}\mathbf{V}_i)-\mathbf{y}'\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{y}\right]. \qquad (2.8)$$

La matriz de información observada contiene términos con trazas de productos de cuatro matrices $n\times n$, muy costosos, y la de Fisher contiene $\operatorname{tr}(\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j)$, también cara. Gilmour, Thompson y Cullis (1995) observaron que el promedio de ambas, cuando $\mathbf{V}$ es lineal en los parámetros, es

$$\mathcal{I}_{A,ij}=\frac{1}{2}\,\mathbf{y}'\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j\mathbf{P}\mathbf{y}, \qquad (2.9)$$

que solo requiere productos de matrices por vectores: si $\mathbf{w}_i=\mathbf{V}_i\mathbf{P}\mathbf{y}$ y $\mathbf{W}=[\mathbf{w}_1,\ldots,\mathbf{w}_m]$, entonces $\mathcal{I}_A=1/2\,\mathbf{W}'\mathbf{P}\mathbf{W}$. La matriz de información promedio es semidefinida positiva y es

una aproximación de tipo Gauss–Newton a la información también cuando $\mathbf{V}$ no es lineal en $\boldsymbol{\theta}$ (correlaciones autorregresivas, cargas factoriales). La actualización es

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \left[\mathcal{I}_A^{(t)}\right]^{-1}\mathbf{s}^{(t)}. \qquad (2.10)$$

2.4.1 Implementación en MLREML

En cada iteración el motor denso de MLREML (i) construye $\mathbf{V}$ con la identidad (2.4); (ii) calcula su factor de Cholesky $\mathbf{V} = \mathbf{U}'\mathbf{U}$, $\log|\mathbf{V}| = 2\sum \log u_{ii}$, $\mathbf{V}^{-1}$ y $\mathbf{P}$; (iii) para cada parámetro libre forma $\mathbf{V}_i$ —de nuevo mediante productos de Hadamard, sustituyendo la matriz del componente por su derivada—, calcula $tr(\mathbf{PV}_i) = \sum_{jk} P_{jk}\, V_{i,jk}$ y $\mathbf{w}_i$; y (iv) obtiene $\mathbf{s}$ e $\mathcal{I}_A$. El motor disperso obtiene las mismas cantidades a partir de las ecuaciones del modelo mixto (Sección 2.12). Las derivadas de cada estructura son analíticas; por ejemplo, para una autorregresiva $\partial\rho^d / \partial\rho = d\rho^{d-1}$ para toda distancia $d > 0$, incluidas las distancias menores que uno, y cero para $d = 0$, para una no estructurada la derivada respecto de σ_{ab} es $\mathbf{E}_{ab} + \mathbf{E}_{ba}$, y para un modelo factor analítico $\boldsymbol{\Lambda}\boldsymbol{\Lambda}' + \boldsymbol{\Psi}$ la derivada respecto de la carga λ_{lm} es $\mathbf{e}_l\boldsymbol{\lambda}_m{}' + \boldsymbol{\lambda}_m\mathbf{e}_l{}'$.

El paso de Newton puro (2.10) puede salir del espacio de parámetros (varianzas negativas, correlaciones mayores que uno, matrices no definidas positivas) o disminuir la verosimilitud lejos del óptimo. MLREML incorpora tres salvaguardas:

1. **Amortiguamiento de Levenberg–Marquardt.** El paso se calcula como $\left(\mathcal{I}_A + \lambda\,\mathrm{diag}(\mathcal{I}_A)\right)^{-1}\mathbf{s}$. Se empieza con $\lambda = 0$ (paso AI puro); si el candidato no es válido o reduce ℓ_R, λ se multiplica por 10. Cuando λ es grande el paso se acerca a un paso de gradiente corto, que siempre aumenta la verosimilitud. Tras un paso aceptado, λ se reduce.
2. **Restricciones de frontera.** Las varianzas se mantienen por encima de un límite inferior positivo muy pequeño; la correlación de `cs` en $(-1/(L-1) + 10^{-4}, 0.999)$, porque la matriz de simetría compuesta $L \times L$ es definida positiva solo para $\rho > -1/(L-1)$; y las correlaciones autorregresivas en $(-0.99, 0.999)$ cuando todas las distancias son enteras y en $(10^{-6}, 0.999)$ en otro caso, porque ρ^d no está definida para $\rho < 0$ y d no entero. Un parámetro que termina en su límite se marca con el código `B` en la salida, como en ASReml. Una varianza que está en su límite y cuyo score, o cuyo paso de Newton, apunta fuera del espacio paramétrico se mantiene en el límite y se elimina del sistema AI, y el paso de los demás parámetros se calcula condicionalmente (un método de conjunto activo). Recortar en cambio el paso conjunto podría dejar todos los parámetros prácticamente detenidos en un punto que no es un máximo.
3. **Proyección semidefinida.** Si una matriz no estructurada (`us`) deja de ser definida positiva, se proyecta sobre el cono de matrices semidefinidas positivas reemplazando sus valores propios negativos por un valor positivo pequeño. Esto permite que el algoritmo se acerque a soluciones en la frontera (por ejemplo, correlaciones genéticas cercanas a uno), frecuentes en ensayos multiambientales.

La convergencia se declara cuando el cambio de ℓ_R es menor que `tol` (por defecto 10^{-6}) y el decremento de Newton $\mathbf{s}'\mathcal{I}_A^{-1}\mathbf{s}$, calculado sin los parámetros detenidos en su frontera, es menor que 10^{-3}, o el cambio relativo de los parámetros es despreciable. En este último caso, si el decremento de Newton sigue siendo grande —lo que ocurre cuando una matriz no estructurada converge a una matriz singular—, una advertencia recomienda revisar las estimaciones y considerar una estructura reducida.

Valores iniciales. Se calcula la varianza v_p de los residuos de mínimos cuadrados ordinarios del modelo de efectos fijos y se reparte por igual entre los términos aleatorios y el residuo. Las varianzas específicas de nivel (estructuras `diag`, `us`, `fa`) empiezan en la varianza residual observada dentro de cada nivel, multiplicada por la misma proporción, lo que da valores de la escala correcta en modelos multivariados con caracteres medidos en unidades distintas. Las correlaciones autorregresivas empiezan en el valor que da una correlación de 0.5 a la distancia mínima entre niveles. El usuario puede dar sus propios valores iniciales con el argumento `start` y fijar parámetros con `fix`.

Errores estándar. La inversa de $\mathcal{I}_A$ en el óptimo es una aproximación a la matriz de covarianza asintótica de $\widehat{\boldsymbol{\theta}}$ y da los errores estándar de la tabla de componentes. Para parámetros en la frontera esta aproximación no es válida y el error estándar debe ignorarse.

2.5 Estimación de efectos fijos y predicción de efectos aleatorios

Dado $\widehat{\boldsymbol{\theta}}$, el estimador de mínimos cuadrados generalizados (BLUE empírico) de los efectos fijos y su varianza son

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}\mathbf{y}, \qquad \operatorname{Var}(\widehat{\boldsymbol{\beta}}) = \mathbf{C}_{11} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}, \qquad (2.11)$$

y el mejor predictor lineal insesgado (BLUP empírico) de cada término aleatorio es

$$\widehat{\mathbf{u}}_k = \mathbf{G}_k\mathbf{Z}_k'\mathbf{V}^{-1}\left(\mathbf{y} - \mathbf{X}\widehat{\boldsymbol{\beta}}\right) = \mathbf{G}_k\mathbf{Z}_k'\mathbf{P}\mathbf{y}. \qquad (2.12)$$

Estas expresiones son la solución de las ecuaciones del modelo mixto de Henderson (1975),

$$\begin{bmatrix} \mathbf{X}'\mathbf{R}^{-1}\mathbf{X} & \mathbf{X}'\mathbf{R}^{-1}\mathbf{Z} \\ \mathbf{Z}'\mathbf{R}^{-1}\mathbf{X} & \mathbf{Z}'\mathbf{R}^{-1}\mathbf{Z} + \mathbf{G}^{-1} \end{bmatrix} \begin{bmatrix} \widehat{\boldsymbol{\beta}} \\ \widehat{\mathbf{u}} \end{bmatrix} = \begin{bmatrix} \mathbf{X}'\mathbf{R}^{-1}\mathbf{y} \\ \mathbf{Z}'\mathbf{R}^{-1}\mathbf{y} \end{bmatrix},$$

pero el motor denso de MLREML las evalúa directamente en la forma (2.12), que no requiere invertir $\mathbf{G}$. Por eso las matrices de relación singulares (por ejemplo, una matriz genómica con menos marcadores que individuos, o líneas genéticamente idénticas) no causan problemas, siempre que $\mathbf{V}$ sea definida positiva. El motor disperso (Sección 2.12) resuelve las propias ecuaciones y necesita $\mathbf{G}^{-1}$.

Cuando un término usa una matriz de relación con más niveles que los observados —por ejemplo, la matriz $\mathbf{A}$ de un pedigree con antepasados sin registros, o la matriz genómica de candidatos a selección no fenotipados—, (2.12) predice también esos niveles, porque $\mathbf{G}_k\mathbf{Z}_k'$ incluye sus covarianzas con los individuos observados.

Varianza del error de predicción. La matriz inversa de los coeficientes de las ecuaciones de Henderson, $\mathbf{C}$, da la covarianza conjunta de los errores de estimación y predicción. Sus bloques, en términos de $\mathbf{V}$, son

$$\mathbf{C}_{11} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}, \quad \mathbf{C}_{21} = -\mathbf{G}\mathbf{Z}'\mathbf{V}^{-1}\mathbf{X}\mathbf{C}_{11}, \quad \mathbf{C}_{22} = \operatorname{Var}(\widehat{\mathbf{u}} - \mathbf{u}) = \mathbf{G} - \mathbf{G}\mathbf{Z}'\mathbf{P}\mathbf{Z}\mathbf{G}. \qquad (2.13)$$

La diagonal de $\mathbf{C}_{22}$ es la PEV de cada BLUP. La confiabilidad (reliability), muy usada en animal breeding, es $r_i^2 = 1 - PEV_i/G_{ii}$, el cuadrado de la correlación entre el valor genético verdadero y el predicho. El motor denso de MLREML calcula la PEV por bloques de filas de $\mathbf{G}\mathbf{Z}'$, sin formar $\mathbf{G}$, de modo que la memoria necesaria es proporcional a n por el tamaño del bloque; el motor disperso las toma de la diagonal de la inversa dispersa de $\mathbf{C}$ (Sección 2.12).

2.6 Medias predichas y errores estándar de diferencias

Una predicción es una combinación lineal $\boldsymbol{\tau} = \mathbf{L}\boldsymbol{\beta} + \mathbf{M}\mathbf{u}$ (Welham et al., 2004). La función `predict()` construye $\mathbf{L}$ promediando las filas de la matriz de diseño sobre los niveles de los factores fijos que no están en la clasificación (promedio simple) y evaluando las covariables en su media; $\mathbf{M}$ selecciona los BLUP de los términos aleatorios cuyos factores están todos en la clasificación, y los demás términos aleatorios se ignoran porque su esperanza es cero. La varianza del error de predicción es

$$\operatorname{Var}(\hat{\boldsymbol{\tau}} - \boldsymbol{\tau}) = \mathbf{L}\mathbf{C}_{11}\mathbf{L}' + \mathbf{L}\mathbf{C}_{12}\mathbf{M}' + \mathbf{M}\mathbf{C}_{21}\mathbf{L}' + \mathbf{M}\mathbf{C}_{22}\mathbf{M}', \qquad (2.14)$$

y el error estándar de la diferencia entre dos predicciones es $\mathrm{SED}_{ab} = \sqrt{v_{aa} + v_{bb} - 2v_{ab}}$. El promedio de los SED es la base de la heredabilidad generalizada de Cullis, Smith y Coombes (2006), $H_C^2 = 1 - \overline{\mathrm{SED}}^2 / \left(2\sigma_g^2\right)$, útil en ensayos desbalanceados.

2.7 Pruebas de hipótesis

Efectos fijos. El estadístico de Wald de una hipótesis $\mathbf{L}\boldsymbol{\beta} = \mathbf{0}$, con $q = rank(\mathbf{L})$, es $W = \left(\mathbf{L}\hat{\boldsymbol{\beta}}\right)'\left(\mathbf{L}\mathbf{C}_{11}\mathbf{L}'\right)^{-1}\mathbf{L}\hat{\boldsymbol{\beta}}$, asintóticamente χ_q^2. La función `wald()` da dos versiones para cada término T de la fórmula fija. El estadístico *incremental* ajusta T por los términos que lo preceden en la fórmula (pruebas secuenciales o de "tipo I"): si $\mathbf{U}'\mathbf{U} = \mathbf{C}_{11}^{-1} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{X}$ es la factorización de Cholesky, $\mathbf{L}$ se forma con las filas de $\mathbf{U}$ correspondientes a las columnas de T, y $W_T = \sum_{j \in T} \left(\mathbf{U}\hat{\boldsymbol{\beta}}\right)_j^2$. El estadístico *condicional* ajusta T por todos los términos que no lo contienen, respetando el principio de marginalidad como lo hace ASReml (pruebas de "tipo II"): en `y ~ A * B` el efecto principal `A` se ajusta por `B` pero no por `A:B`, y la interacción se ajusta por ambos efectos principales. Se calcula como el estadístico incremental después de reordenar las columnas de modo que T siga a los términos que no lo contienen y preceda a los que lo contienen. Ajustar cada término por todos los demás, incluidas las interacciones que lo contienen, probaría, con contrastes de tratamiento, el efecto de `A` en el nivel de referencia de `B`. Cada estadístico se acompaña de una prueba F. Por defecto (`ddf = "auto"`), los ajustes del motor denso usan el método de Kenward y Roger (1997), que ajusta la matriz de covarianza de $\hat{\boldsymbol{\beta}}$ y el estadístico F por la estimación de los parámetros de varianza y da los grados de libertad del denominador de cada prueba (Sección 2.13). Con `ddf = "residual"`, y siempre para los ajustes del motor disperso, $F = W/q$ con $n - p$ grados de libertad en el denominador; esta aproximación es adecuada con muchos datos y liberal en experimentos pequeños.

Parámetros de varianza. Dos modelos anidados con los mismos efectos fijos se comparan con la razón de verosimilitudes $LR = 2\left(\ell_{R,1} - \ell_{R,0}\right)$. Cuando se prueba si una sola varianza es cero, el parámetro está en la frontera y la distribución nula es una mezcla 50:50 de χ_0^2 y χ_1^2 (Self y Liang, 1987; Stram y Lee, 1994), de modo que el valor p es la mitad del de una χ_1^2. La función `lrt()` aplica esta corrección. Para modelos no anidados se usan el $\mathrm{AIC} = -2\ell_R + 2m$ y el $\mathrm{BIC} = -2\ell_R + m\log(n - p)$, con m el número de parámetros de varianza.

2.8 Funciones de los parámetros de varianza

Las heredabilidades, repetibilidades y correlaciones genéticas son funciones no lineales $f(\boldsymbol{\theta})$. Su error estándar aproximado se obtiene por el método delta,

$$\operatorname{Var}\left[f\left(\hat{\boldsymbol{\theta}}\right)\right] \approx \nabla f\left(\hat{\boldsymbol{\theta}}\right)' \mathcal{I}_A^{-1} \nabla f\left(\hat{\boldsymbol{\theta}}\right), \qquad (2.15)$$

que es lo que hace `vpredict()`, con el gradiente calculado por diferencias centrales. Los parámetros se nombran `V1`, `V2`, … en el orden de la tabla de componentes.

2.9 Matrices de relación

Pedigree. El coeficiente de parentesco aditivo entre dos individuos es el doble del coeficiente de coancestría. La matriz $\mathbf{A}$ se construye con el método tabular: si s y d son los padres de i, $A_{ji} = 1/2\,(A_{js} + A_{jd})$ para $j < i$ y $A_{ii} = 1 + 1/2\,A_{sd}$, donde $F_i = 1/2\,A_{sd}$ es el coeficiente de consanguinidad. La inversa se construye directamente con las reglas de Henderson (1976) con consanguinidad (Quaas, 1976): con $b_i = [1/2 - 1/4\,(F_s + F_d)]^{-1}$ (usando $F = -1$ para padres desconocidos), se suma b_i al elemento (i,i), $-b_i/2$ a (i,s), (s,i), (i,d), (d,i), y $b_i/4$ a (s,s), (d,d), (s,d), (d,s). Los coeficientes F se calculan con el algoritmo de Meuwissen y Luo (1992).

Genómica. Con la matriz $\mathbf{M}$ de genotipos codificados $0/1/2$ y las frecuencias alélicas p_j, VanRaden (2008) define $\mathbf{W} = \mathbf{M} - 2\mathbf{p}'$ y $\mathbf{G} = \mathbf{W}\mathbf{W}'/\left[2\sum_j p_j\,(1-p_j)\right]$. La alternativa estandarizada divide cada columna de $\mathbf{W}$ por $\sqrt{2p_j(1-p_j)}$ y el producto por el número de marcadores. GBLUP con $\sigma^2_g\mathbf{G}$ es equivalente a una regresión ridge sobre los marcadores (Meuwissen et al., 2001; VanRaden, 2008).

Single-step. Cuando solo una parte de los individuos está genotipada, Legarra et al. (2009) combinan pedigree y genómica en

$$\mathbf{H} = \mathbf{A} + \begin{bmatrix}\mathbf{A}_{12}\mathbf{A}_{22}^{-1}\\ \mathbf{I}\end{bmatrix}(\mathbf{G} - \mathbf{A}_{22})[\mathbf{A}_{22}^{-1}\mathbf{A}_{21} \quad \mathbf{I}], \qquad \mathbf{H}^{-1} = \mathbf{A}^{-1} + \begin{bmatrix}\mathbf{0} & \mathbf{0}\\ \mathbf{0} & \tau\mathbf{G}^{-1} - \omega\mathbf{A}_{22}^{-1}\end{bmatrix}, \tag{2.16}$$

donde el índice 2 corresponde a los genotipados. Para que $\mathbf{G}$ y $\mathbf{A}_{22}$ estén en la misma escala, $\mathbf{G}$ se alinea como $\alpha + \beta\mathbf{G}$ de modo que las medias de su diagonal y de todos sus elementos coincidan con las de $\mathbf{A}_{22}$ (Christensen et al., 2012), y se mezcla con una pequeña proporción de $\mathbf{A}_{22}$ para asegurar que sea invertible.

2.10 Costo computacional

El paso dominante de cada iteración del motor denso es la factorización y la inversión de $\mathbf{V}$, con costo $O(n^3)$ en tiempo y $O(n^2)$ en memoria. Cada parámetro añade $O(n^2)$ operaciones para su traza y su vector $\mathbf{w}_i$. La predicción de los BLUP y sus PEV cuesta $O(qn^2)$ con memoria $O(n \times \text{block})$. El costo del motor disperso es el de la factorización de Cholesky dispersa de la matriz de coeficientes de las ecuaciones del modelo mixto y el de su inversa dispersa, ambos proporcionales a $\sum_i |S_i|^2$, donde $|S_i|$ es el número de elementos distintos de cero por debajo de la diagonal en la columna i del factor de Cholesky, más una resolución con el factor por parámetro; su memoria es proporcional al número de elementos distintos de cero del factor (Sección 2.12). El Capítulo 15 presenta tiempos medidos.

2.11 Efectos correlacionados de varios factores: modelos directo–materno

En muchas especies el fenotipo de un individuo depende de sus propios genes (efecto genético directo) y de los genes de su madre, expresados a través del ambiente materno que ella proporciona: producción de leche, habilidad materna, capacidad uterina (efecto genético materno). Los dos efectos de un mismo animal suelen estar correlacionados, y el modelo de Willham (1963) los trata conjuntamente:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}_d\mathbf{a}_d + \mathbf{Z}_m\mathbf{a}_m + \cdots + \mathbf{e}, \qquad Var\begin{pmatrix}\mathbf{a}_d\\ \mathbf{a}_m\end{pmatrix} = \begin{pmatrix}\sigma^2_d & \sigma_{dm}\\ \sigma_{dm} & \sigma^2_m\end{pmatrix} \otimes \mathbf{A} = \boldsymbol{\Sigma} \otimes \mathbf{A}, \tag{2.17}$$

donde $\mathbf{a}_d$ y $\mathbf{a}_m$ contienen los valores genéticos directos y maternos de todos los animales del pedigree, $\mathbf{Z}_d$ vincula cada registro con el animal que lo produjo y $\mathbf{Z}_m$ lo vincula con la madre de ese animal. Ambos vectores están indexados por los mismos animales, pero un registro recibe el valor directo de un animal y el valor materno de otro, de modo que el término no puede escribirse como un producto directo de componentes y la identidad (2.4) no se aplica. Su contribución a $\mathbf{V}$ es una suma de tres matrices,

$$\mathbf{Z}(\boldsymbol{\Sigma} \otimes \mathbf{A})\mathbf{Z}' = \sigma_d^2\, \mathbf{Z}_d\mathbf{A}\mathbf{Z}_d{}' + \sigma_m^2\, \mathbf{Z}_m\mathbf{A}\mathbf{Z}_m{}' + \sigma_{dm}(\mathbf{Z}_d\mathbf{A}\mathbf{Z}_m{}' + \mathbf{Z}_m\mathbf{A}\mathbf{Z}_d{}'), \qquad \mathbf{Z} = [\mathbf{Z}_d, \mathbf{Z}_m], \tag{2.18}$$

que es lineal en los tres parámetros; sus derivadas $\mathbf{V}_i$ son las tres matrices de (2.18). El elemento (i, j) de $\mathbf{Z}_d\mathbf{A}\mathbf{Z}_m{}'$ es el parentesco entre el animal del registro i y la madre del registro j. Los registros cuya madre es desconocida tienen una fila de ceros en $\mathbf{Z}_m$: contribuyen a la estimación de los efectos directos y se conservan en el análisis.

MLREML ajusta este modelo con el término `str(f1, f2, ..., K = K, cov = "us")`, que generaliza (2.17) a $m \geq 2$ factores cuyos niveles son las filas de la misma matriz $\mathbf{K}$: $\mathbf{u} = (\mathbf{u}_1{}', \ldots, \mathbf{u}_m{}')'$, $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_m]$ y $Var(\mathbf{u}) = \boldsymbol{\Sigma} \otimes \mathbf{K}$, con $\boldsymbol{\Sigma}$ no estructurada (`cov = "us"`, $m(m+1)/2$ parámetros) o diagonal (`cov = "diag"`). El BLUP del efecto a y su varianza del error de predicción se obtienen de (2.12) y (2.13):

$$\hat{\mathbf{u}}_a = \sum_{b=1}^{m} \Sigma_{ab}\ \mathbf{K}\mathbf{Z}_b{}'\mathbf{P}\mathbf{y}, \qquad PEV_{a,l} = \Sigma_{aa}K_{ll} - \mathbf{g}_{a,l}{}'\mathbf{P}\mathbf{g}_{a,l}, \qquad \mathbf{g}_{a,l}{}' = \sum_{b=1}^{m} \Sigma_{ab}\ \mathbf{k}_l{}'\mathbf{Z}_b{}', \tag{2.19}$$

donde $\mathbf{k}_l{}'$ es la fila l de $\mathbf{K}$. Además de la heredabilidad directa $h_d^2 = \sigma_d^2/\sigma_P^2$ y la correlación genética directa–materna $r_{dm} = \sigma_{dm}/(\sigma_d\sigma_m)$, la heredabilidad total de Willham (1972), $h_T^2 = (\sigma_d^2 + 1.5\sigma_{dm} + 0.5\sigma_m^2)/\sigma_P^2$, mide la respuesta a la selección sobre el fenotipo. La varianza genética materna solo es identificable si el pedigree vincula madres con registros propios o de parientes, y las estimaciones de σ_{dm} suelen ser negativas e imprecisas (Meyer, 1992); si es necesario, se añade un efecto materno de ambiente permanente como un término separado `id(dam)`.

2.12 El motor disperso: REML con las ecuaciones del modelo mixto

El método de las secciones anteriores trabaja con la matriz $n \times n$ $\mathbf{V}$ y es exacto y general, pero su costo crece con n^3. Los programas usados en las evaluaciones genéticas nacionales (ASReml, DMU, WOMBAT, BLUPF90) trabajan en cambio con las ecuaciones del modelo mixto de Henderson, cuya matriz de coeficientes es dispersa. MLREML incluye este segundo **motor disperso**. Sean $\mathbf{T} = [\mathbf{X}, \mathbf{Z}]$, $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_K]$ y $\mathbf{G} = \text{blockdiag}(\mathbf{G}_1, \ldots, \mathbf{G}_K)$. La matriz de coeficientes de las ecuaciones es

$$\mathbf{C} = \mathbf{T}'\mathbf{R}^{-1}\mathbf{T} + \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{G}^{-1} \end{bmatrix}, \qquad \mathbf{C}\begin{bmatrix} \hat{\boldsymbol{\beta}} \\ \hat{\mathbf{u}} \end{bmatrix} = \mathbf{T}'\mathbf{R}^{-1}\mathbf{y}. \tag{2.20}$$

Como $|\mathbf{V}||\mathbf{X}'\mathbf{V}^{-1}\mathbf{X}| = |\mathbf{R}||\mathbf{G}||\mathbf{C}|$, la log-verosimilitud restringida (2.6) puede escribirse sin $\mathbf{V}$:

$$\ell_R = -\frac{1}{2}[\log|\mathbf{R}| + \log|\mathbf{G}| + \log|\mathbf{C}| + \mathbf{y}'\mathbf{P}\mathbf{y} + (n-p)\log(2\pi)], \qquad \mathbf{y}'\mathbf{P}\mathbf{y} = \mathbf{y}'\mathbf{R}^{-1}\mathbf{y} - \left[\hat{\boldsymbol{\beta}}', \hat{\mathbf{u}}'\right]\mathbf{T}'\mathbf{R}^{-1}\mathbf{y}. \tag{2.21}$$

El valor de ℓ_R es el mismo que el del motor denso. Con $\hat{\mathbf{e}} = \mathbf{y} - \mathbf{T}\left[\hat{\boldsymbol{\beta}}', \hat{\mathbf{u}}'\right]'$, el score (2.8) se convierte en

$$s_i = -\frac{1}{2}\left[tr(\mathbf{R}^{-1}\mathbf{R}_i) + tr(\mathbf{G}^{-1}\mathbf{G}_i) + tr(\mathbf{C}^{-1}\mathbf{C}_i) + \hat{\mathbf{e}}'\frac{\partial \mathbf{R}^{-1}}{\partial \theta_i}\hat{\mathbf{e}} + \hat{\mathbf{u}}'\frac{\partial \mathbf{G}^{-1}}{\partial \theta_i}\hat{\mathbf{u}}\right], \qquad (2.22)$$

con $\mathbf{R}_i = \partial\mathbf{R}/\,\partial\theta_i$, $\mathbf{G}_i = \partial\mathbf{G}/\,\partial\theta_i$ y $\mathbf{C}_i = \partial\mathbf{C}/\,\partial\theta_i$, que es blockdiag$(\mathbf{0}, \partial\mathbf{G}^{-1}/\,\partial\theta_i)$ para un parámetro de $\mathbf{G}$ y $\mathbf{T}'(\partial\mathbf{R}^{-1}/\,\partial\theta_i)\mathbf{T}$ para un parámetro de $\mathbf{R}$.

Inversa dispersa. La traza $tr(\mathbf{C}^{-1}\mathbf{C}_i)$ solo necesita los elementos de $\mathbf{C}^{-1}$ en las posiciones donde $\mathbf{C}_i$ no es cero, todas las cuales están en el patrón del factor de Cholesky de $\mathbf{C}$. Sea $\mathbf{C} = \mathbf{L}\mathbf{L}'$ después de una permutación de filas y columnas que reduce el relleno (fill-in), y sea S_i el conjunto de filas de los elementos distintos de cero de la columna i de $\mathbf{L}$ por debajo de la diagonal. Los elementos Z_{ji} de $\mathbf{C}^{-1}$ en el patrón de $\mathbf{L}$ se obtienen, columna por columna de la última a la primera, con las recursiones de Takahashi et al. (1973) y Erisman y Tinney (1975), usadas en mejoramiento animal por Misztal y Perez-Enciso (1993):

$$Z_{ji} = -\sum_{k\in S_i}\frac{L_{ki}}{L_{ii}}Z_{kj} \quad (j \in S_i), \qquad Z_{ii} = \frac{1}{L_{ii}^2} - \sum_{k\in S_i}\frac{L_{ki}}{L_{ii}}Z_{ki}. \qquad (2.23)$$

Del lado derecho solo se necesitan elementos del patrón de $\mathbf{L}$, porque ese patrón es cerrado: si k y j pertenecen a S_i, el elemento (k, j) pertenece al patrón. El costo es proporcional a $\sum_i |S_i|^2$, el mismo orden que el de la propia factorización, y la memoria es igual a la de $\mathbf{L}$. La diagonal del bloque de los efectos aleatorios da las varianzas del error de predicción, y el bloque de los efectos fijos da $Var(\hat{\boldsymbol{\beta}})$, sin costo adicional.

Información promedio. Las variables de trabajo $\mathbf{w}_i = \mathbf{V}_i\mathbf{P}\mathbf{y}$ de (2.9) se calculan sin $\mathbf{V}$, ya que $\mathbf{P}\mathbf{y} = \mathbf{R}^{-1}\hat{\mathbf{e}}$ y $\mathbf{Z}'\mathbf{P}\mathbf{y} = \mathbf{G}^{-1}\hat{\mathbf{u}}$: $\mathbf{w}_i = \mathbf{Z}\mathbf{G}_i\mathbf{G}^{-1}\hat{\mathbf{u}}$ para un parámetro de $\mathbf{G}$ y $\mathbf{w}_i = \mathbf{R}_i\mathbf{R}^{-1}\hat{\mathbf{e}}$ para un parámetro de $\mathbf{R}$. Como $\mathbf{P} = \mathbf{R}^{-1} - \mathbf{R}^{-1}\mathbf{T}\mathbf{C}^{-1}\mathbf{T}'\mathbf{R}^{-1}$,

$$\mathcal{I}_{A,ij} = \frac{1}{2}\mathbf{w}_i'\mathbf{P}\mathbf{w}_j = \frac{1}{2}\left(\mathbf{w}_i'\mathbf{R}^{-1}\mathbf{w}_j - \mathbf{t}_i'\mathbf{C}^{-1}\mathbf{t}_j\right), \qquad \mathbf{t}_i = \mathbf{T}'\mathbf{R}^{-1}\mathbf{w}_i, \qquad (2.24)$$

lo que requiere una resolución con el factor de $\mathbf{C}$ por parámetro (Johnson y Thompson, 1995; Gilmour et al., 1995).

Estructuras. Para un término $\mathbf{G}_k = \mathbf{K}_1 \otimes \boldsymbol{\Sigma} \otimes \mathbf{K}_2$, donde $\boldsymbol{\Sigma}$ es la única matriz con parámetros (o σ^2 para un término sin estructura de varianza) y $\mathbf{K}_1$, $\mathbf{K}_2$ son identidades o matrices de relación, $\mathbf{G}_k^{-1} = \mathbf{K}_1^{-1} \otimes \boldsymbol{\Sigma}^{-1} \otimes \mathbf{K}_2^{-1}$; $\log|\mathbf{G}_k|$ se obtiene de $\log|\mathbf{A} \otimes \mathbf{B}| = b\log|\mathbf{A}| + a\log|\mathbf{B}|$ para matrices de órdenes a y b; y $\mathbf{G}_i\mathbf{G}^{-1}\hat{\mathbf{u}} = (\mathbf{I} \otimes \boldsymbol{\Sigma}_i\boldsymbol{\Sigma}^{-1} \otimes \mathbf{I})\hat{\mathbf{u}}$ solo involucra la matriz pequeña $\boldsymbol{\Sigma}$. Las matrices de relación solo se necesitan a través de sus inversas, de modo que $\mathbf{A}^{-1}$ construida con las reglas de Henderson (Sección 2.9) puede darse directamente. Un término `str()` es $\boldsymbol{\Sigma} \otimes \mathbf{K}$ con $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_m]$. La matriz residual es diagonal por bloques, con un bloque por registro: $\sigma_e^2\mathbf{I}$ (`units`), $diag(\sigma^2_{e(i)})$ (`diag(f):units`) o, en modelos multivariados, la submatriz de $\boldsymbol{\Sigma}_e$ de los caracteres observados en cada registro (`us(trait):units`).

Implementación. $\mathbf{C}$ se ensambla con un patrón fijo (la unión de todas las posiciones que pueden ser distintas de cero), de modo que el ordenamiento que reduce el relleno y la factorización simbólica de CHOLMOD (Chen et al., 2008), disponible en R a través del paquete `Matrix`, se calculan una sola vez y en cada iteración solo se repite la factorización numérica. Las recursiones (2.23) están programadas en C++ y usan una mezcla ordenada de cada columna con las columnas de sus filas, sin búsquedas. Los puntos candidatos de los pasos amortiguados solo necesitan la factorización, porque $\log|\mathbf{C}| = 2\sum_i \log L_{ii}$. Las medias predichas usan $Var(\hat{\boldsymbol{\tau}} - \boldsymbol{\tau}) = [\mathbf{L}, \mathbf{M}]\mathbf{C}^{-1}[\mathbf{L}, \mathbf{M}]'$, equivalente a (2.14), obtenida resolviendo con el factor de $\mathbf{C}$.

El motor disperso admite términos aleatorios construidos con componentes `id`, `vm`, `diag` y `us` (con factores o bases `pol()`, `leg()` o `mm()`), términos `str()` y las estructuras residuales `units`, `diag(f):units` y `us(trait):units`. Las estructuras con parámetros de correlación (`cs`, `ar1`, `expc`, `sexp`) y `fa` se ajustan con el motor denso. Con `engine = "auto"`, la opción por defecto, MLREML usa el motor disperso cuando los datos tienen más de 5,000 registros y el modelo es admitido, y el motor denso en otro caso. Ambos motores dan la misma verosimilitud, estimaciones, BLUP y varianzas del error de predicción salvo errores de redondeo (Sección 15.1). Como el motor disperso necesita $\mathbf{G}^{-1}$, las matrices de relación deben ser definidas positivas; una matriz genómica singular debe corregirse (bending) o combinarse (Sección 13.7), o bien ajustarse con el motor denso.

2.13 Grados de libertad de Kenward–Roger

La matriz de covarianza $\boldsymbol{\Phi} = \left(\mathbf{X}'\widehat{\mathbf{V}}^{-1}\mathbf{X}\right)^{-1}$ de (2.11) trata los parámetros de varianza como conocidos. En experimentos pequeños subestima la varianza de $\widehat{\boldsymbol{\beta}}$, y las pruebas F con $n-p$ grados de libertad en el denominador rechazan una hipótesis verdadera con demasiada frecuencia. Kenward y Roger (1997) propusieron una matriz de covarianza ajustada,

$$\boldsymbol{\Phi}_A = \boldsymbol{\Phi} + 2\boldsymbol{\Phi}\left\{\sum_{i,j} W_{ij}\left(\mathbf{Q}_{ij} - \mathbf{P}_i\boldsymbol{\Phi}\mathbf{P}_j - \frac{1}{4}\mathbf{R}_{ij}\right)\right\}\boldsymbol{\Phi}, \qquad (2.25)$$

con $\mathbf{P}_i = -\mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_i\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{Q}_{ij} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_i\mathbf{V}^{-1}\mathbf{V}_j\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{R}_{ij} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_{ij}\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{V}_{ij} = \partial^2\mathbf{V}/\,\partial\theta_i\,\partial\theta_j$, y $\mathbf{W}$ la inversa de la matriz de información esperada, cuyos elementos son $\frac{1}{2}tr\left(\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j\right)$. Para una hipótesis $\mathbf{L}\boldsymbol{\beta} = \mathbf{0}$ con $q = rank(\mathbf{L})$ usan el estadístico

$$F = \frac{\lambda}{q}\left(\mathbf{L}\widehat{\boldsymbol{\beta}}\right)'\left(\mathbf{L}\boldsymbol{\Phi}_A\mathbf{L}'\right)^{-1}\mathbf{L}\widehat{\boldsymbol{\beta}}, \qquad (2.26)$$

y eligen la escala λ y los grados de libertad del denominador m de modo que la media y la varianza de F coincidan aproximadamente con las de una distribución $F(q,m)$. Con $\boldsymbol{\Theta} = \mathbf{L}'\left(\mathbf{L}\boldsymbol{\Phi}\mathbf{L}'\right)^{-1}\mathbf{L}$,

$$A_1 = \sum_{i,j} W_{ij}\; tr(\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_i\boldsymbol{\Phi})tr\left(\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_j\boldsymbol{\Phi}\right), \qquad A_2 = \sum_{i,j} W_{ij}\; tr\left(\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_i\boldsymbol{\Phi}\boldsymbol{\Theta}\boldsymbol{\Phi}\mathbf{P}_j\boldsymbol{\Phi}\right),$$

$$B = \frac{A_1 + 6A_2}{2q}, \quad g = \frac{(q+1)A_1 - (q+4)A_2}{(q+2)A_2}, \quad c_1 = \frac{g}{3q + 2(1-g)}, \quad c_2 = \frac{q-g}{3q+2(1-g)}, \quad c_3 = \frac{q+2-g}{3q+2(1-g)},$$

$$E^* = \left(1 - \frac{A_2}{q}\right)^{-1}, \quad V^* = \frac{2}{q}\,\frac{1 + c_1B}{(1-c_2B)^2(1-c_3B)}, \quad \rho = \frac{V^*}{2E^{*2}}, \quad m = 4 + \frac{q+2}{q\rho - 1}, \quad \lambda = \frac{m}{E^*(m-2)}. \qquad (2.27)$$

En los diseños balanceados en los que la prueba F es exacta, el método la reproduce: $\lambda = 1$ y m es igual a los grados de libertad clásicos (por ejemplo, los del error de parcela grande en un diseño de parcelas divididas).

MLREML calcula (2.25)–(2.27) para cada prueba incremental y condicional de `wald()` (Sección 2.7); la matriz $\mathbf{L}$ de cada prueba la dan las filas del factor de Cholesky usado para calcular el estadístico de Wald. Se incluyen todos los parámetros de varianza libres excepto

los que están en la frontera del espacio paramétrico, que se tratan como conocidos. Las segundas derivadas $\mathbf{V}_{ij}$ son cero para las estructuras lineales en sus parámetros (`id`, `vm`, `diag`, `us`, `str()`) y se incluyen analíticamente para las demás (`cs`, `ar1`, `expc`, `sexp`, cargas de `fa` y productos de dos componentes con parámetros, como `ar1(row):ar1(col)` con su escala). El ajuste usa las matrices de derivadas $n \times n$ y está disponible para los ajustes del motor denso guardados con `store = TRUE`; cuando la información esperada necesitaría más de 2 GB de memoria, se usa en su lugar la matriz de información promedio. Con el motor disperso, pensado para conjuntos de datos grandes en los que el ajuste es despreciable, `wald()` usa $n - p$ grados de libertad. Los resultados coinciden con los del paquete `pbkrtest` (Halekoh y Højsgaard, 2014) en cinco o más dígitos significativos (Sección 15.1).

3 Sintaxis de los modelos

3.1 La función `mlreml()`

La llamada general es

```
fit <- mlreml(fixed, random = NULL, residual = NULL, data,
              start = NULL, fix = NULL, maxit = 50, tol = 1e-6,
              pev = TRUE, trace = TRUE, store = TRUE,
              engine = c("auto", "dense", "sparse"))
```

Argumentos de `mlreml()`.

Argumento	Significado
`fixed`	Fórmula de dos lados para la respuesta y los efectos fijos, con la sintaxis habitual de R (`y ~ rep + gen`).
`random`	Fórmula de un lado con los términos aleatorios (`~ gen + rep:block`).
`residual`	Fórmula de un lado con un solo término para la estructura residual. Por defecto `~ units`.
`data`	Marco de datos. Las filas con respuesta o predictores faltantes se eliminan, excepto los valores faltantes de los factores de un término `str()` (Sección 2.11).
`start`	Vector con nombres de valores iniciales de los parámetros de varianza.
`fix`	Nombres de parámetros que se mantienen fijos en su valor inicial.
`maxit`, `tol`	Número máximo de iteraciones y tolerancia del cambio de ℓ_R.
`pev`	Calcular las varianzas del error de predicción de los BLUP.
`trace`	Mostrar la historia de iteraciones.
`store`	Guardar las matrices que necesita `predict()` y el ajuste de Kenward–Roger de `wald()`; con `FALSE` se ahorra memoria.
`engine`	Motor de cálculo: `"dense"` (matriz de varianzas $\mathbf{V}$), `"sparse"` (ecuaciones del modelo mixto) o `"auto"` (disperso con más de 5,000 registros cuando el modelo es admitido, denso en otro caso). Ver la Sección 2.12.

Los efectos fijos se tratan como en `lm()`: los factores generan contrastes de tratamiento, las interacciones se escriben con `:` o `*`, y las columnas linealmente dependientes se eliminan automáticamente (los efectos “aliased” de ASReml).

3.2 Términos aleatorios

Cada término aleatorio es un producto de **componentes** separados por dos puntos. Cada componente consta de una **estructura** aplicada a una **base**. La base define a qué nivel pertenece cada observación y la estructura define la matriz de covarianza (o de correlación) de los efectos de esa base.

3.2.1 Bases

Bases de un componente.

Base	Descripción
`f`	Un factor o una variable de clasificación de los datos. Si es numérica, sus valores distintos son los niveles, ordenados.
`pol(x, k)`	Polinomio crudo de orden k con intercepto: columnas $1, x, \ldots, x^k$.
`leg(x, k)`	Polinomios de Legendre normalizados de orden k sobre x reescalado a $[-1,1]$, como en los modelos de regresión aleatoria de animal breeding.
`mm(f1, f2, ...)`	Factor de pertenencia múltiple: cada observación pertenece a los niveles de `f1` y de `f2`. Sirve para la aptitud combinatoria general de los dos padres.
`units`	Las unidades de observación (un nivel por registro; en modelos multivariados, un nivel por registro original).

3.2.2 Estructuras

La tabla siguiente lista las estructuras disponibles, la matriz que generan para un componente con L niveles y su número de parámetros. Las estructuras de **correlación** no tienen escala propia; las de **varianza** sí.

Estructuras de varianza.

Estructura	Matriz $L \times L$	Parámetros	Tipo
`id(f)` o `f`	$\mathbf{I}_L$	0	correlación
`vm(f, K)`	matriz conocida $\mathbf{K}$ (A, G, H, parentesco)	0	correlación
`cs(f)`	$(1-\rho)\mathbf{I} + \rho\mathbf{J}$	1	correlación
`ar1(f)`	$\rho^{\lvert x_a - x_b \rvert}$ (distancias reales si los niveles son números)	1	correlación
`expc(x)`	$\rho^{\lvert x_a - x_b \rvert}$ con x continua irregular	1	correlación
`sexp(x, y)`	$\rho^{d_{ab}}$ con d la distancia euclidiana en el plano	1	correlación
`diag(f)`	$\mathrm{diag}(\sigma_1^2, \ldots, \sigma_L^2)$	L	varianza
`us(f)`	matriz no estructurada	$L(L+1)/2$	varianza
`fa(f, k)`	$\mathbf{\Lambda\Lambda}' + \mathbf{\Psi}$, $\mathbf{\Lambda}$ de $L \times k$	$Lk + L$ (menos $k(k-1)/2$ ceros)	varianza

Un término formado solo por componentes de correlación recibe un parámetro de escala σ^2. Un término puede tener como máximo un componente de varianza, que entonces lleva la escala; por ejemplo, en `us(env):vm(gen, G)` la matriz `us` es la covarianza genética entre ambientes en la escala de los datos y `vm` aporta las correlaciones entre genotipos. Dos componentes de varianza en el mismo término no son identificables (multiplicar uno por c y dividir el otro por c no cambia el modelo), y MLREML lo rechaza con un mensaje de error.

En `fa(f, k)` con $k > 1$, las cargas por encima de la diagonal de $\mathbf{\Lambda}$ se fijan en cero para eliminar la indeterminación por rotación. Las cargas estimadas pueden rotarse después para interpretarlas; `vc_matrix()` devuelve también la solución rotada por componentes principales.

Términos correlacionados. El término `str(f1, f2, ..., K = K, cov = "us")` no es un producto sino una suma de componentes (Sección 2.11): define un efecto aleatorio para cada uno de los factores `f1`, `f2`, …, cuyos niveles son las filas de la matriz `K`, con $Var(\mathbf{u}) = \mathbf{\Sigma} \otimes \mathbf{K}$, donde $\mathbf{\Sigma}$ es no estructurada (`cov = "us"`, la opción por defecto) o diagonal (`cov = "diag"`). Los factores deben tomar valores en `rownames(K)`; un valor faltante (`NA`, `""` o `"0"`) significa que el registro no recibe efecto de ese factor, y el registro se conserva. Si se omite `K`, se usa la matriz identidad. Los parámetros se nombran como los de `us()`: por ejemplo, `str(animal, dam, K = A)!dam:animal` es la covarianza directa–materna.

3.2.3 Ejemplos de términos

Ejemplos de términos aleatorios.

Término	Modelo
`~ gen`	Genotipos independientes, una varianza.
`~ rep:block`	Bloques incompletos dentro de repeticiones.
`~ vm(animal, A)`	Modelo animal: $\mathbf{a} \sim N(\mathbf{0}, \sigma_a^2 \mathbf{A})$.
`~ vm(animal, Ai), Ai <- Ainverse(ped)`	El mismo modelo con $\mathbf{A}$ dada por su inversa dispersa.
`~ vm(animal, A) + animal`	Modelo de repetibilidad: valor genético más ambiente permanente.
`~ str(animal, dam, K = A)`	Efectos genéticos directos y maternos con su covarianza, $\mathbf{\Sigma} \otimes \mathbf{A}$.
`~ str(animal, dam, K = A) + id(dam)`	Lo mismo con un efecto materno de ambiente permanente.
`~ vm(gen, G)`	GBLUP.
`~ env:gen`	Interacción G×E homogénea (simetría compuesta).
`~ diag(env):id(gen)`	Varianza genética distinta en cada ambiente, sin covarianza.
`~ us(env):vm(gen, G)`	MET genómico con covarianza genética no estructurada entre ambientes.
`~ fa(env, 2):id(gen)`	MET con modelo factor analítico de dos factores.
`~ us(trait):vm(animal, A)`	Matriz de covarianza genética aditiva entre caracteres.
`~ us(leg(dim, 2)):vm(animal, A)`	Regresión aleatoria genética de orden 2.
`~ mm(p1, p2) + cruza`	Dialelo: ACG de los padres y ACE de la cruza.
`~ units`	Efecto *nugget* (error independiente adicional) en modelos espaciales.

Las matrices de `vm()` y `str()` se buscan en el ambiente donde se creó la fórmula fija, normalmente el espacio de trabajo. Deben tener nombres de fila que contengan todos los niveles del factor; pueden incluir niveles adicionales. Una matriz de relación también puede darse mediante su inversa si lleva el atributo `INVERSE = TRUE`, como el resultado de `Ainverse()`: el motor disperso la usa directamente y el motor denso la invierte.

3.3 Estructuras residuales

El argumento `residual` acepta un solo término con la misma sintaxis. Las estructuras más usadas son:

Estructuras residuales.

Residual	**R**
`~ units`	$\sigma_e^2 \mathbf{I}$ (por defecto)
`~ diag(env):units`	Varianza residual distinta en cada ambiente
`~ ar1(row):ar1(col)`	Espacial separable (2.5)
`~ diag(env):ar1(row):ar1(col)`	Espacial por ambiente con varianzas propias y correlaciones comunes
`~ ar1(time):id(animal)`	Medidas repetidas autocorrelacionadas dentro de cada individuo
`~ expc(time):id(animal)`	Igual, con tiempos irregulares
`~ sexp(x, y)`	Espacial isotrópico con coordenadas irregulares
`~ us(trait):units`	Covarianza residual entre caracteres de un mismo registro (multivariado, por defecto)

Cuando el residuo es espacial suele añadirse un término aleatorio `units`, que representa el error de medición independiente.

3.4 Modelos multivariados

Si el lado izquierdo de `fixed` es `cbind(y1, y2, ...)`, MLREML apila los datos en formato largo, crea el factor `trait` con un nivel por carácter y la base `units` que identifica cada registro original. La fórmula de efectos fijos se escribe en términos de `trait`:

```
mlreml(cbind(peso, altura) ~ trait + trait:sexo,
       random = ~ us(trait):vm(animal, A),
       residual = ~ us(trait):units, data = datos)
```

Los registros con algún carácter faltante se conservan para los caracteres observados, que es una de las ventajas del análisis multivariado por REML.

3.5 Nombres de parámetros, valores iniciales y restricciones

Cada parámetro recibe un nombre con la forma `término!componente_nivel`; los del residuo empiezan con `R!`. Por ejemplo, `us(env):vm(gen, G)!env_E2:E1` es la covarianza genética entre los ambientes E2 y E1, y `R!diag(env):units!env_E1` es la varianza residual de E1. Estos nombres se usan en `start` y `fix`. Para fijar la varianza residual en 1 y estimar el resto:

```
fit0 <- mlreml(y ~ 1, random = ~ gen, data = d, trace = FALSE)
fit1 <- mlreml(y ~ 1, random = ~ gen, data = d,
               start = c("R!units!sigma2" = 1), fix = "R!units!sigma2")
```

Fijar todos los parámetros permite evaluar la verosimilitud y los BLUP con componentes conocidos, por ejemplo componentes publicados o estimados en otro conjunto de datos.

3.6 El objeto ajustado

`mlreml()` devuelve una lista de clase `"mlreml"`. Sus elementos principales son:

Elementos del objeto `mlreml`.

Elemento	Contenido
`varcomp`	Tabla de parámetros de varianza: estimación, error estándar, cociente z, código de frontera.
`theta`, `vcov_theta`	Vector de parámetros y su covarianza aproximada ($\mathcal{I}_A^{-1}$).
`beta`, `vcov_beta`	Efectos fijos y su matriz de covarianza (con el motor disperso y más de 1,000 efectos fijos, una matriz diagonal con las varianzas; `wald()` calcula la matriz completa cuando la necesita).
`blups`	Lista con un marco de datos por término aleatorio: nivel, BLUP, PEV, error estándar y confiabilidad (para un término `str()`, también la columna `effect`).
`fitted`, `residuals`	Valores ajustados ($\mathbf{X}\hat{\boldsymbol{\beta}} + \mathbf{Z}\hat{\mathbf{u}}$) y residuos condicionales.
`loglik`, `AIC`, `BIC`	Verosimilitud restringida y criterios de información.
`history`, `converged`	Historia de iteraciones y señal de convergencia.
`data`	Datos usados (en formato largo si el modelo es multivariado).
`engine`	Motor usado, `"dense"` o `"sparse"`.
`sparse`	Solo motor disperso: número de ecuaciones (`N`), elementos distintos de cero del factor de Cholesky (`nnzL`) y, con `store = TRUE`, el propio factor, usado por `predict()` y `wald()`.

Los métodos disponibles son `summary()`, `print()`, `coef()` (fijos o aleatorios), `fitted()`, `residuals()`, `vcov()`, `logLik()`, `AIC()`, `BIC()`, `plot()` y `predict()`. Las funciones `wald()`, `vpredict()`, `lrt()`, `vc_matrix()` y `variogram()` completan la inferencia.

4 Ejemplo 1: un ensayo de avena en diseño alfa-látice

4.1 Los datos

El conjunto `john.alpha` del paquete `agridat` proviene de John y Williams (1995). Son 24 variedades de avena evaluadas en un diseño alfa con 3 repeticiones, cada una dividida en 6 bloques incompletos de 4 parcelas. La respuesta es el rendimiento (t/ha). Es un ejemplo clásico de la recuperación de información interbloque: como cada bloque contiene solo 4 de las 24 variedades, las comparaciones entre variedades que no comparten bloque dependen de la varianza de bloques.

```
library(agridat)
data(john.alpha)
d <- john.alpha
str(d)

## 'data.frame':    72 obs. of  7 variables:
##  $ plot : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep  : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ block: Factor w/ 6 levels "B1","B2","B3",..: 1 1 1 1 2 2 2 2 3 3 ...
##  $ gen  : Factor w/ 24 levels "G01","G02","G03",..: 11 4 5 22 21 10 20 2 23
```

```
14 ...
##  $ yield: num  4.12 4.45 5.88 4.58 4.65 ...
##  $ row  : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ col  : int  1 1 1 1 1 1 1 1 1 1 ...
```

```
with(d, table(rep, block))[, 1:6]
```

```
##     block
## rep  B1 B2 B3 B4 B5 B6
##   R1  4  4  4  4  4  4
##   R2  4  4  4  4  4  4
##   R3  4  4  4  4  4  4
```

4.2 Variedades como efecto fijo: medias ajustadas

Cuando el objetivo es comparar variedades, estas se tratan como fijas y los bloques como aleatorios. El modelo es

$$y_{ijk} = \mu + r_j + \tau_i + b_{k(j)} + e_{ijk}, \qquad b_{k(j)} \sim N(0, \sigma_b^2), \qquad e_{ijk} \sim N(0, \sigma_e^2).$$

```
fit_f <- mlreml(yield ~ rep + gen, random = ~ rep:block, data = d, trace =
FALSE)
summary(fit_f)
```

```
## MLREML fit (AI-REML)
## Observations: 72   Fixed-effect columns: 26   Iterations: 5 (converged)
## REML log-likelihood: -32.4492   AIC: 68.90   BIC: 72.56
##
## Variance parameters:
##                  estimate std.error z.ratio bound
## rep:block!sigma2  0.06195   0.03656    1.69     P
## R!units!sigma2    0.08522   0.02185    3.90     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
wald(fit_f)
```

```
##          term df wald.inc    F.inc ddf.inc     p.inc wald.con    F.con
ddf.con
## 1 (Intercept)  1  4338.49 4338.495   11.52 3.307e-16  4338.49 4338.495
11.52
## 2         rep  2    18.42    9.212   11.52 4.078e-03    18.42    9.212
11.52
## 3         gen 23   125.30    5.363   35.50 4.495e-06   125.30    5.363
35.50
##       p.con   p.chisq
## 1 3.307e-16 0.000e+00
## 2 4.078e-03 9.981e-05
## 3 4.495e-06 4.588e-16
```

La prueba de Wald condicional para gen evalúa la hipótesis de igualdad de todas las variedades, ajustada por repeticiones. Con 72 parcelas y 26 columnas de efectos fijos, los grados de libertad residuales, 46, sobrestiman la información sobre las variedades, parte de la cual proviene de la comparación de bloques incompletos. El método de Kenward–Roger da 35.5 grados de libertad en el denominador para gen y 11.5 para rep, cuya información proviene sobre todo de los 18 bloques incompletos, y reduce ligeramente el estadístico F de gen (5.363 en lugar de 5.448). Los valores p de la prueba χ^2 (`p.chisq`) ignoran esta

incertidumbre y son demasiado pequeños. El estadístico condicional del intercepto es igual a su estadístico incremental, ya que todos los términos contienen al intercepto.

Las medias ajustadas de las variedades y sus errores estándar de diferencias se obtienen con `predict()`:

```
pf <- predict(fit_f, classify = "gen")
head(pf$pvals[order(-pf$pvals$predicted.value), ], 8)

##    gen predicted.value std.error
## 1  G01           5.108    0.1955
## 5  G05           5.037    0.1955
## 15 G15           4.969    0.1955
## 19 G19           4.840    0.1955
## 21 G21           4.795    0.1955
## 14 G14           4.776    0.1955
## 13 G13           4.758    0.1955
## 12 G12           4.755    0.1955

pf$avsed

## [1] 0.2647

range(pf$sed[upper.tri(pf$sed)])

## [1] 0.2574 0.2699
```

El SED no es constante: dos variedades que comparten bloques se comparan con más precisión que dos que nunca coinciden. Esta es la información que un análisis de bloques completos ignoraría. Una diferencia mínima significativa aproximada al 5 % es $t_{0.975,\mathrm{gl}} \times \overline{\mathrm{SED}}$:

```
gl_error <- nrow(d) - fit_f$p
qt(0.975, gl_error) * pf$avsed

## [1] 0.5329
```

4.3 Variedades como efecto aleatorio: BLUP y heredabilidad

En un programa de mejoramiento, las variedades son una muestra de una población y el objetivo es predecir su valor genético. Con variedades aleatorias se obtienen BLUP, que contraen las medias hacia la media general en proporción a su incertidumbre.

```
fit_r <- mlreml(yield ~ rep, random = ~ gen + rep:block, data = d, trace =
FALSE)
summary(fit_r)

## MLREML fit (AI-REML)
## Observations: 72   Fixed-effect columns: 3   Iterations: 4 (converged)
## REML log-likelihood: -46.5969   AIC: 99.19   BIC: 105.90
##
## Variance parameters:
##                  estimate std.error z.ratio bound
## gen!sigma2        0.14290   0.05208    2.74     P
## rep:block!sigma2  0.07022   0.03734    1.88     P
## R!units!sigma2    0.08162   0.02029    4.02     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
pr <- predict(fit_r, classify = "gen")
head(pr$pvals, 4)

##   gen predicted.value std.error
## 1 G01           4.981    0.1766
## 2 G02           4.484    0.1766
## 3 G03           3.695    0.1766
## 4 G04           4.486    0.1766
```

La heredabilidad en sentido amplio a nivel de media de variedad puede calcularse con la fórmula clásica para $r = 3$ repeticiones o con la heredabilidad generalizada de Cullis et al. (2006), que usa el SED promedio de los BLUP y es apropiada para diseños desbalanceados:

```
vpredict(fit_r, H2_clasica ~ V1 / (V1 + V3 / 3))

##            estimate std.error
## H2_clasica   0.8401   0.06254

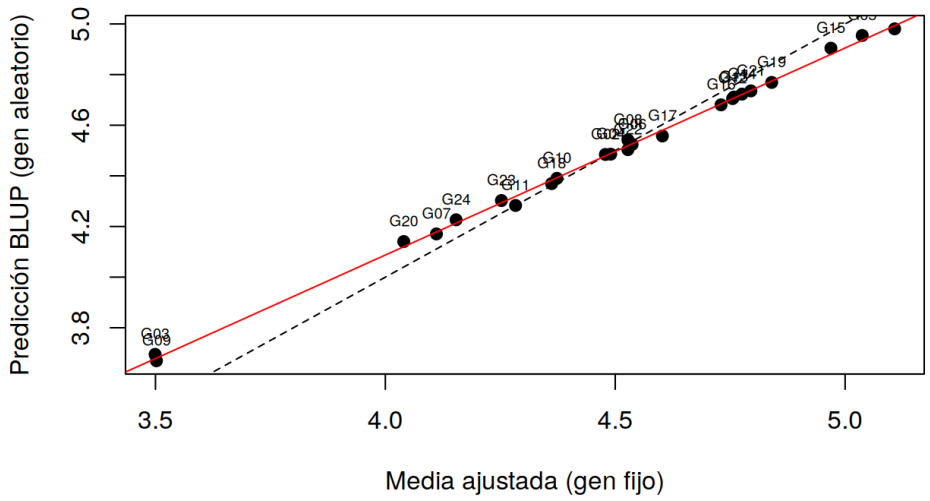
sg2 <- fit_r$theta[1]
H2_cullis <- 1 - pr$avsed^2 / (2 * sg2)
H2_cullis

## gen!sigma2
##     0.8092
```

4.4 Comparación de medias ajustadas y BLUP

La figura siguiente compara las medias ajustadas (variedades fijas) con los BLUP (variedades aleatorias). La pendiente menor que uno es la contracción del BLUP.

```
plot(pf$pvals$predicted.value, pr$pvals$predicted.value, pch = 19,
     xlab = "Media ajustada (gen fijo)", ylab = "Predicción BLUP (gen
aleatorio)")
abline(0, 1, lty = 2)
abline(lm(pr$pvals$predicted.value ~ pf$pvals$predicted.value), col = "red")
text(pf$pvals$predicted.value, pr$pvals$predicted.value, pf$pvals$gen, pos = 3,
cex = 0.6)
```



Medias ajustadas (variedades fijas) frente a predicciones BLUP (variedades aleatorias) en el ensayo de avena.

El ordenamiento de las variedades es casi idéntico, pero los BLUP están más cerca de la media. En diseños desbalanceados el BLUP puede cambiar también el orden, porque las variedades con menos información se contraen más.

4.5 ¿Vale la pena el diseño de bloques incompletos?

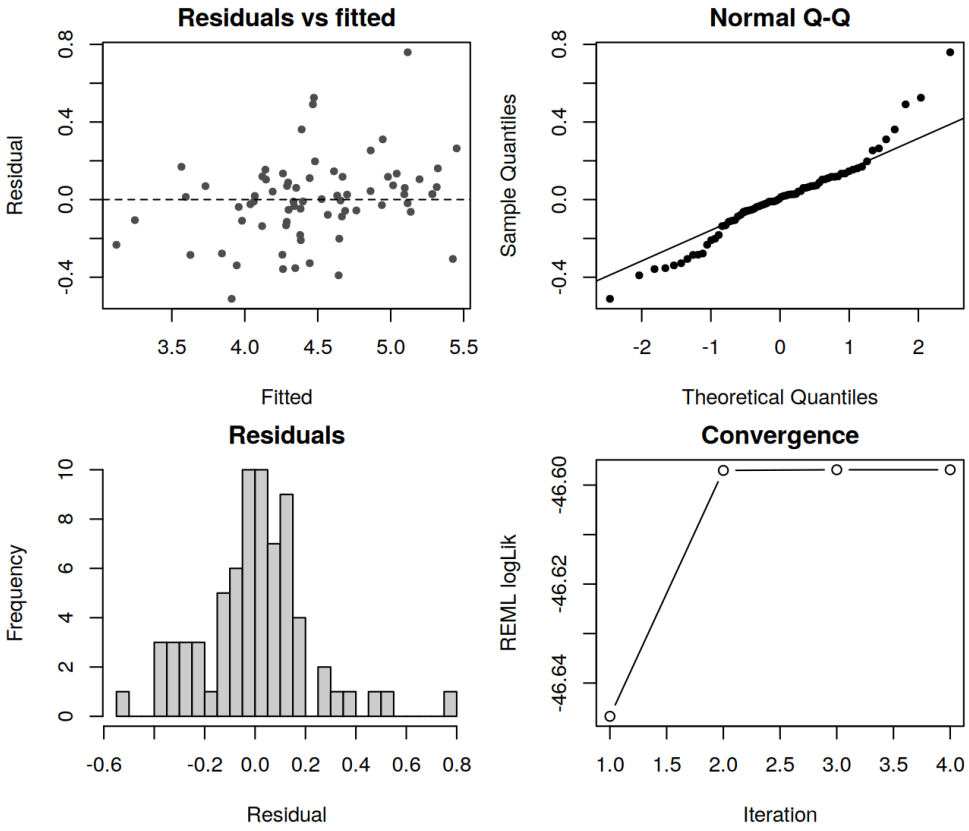
Para saber si los bloques incompletos capturan variación, se compara el modelo con y sin `rep:block` mediante la razón de verosimilitudes. Ambos modelos tienen los mismos efectos fijos, así que sus verosimilitudes REML son comparables, y como se prueba una varianza en la frontera se usa la mezcla 50:50.

```
fit_r0 <- mlreml(yield ~ rep, random = ~ gen, data = d, trace = FALSE)
lrt(fit_r0, fit_r)
```

```
##   df LR.statistic  p.value  AIC0  AIC1  BIC0  BIC1
## 1  1        8.606 0.001675 105.8 99.19 110.3 105.9
```

4.6 Diagnóstico

```
plot(fit_r)
```



Gráficos de diagnóstico del modelo con variedades aleatorias.

Los residuos condicionales no muestran tendencia con los valores ajustados ni desviaciones importantes de la normalidad. El panel inferior derecho muestra la convergencia del algoritmo AI, que en este modelo sencillo requiere pocas iteraciones.

4.7 Verificación con lme4

```
library(lme4)
l <- lmer(yield ~ rep + (1 | gen) + (1 | rep:block), data = d)
data.frame(MLREML = unname(fit_r$theta),
           lme4 = as.data.frame(VarCorr(l))$vcov,
           row.names = c("gen", "rep:block", "residual"))
```

```
##              MLREML    lme4
## gen         0.14290 0.14290
## rep:block 0.07022 0.07022
## residual   0.08162 0.08162
```

```
c(MLREML = fit_r$loglik, lme4 = as.numeric(logLik(l)))
```

```
## MLREML   lme4
##  -46.6  -46.6
```

Los dos programas coinciden hasta el sexto decimal en la verosimilitud.

5 Ejemplo 2: análisis espacial de un ensayo de trigo

5.1 Los datos

Los datos `gilmour.serpentine` provienen del artículo de Gilmour, Cullis y Verbyla (1997), que estableció el análisis espacial con estructuras autorregresivas separables como práctica estándar en los ensayos de cultivos de Australia. Son 108 variedades de trigo en un diseño de bloques completos al azar con 3 repeticiones, en una cuadrícula de 22 filas por 15 columnas (330 parcelas). La respuesta es el rendimiento.

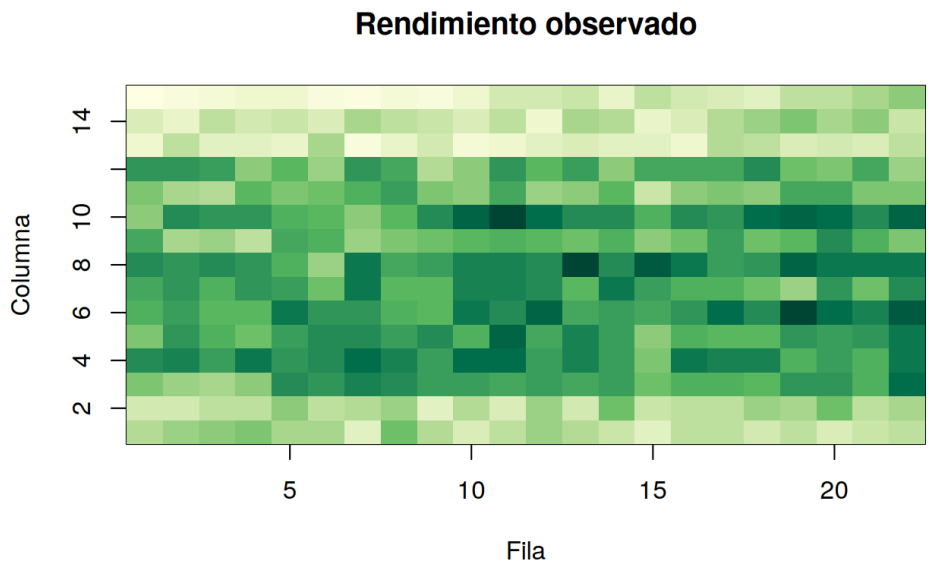
```
data(gilmour.serpentine)
d <- gilmour.serpentine
str(d)
```

```
## 'data.frame':    330 obs. of  5 variables:
##  $ col  : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ row  : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep  : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ gen  : Factor w/ 107 levels "(WWH*MM)*WR*",..: 4 10 15 17 21 32 33 34 72
74 ...
##  $ yield: int  483 526 557 564 498 510 344 600 466 370 ...
```

```
c(filas = length(unique(d$row)), columnas = length(unique(d$col)))
```

```
##    filas columnas
##       22       15
```

```
Ym <- tapply(d$yield, list(d$row, d$col), mean)
image(1:22, 1:15, Ym, col = hcl.colors(30, "YlGn", rev = TRUE),
      xlab = "Fila", ylab = "Columna", main = "Rendimiento observado")
```



Rendimiento observado en la cuadrícula del ensayo (filas × columnas). Los colores más oscuros indican mayor rendimiento.

La figura anterior ya sugiere un gradiente de fertilidad a lo largo de las filas y variación local que las repeticiones completas no pueden absorber.

5.2 Modelo de bloques completos

```
m1 <- mlreml(yield ~ rep, random = ~ gen, data = d, trace = FALSE)
summary(m1)$varcomp
```

```
##                 estimate std.error z.ratio bound
## gen!sigma2          1934     947.3   2.042     P
## R!units!sigma2     13329    1263.6  10.548     P
```

5.3 Modelo espacial AR1 × AR1

Siguiendo a Gilmour et al. (1997), el residuo se modela como un proceso autorregresivo separable en filas y columnas más un error independiente (*nugget*), que en MLREML es el término aleatorio `units`:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}_g\mathbf{g} + \boldsymbol{\eta} + \boldsymbol{\xi}, \qquad \boldsymbol{\xi} \sim N\left(\mathbf{0}, \sigma^2_{\xi}\,\boldsymbol{\Sigma}_r(\rho_r) \otimes \boldsymbol{\Sigma}_c(\rho_c)\right), \qquad \boldsymbol{\eta} \sim N(\mathbf{0}, \sigma^2_{\eta}\mathbf{I}).$$

```
m2 <- mlreml(yield ~ rep, random = ~ gen + units,
             residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
summary(m2)
```

```
## MLREML fit (AI-REML)
## Observations: 330   Fixed-effect columns: 3   Iterations: 16 (converged)
## REML log-likelihood: -1853.6433   AIC: 3717.29   BIC: 3736.24
##
## Variance parameters:
##                               estimate std.error z.ratio bound
## gen!sigma2                   2.583e+03 4.764e+02    5.42     P
## units!sigma2                 1.702e+03 2.356e+02    7.22     P
## R!ar1(row):ar1(col)!sigma2   3.954e+04 2.093e+04    1.89     P
## R!ar1(row):ar1(col)!row_cor 9.903e-01 5.309e-03  186.55     P
## R!ar1(row):ar1(col)!col_cor 8.202e-01 6.620e-02   12.39     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

Ambos modelos tienen los mismos efectos fijos, de modo que pueden compararse con el AIC y el BIC:

```
data.frame(modelo = c("bloques completos", "AR1 x AR1 + nugget"),
           logLik = c(m1$loglik, m2$loglik), AIC = c(m1$AIC, m2$AIC), BIC =
c(m1$BIC, m2$BIC))
```

```
##               modelo logLik  AIC  BIC
## 1  bloques completos   -2043 4091 4099
## 2 AR1 x AR1 + nugget   -1854 3717 3736
```

La correlación entre filas vecinas es muy alta, lo que indica una tendencia suave a lo largo de las filas, mientras que la correlación entre columnas es menor. La mejora del AIC es grande.

5.4 Tendencia global

Gilmour et al. (1997) recomiendan añadir una tendencia lineal como efecto fijo cuando el variograma muestra que la semivarianza crece sin estabilizarse. La covariable `row` numérica se incluye en la parte fija y se prueba con Wald:

```
m3 <- mlreml(yield ~ rep + row, random = ~ gen + units,
             residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
summary(m3)$varcomp
```

```
##                               estimate std.error z.ratio bound
## gen!sigma2                   2.581e+03 4.761e+02   5.421     P
## units!sigma2                 1.697e+03 2.351e+02   7.218     P
## R!ar1(row):ar1(col)!sigma2   4.194e+04 2.267e+04   1.850     P
## R!ar1(row):ar1(col)!row_cor 9.905e-01 5.225e-03 189.573     P
## R!ar1(row):ar1(col)!col_cor 8.303e-01 6.455e-02  12.863     P
```

```
wald(m3)
```

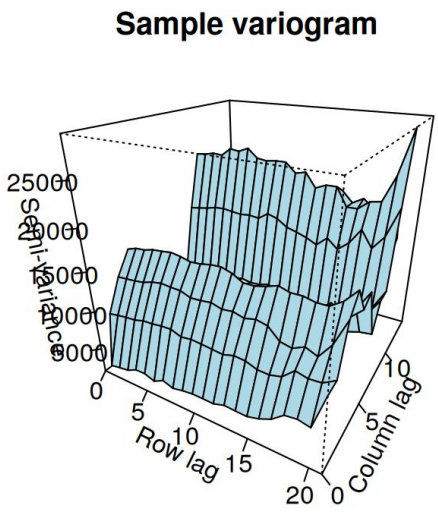
```
##          term df wald.inc   F.inc ddf.inc   p.inc wald.con   F.con ddf.con
p.con
## 1 (Intercept)  1  16.7395 20.0048   3.443 0.01547  16.7395 20.0048   3.443
0.01547
## 2         rep  2   1.8132  0.7871  11.472 0.47829   1.8166  0.7886  11.472
0.47766
## 3         row  1   0.6921  0.5961   9.759 0.45838   0.6921  0.5961   9.759
0.45838
##     p.chisq
## 1 4.288e-05
## 2 4.032e-01
## 3 4.055e-01
```

En este ensayo la tendencia lineal no es significativa (p = 0.46 con el método de Kenward–Roger, que da solo 9.8 grados de libertad en el denominador para esta prueba): la autorregresión entre filas, con una correlación cercana a uno, ya representa el gradiente. Los modelos `m2` y `m3` tienen efectos fijos distintos, de modo que sus verosimilitudes REML no son comparables y la decisión se basa en la prueba de Wald. Conservamos el término lineal en lo que sigue solo para ilustrar su uso; los resultados con `m2` son prácticamente iguales.

5.5 Variograma muestral

El variograma de los residuos espaciales es la herramienta de diagnóstico más útil. Para el modelo de bloques completos se calcula con los residuos condicionales; un variograma que crece con la distancia indica correlación espacial no modelada.

```
v1 <- variogram(m1, "row", "col")
```



Variograma muestral de los residuos del modelo de bloques completos.

```
head(v1[order(v1$row.lag, v1$col.lag), ], 6)
```

```
##     row.lag col.lag gamma npairs
## 22        0       1  8657    308
## 44        0       2 12548    286
## 66        0       3 15256    264
## 88        0       4 10720    242
## 110       0       5 11402    220
## 132       0       6  9477    198
```

La semivarianza del modelo de bloques completos crece con el desfase en filas y en columnas y no se estabiliza dentro del ensayo; esa es la correlación espacial que el modelo AR1 × AR1 representa.

5.6 Ganancia en precisión

La consecuencia práctica de un mejor modelo es una comparación más precisa de las variedades. Para comparar las precisiones se ajusta el modelo con variedades fijas y se calcula el SED promedio:

```
m1f <- mlreml(yield ~ rep + gen, data = d, trace = FALSE)
m3f <- mlreml(yield ~ rep + row + gen, random = ~ units,
              residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
c(SED_bloques = predict(m1f, "gen")$avsed, SED_espacial = predict(m3f,
"gen")$avsed)
```

```
##  SED_bloques SED_espacial
##        93.66        38.81
```

El análisis espacial reduce el error de las comparaciones, lo que equivale a evaluar las variedades con más repeticiones sin aumentar el costo del ensayo. La heredabilidad generalizada aumenta en la misma proporción:

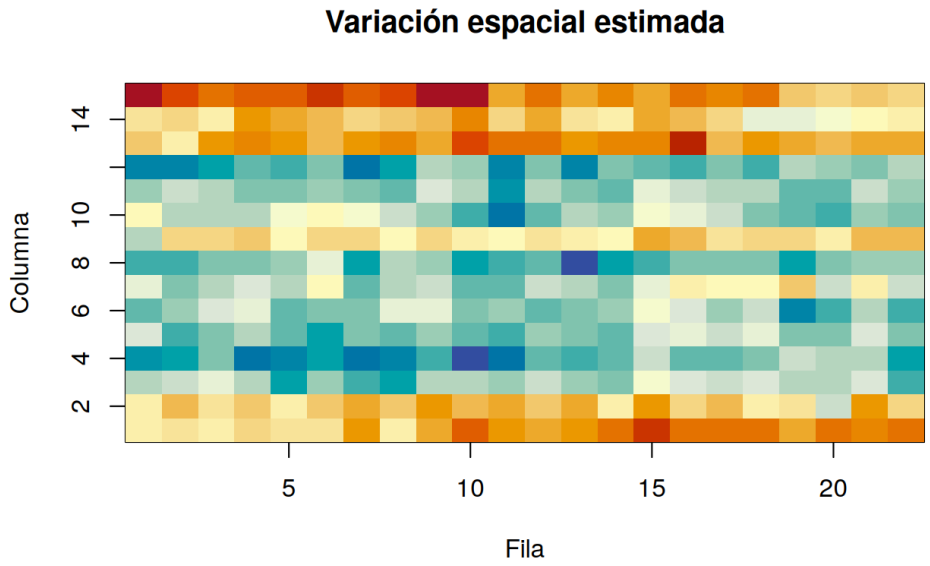
```
H2 <- function(fit) 1 - predict(fit, "gen")$avsed^2 / (2 * fit$theta[1])
c(bloques = H2(m1), espacial = H2(m3))
```

```
##  bloques.gen!sigma2 espacial.gen!sigma2
##              0.3080              0.7698
```

5.7 Tendencia espacial estimada

La componente espacial predicha por el modelo es la diferencia entre los valores ajustados y la parte fija más los efectos genéticos. La figura siguiente la muestra en la cuadrícula.

```
gb <- coef(m3, "random", "gen")
geff <- gb$blup[match(as.character(m3$data$gen), gb$level)]
espacial <- m3$y - m3$fixed_fitted - geff
Em <- tapply(espacial, list(m3$data$row, m3$data$col), mean)
image(1:22, 1:15, Em, col = hcl.colors(30, "RdYlBu"), xlab = "Fila", ylab =
"Columna",
      main = "Variación espacial estimada")
```



Residuos del modelo con efectos genéticos y fijos únicamente (estimación del componente espacial).

5.8 Datos espaciales irregulares

Cuando las unidades no están en una cuadrícula regular (árboles en un rodal, parcelas georreferenciadas con GPS, plantas faltantes), la estructura `sexp(x, y)` define una correlación exponencial isotrópica $\rho^{d_{ij}}$ con la distancia euclidiana real. Como ejemplo se usan los datos de volumen de 1,600 abetos de Douglas (Johnson, 1998, en `agridat`), de los que se toma una muestra aleatoria de 450 árboles para obtener un patrón espacial irregular.

```
data(johnson.douglasfir)
set.seed(11)
df <- johnson.douglasfir[sample(nrow(johnson.douglasfir), 450), ]
df$lv <- log(df$volume + 1)
f_iid <- mlreml(lv ~ 1, data = df, trace = FALSE)
f_exp <- mlreml(lv ~ 1, random = ~ units, residual = ~ sexp(row, col), data =
df, trace = FALSE)
summary(f_exp)$varcomp
```

```
##                                     estimate std.error z.ratio bound
## units!sigma2                          5.8841    2.4044  2.4473     P
## R!sexp(row, col)!sigma2               0.8822    2.3803  0.3706     P
## R!sexp(row, col)!sexp(row,col)_cor    0.3581    0.6771  0.5289     P
```

```
lrt(f_iid, f_exp, boundary = FALSE)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  2        1.031  0.5971 2141 2144 2145 2156
```

La correlación por unidad de distancia y la mejora de la verosimilitud indican que árboles cercanos tienen volúmenes parecidos, por competencia y por micrositio. La correlación entre dos árboles a distancia d es $\hat{\rho}^d$; por ejemplo, a distancias de 1, 3 y 5 posiciones:

```
rho <- f_exp$theta[grep("_cor$", names(f_exp$theta))]
unname(rho^c(1, 3, 5))
```

```
## [1] 0.358088 0.045917 0.005888
```

6 Ejemplo 3: ensayos multiambientales y modelos factor analíticos

6.1 Los datos

El conjunto `besag.met` (Besag y Higdon, 1999; en `agridat`) contiene el rendimiento de 64 variedades de maíz evaluadas en 6 localidades (*counties*), cada una con 3 repeticiones divididas en 8 bloques incompletos de 8 parcelas: 1,152 parcelas en total. Es un ensayo multiambiental (MET) típico de un programa de mejoramiento.

```
data(besag.met)
d <- besag.met
str(d)
```

```
## 'data.frame':    1188 obs. of  7 variables:
##  $ county: Factor w/ 6 levels "C1","C2","C3",..: 1 1 1 1 1 1 1 1 1 ...
##  $ row   : int  1 1 1 1 1 1 1 1 1 ...
##  $ col   : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep   : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 ...
##  $ block : Factor w/ 8 levels "B1","B2","B3",..: 2 2 2 2 2 2 2 5 5 ...
##  $ yield : num  170 130 129 131 132 ...
##  $ gen   : Factor w/ 64 levels "G01","G02","G03",..: 42 58 26 10 34 50 2 18
45 61 ...
```

```
round(tapply(d$yield, d$county, mean), 1)
```

```
## C1 C2 C3 C4 C5 C6
## NA NA NA NA NA NA
```

```
round(tapply(d$yield, d$county, sd), 1)
```

```
## C1 C2 C3 C4 C5 C6
## NA NA NA NA NA NA
```

Las medias y las desviaciones estándar difieren mucho entre localidades, lo que anticipa varianzas genéticas y residuales heterogéneas.

6.2 La sucesión de modelos

El análisis MET de referencia (Smith, Cullis y Thompson, 2001; 2005) modela los efectos genéticos en cada ambiente, $\mathbf{u} = (\mathbf{u}_1', \ldots, \mathbf{u}_E')'$, con

$$\mathrm{Var}(\mathbf{u}) = \boldsymbol{\Sigma}_E \otimes \mathbf{I}_N,$$

y compara estructuras para la matriz $E \times E$ de covarianzas genéticas entre ambientes $\boldsymbol{\Sigma}_E$. Todas las estructuras comparten los efectos fijos (medias de localidad), el diseño (repeticiones y bloques dentro de localidad) y una varianza residual propia de cada localidad.

Estructuras para la covarianza genética entre ambientes.

Modelo	$\boldsymbol{\Sigma}_E$	Parámetros genéticos
Diagonal	$\mathrm{diag}(\sigma^2_{g1}, \ldots, \sigma^2_{gE})$	E
FA(1)	$\boldsymbol{\lambda}\boldsymbol{\lambda}' + \boldsymbol{\Psi}$	$2E$
FA(2)	$\boldsymbol{\Lambda}\boldsymbol{\Lambda}' + \boldsymbol{\Psi}$	$3E - 1$
No estructurada	completa	$E(E+1)/2$

El modelo diagonal supone que no hay correlación genética entre ambientes, lo que en la práctica nunca es cierto. El no estructurado es el más general, pero con muchos ambientes tiene demasiados parámetros y suele ser difícil de estimar. Los modelos factor analíticos son el punto intermedio: con pocos factores describen la mayor parte de la covarianza genética y permiten interpretar la interacción genotipo × ambiente.

```
f_diag <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
diag(county):id(gen),
                 residual = ~ diag(county):units, data = d, trace = FALSE)
f_fa1 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
                residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 100)
f_fa2 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 2):id(gen),
                residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 200)
f_us <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
us(county):id(gen),
               residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 100)
cmp <- data.frame(modelo = c("diag", "FA(1)", "FA(2)", "US"),
                  parametros = sapply(list(f_diag, f_fa1, f_fa2, f_us),
function(f) sum(f$free)),
                  logLik = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`,
"loglik"),
                  AIC = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`, "AIC"),
                  BIC = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`, "BIC"),
                  convergio = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`,
"converged"))
cmp
```

```
##   modelo parametros logLik  AIC  BIC convergio
## 1  diag         14  -4849 9726 9796      TRUE
## 2 FA(1)         20  -4814 9668 9769      TRUE
```

```
## 3 FA(2)         25  -4812 9675 9801      TRUE
## 4    US         29  -4812 9681 9827      TRUE
```

El modelo diagonal es claramente el peor: ignorar la correlación genética entre localidades le cuesta 34.9 unidades de log-verosimilitud frente al FA(1), con solo 6 parámetros adicionales. Según el AIC, el mejor modelo es FA(1). La verosimilitud aumenta del FA(1) al FA(2) y al no estructurado, como debe ocurrir porque los modelos están anidados, pero los incrementos son pequeños para el número de parámetros añadidos. La verosimilitud es muy plana en la dirección del segundo factor, una situación común en análisis MET que también se presenta en ASReml. Varias varianzas específicas del modelo FA(2) están en la frontera; gracias al tratamiento de la frontera por conjunto activo (Sección 2.4.1), no detienen el paso AI conjunto y el modelo converge. La log-verosimilitud del FA(2) está entre la del FA(1) y la del modelo no estructurado y no cambia la elección del modelo.

El LRT entre FA(1) y el modelo no estructurado (ambos con los mismos efectos fijos) es:

```
lrt(f_fa1, f_us, boundary = FALSE)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  9        4.712  0.8587 9668 9681 9769 9827
```

6.3 Interpretación del modelo FA(1)

`vc_matrix()` devuelve la matriz de covarianza genética estimada, su matriz de correlación, las cargas, las varianzas específicas y el porcentaje de la varianza genética de cada ambiente explicado por los factores.

```
vfa <- vc_matrix(f_fa1, "fa(county, 1):id(gen)")
round(vfa$cor, 2)
```

```
##      C1   C2   C3   C4   C5   C6
## C1 1.00 0.86 0.68 0.86 0.86 0.40
## C2 0.86 1.00 0.79 1.00 1.00 0.46
## C3 0.68 0.79 1.00 0.79 0.79 0.36
## C4 0.86 1.00 0.79 1.00 1.00 0.46
## C5 0.86 1.00 0.79 1.00 1.00 0.46
## C6 0.40 0.46 0.36 0.46 0.46 1.00
```

```
data.frame(carga = round(vfa$loadings[, 1], 2), psi = signif(vfa$psi, 3),
           pct_explicado = round(vfa$pct_variance, 1))
```

```
##    carga      psi pct_explicado
## C1    6.49 1.46e+01          74.3
## C2    5.15 8.44e-07         100.0
## C3    7.57 3.47e+01          62.3
## C4    5.34 7.30e-07         100.0
## C5    8.90 6.70e-07         100.0
## C6    2.35 2.05e+01          21.3
```

```
summary(f_fa1)$varcomp[grepl("fa1|var_", rownames(summary(f_fa1)$varcomp)), ]
```

```
##                                       estimate std.error   z.ratio bound
## fa(county, 1):id(gen)!county_fa1_C1 6.491e+00     1.366 4.752e+00     P
## fa(county, 1):id(gen)!county_fa1_C2 5.149e+00     1.319 3.904e+00     P
## fa(county, 1):id(gen)!county_fa1_C3 7.569e+00     1.567 4.830e+00     P
## fa(county, 1):id(gen)!county_fa1_C4 5.345e+00     1.419 3.765e+00     P
## fa(county, 1):id(gen)!county_fa1_C5 8.898e+00     1.392 6.391e+00     P
```

```
## fa(county, 1):id(gen)!county_fa1_C6 2.355e+00     1.710 1.377e+00     P
## fa(county, 1):id(gen)!county_var_C1 1.458e+01    16.469 8.855e-01     P
## fa(county, 1):id(gen)!county_var_C2 8.438e-07    16.874 5.001e-08     B
## fa(county, 1):id(gen)!county_var_C3 3.472e+01    21.181 1.639e+00     P
## fa(county, 1):id(gen)!county_var_C4 7.300e-07    19.599 3.725e-08     B
## fa(county, 1):id(gen)!county_var_C5 6.700e-07    16.851 3.976e-08     B
## fa(county, 1):id(gen)!county_var_C6 2.053e+01    29.724 6.907e-01     P
```

Varias varianzas específicas ψ están en la frontera (código `B`). En esas localidades el factor común explica toda la varianza genética, y la correlación genética estimada entre ellas es uno. Este resultado es frecuente en ensayos con pocas repeticiones y no indica un error del ajuste; indica que los datos no permiten distinguir la varianza específica de cero. Los errores estándar de los parámetros en la frontera no son válidos.

```
vus <- vc_matrix(f_us, "us(county):id(gen)")
op <- par(mfrow = c(1, 2), mar = c(3, 3, 2, 1))
for (M in list(US = vus$cor, FA1 = vfa$cor)) {
  image(1:6, 1:6, M[, 6:1], zlim = c(-1, 1), col = hcl.colors(21, "RdBu"),
        axes = FALSE, xlab = "", ylab = "")
  axis(1, 1:6, rownames(M), cex.axis = 0.7); axis(2, 1:6, rev(rownames(M)),
cex.axis = 0.7, las = 1)
  text(rep(1:6, 6), rep(6:1, each = 6), sprintf("%.2f", M), cex = 0.6)
}
```



Correlaciones genéticas entre localidades estimadas con el modelo no estructurado (izquierda) y con FA(1) (derecha).

```
par(op)
```

6.4 Predicciones por ambiente y desempeño general

Las predicciones de cada variedad en cada localidad combinan la media fija de la localidad con el BLUP de la variedad en esa localidad. La clasificación `county:gen` incluye el término factor analítico porque sus dos factores están en la clasificación:

```
pr <- predict(f_fa1, classify = "county:gen", sed = FALSE)
P <- with(pr$pvals, tapply(predicted.value, list(gen, county), mean))
round(head(P[order(-rowMeans(P)), ], 10), 1)

##        C1   C2    C3    C4    C5   C6
## G05 160.6 64.8 100.0 153.7 130.6 92.7
```

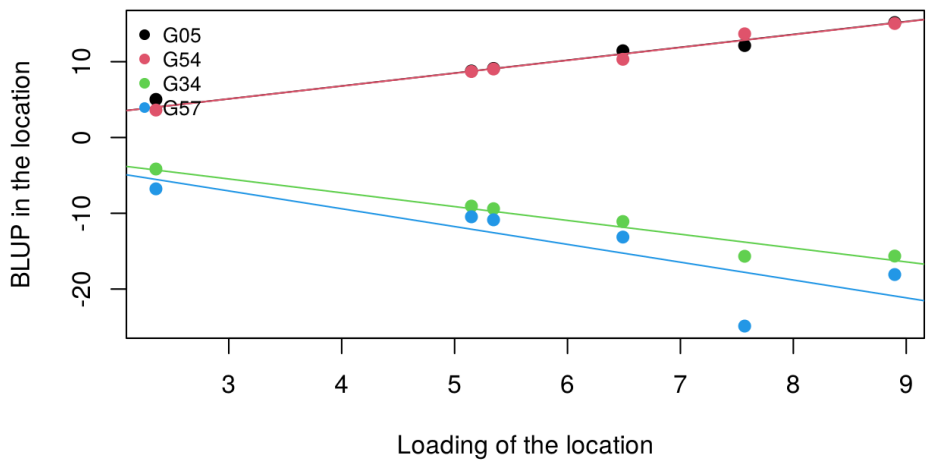
```
## G54 159.5 64.8 101.6 153.7 130.5 91.3
## G45 157.4 63.1 100.1 151.9 127.5 89.8
## G55 158.7 62.6 101.1 151.4 126.8 88.8
## G61 155.7 61.4 101.0 150.1 124.6 91.6
## G32 157.1 61.8 100.2 150.6 125.4 88.5
## G51 157.3 62.0  96.1 150.8 125.7 91.2
## G30 155.4 60.9  98.0 149.7 123.8 89.1
## G22 155.0 61.2  94.0 149.9 124.2 91.0
## G52 155.8 60.6  96.1 149.4 123.3 89.8
```

Una medida natural del desempeño general en un modelo FA(1) con cargas positivas es el *score* del factor de cada variedad multiplicado por la carga media (Smith y Cullis, 2018). Como $\mathbf{u}_i = \boldsymbol{\lambda} f_i + \boldsymbol{\delta}_i$, el score se puede estimar por mínimos cuadrados a partir de los BLUP de cada variedad:

```
U <- matrix(coef(f_fa1, "random", "fa(county, 1):id(gen)")$blup, ncol = 6,
            dimnames = list(levels(d$gen), levels(d$county)))
lam <- vfa$loadings[, 1]
score <- drop(U %*% lam / sum(lam^2))
OP <- mean(lam) * score
head(sort(OP, decreasing = TRUE), 10)

##    G05    G54    G55    G45    G32    G61    G51    G30    G09    G52
## 10.149 10.131  8.381  8.330  7.407  7.220  6.954  6.159  5.860  5.709

top <- names(sort(OP, decreasing = TRUE))[c(1, 2, 63, 64)]
plot(range(lam), range(U[top, ]), type = "n", xlab = "Carga de la localidad",
     ylab = "BLUP en la localidad")
for (k in seq_along(top)) {
  points(lam, U[top[k], ], pch = 19, col = k)
  abline(0, score[top[k]], col = k)
}
legend("topleft", top, col = seq_along(top), pch = 19, bty = "n", cex = 0.8)
```



Gráfica de regresión latente: BLUP de cuatro variedades en cada localidad frente a la carga de la localidad. Las líneas tienen pendiente igual al score de cada variedad.

Las variedades con pendiente positiva grande son superiores en las localidades con carga alta; la dispersión de los puntos alrededor de la recta es la parte de la interacción que el factor común no explica.

6.5 Recomendaciones para análisis MET

1. Empezar con varianzas residuales y genéticas heterogéneas (`diag`) para verificar que el modelo base se ajusta.
2. Ajustar FA(1) y aumentar el número de factores mientras el AIC o el LRT lo justifiquen y el porcentaje de varianza genética explicado sea bajo en algunos ambientes.
3. Usar `us` solo con pocos ambientes; con más de 8–10 ambientes suele no converger o converger a matrices singulares.
4. Revisar los parámetros en la frontera y aumentar `maxit` cuando el algoritmo no converge; un FA sin converger en la verosimilitud puede tener aún predicciones estables, lo que se verifica comparando los BLUP entre iteraciones.

7 Ejemplo 4: predicción genómica (GBLUP) con 599 líneas de trigo

7.1 Los datos

El conjunto `wheat` del paquete `BGLR` (Crossa et al., 2010; Pérez y de los Campos, 2014) contiene 599 líneas de trigo del CIMMYT genotipadas con 1,279 marcadores DArT (codificados 0/1), su matriz de parentesco por pedigree y el rendimiento de grano evaluado en cuatro ambientes (E1 a E4). Es uno de los conjuntos de datos más usados para comparar métodos de selección genómica.

```
library(BGLR)
data(wheat)
ids <- paste0("L", 1:599)
M <- wheat.X; rownames(M) <- ids
A <- wheat.A; dimnames(A) <- list(ids, ids)
Y <- wheat.Y; rownames(Y) <- ids
dim(M); table(M)
```

```
## [1]  599 1279
```

```
## M
##      0      1
## 336588 429533
```

```
round(cor(Y), 2)
```

```
##       1     2     4     5
## 1  1.00 -0.02 -0.19 -0.12
## 2 -0.02  1.00  0.66  0.41
## 4 -0.19  0.66  1.00  0.39
## 5 -0.12  0.41  0.39  1.00
```

7.2 Control de calidad y matriz genómica

Los marcadores DArT son dominantes y están codificados 0/1. Para construir **G** los recodificamos como 0/2 (las líneas son endogámicas, de modo que cada locus es homocigoto) y aplicamos el control de calidad:

```r
qc <- snp_qc(2 * M, maf = 0.05, callrate = 0.9)
qc$report
```

```
##   markers_in removed_callrate removed_maf markers_out individuals_in
individuals_out
## 1       1279                0          96        1183            599
599
```

```r
G <- Gmatrix(qc$M, method = "VanRaden")
summary(diag(G))
```

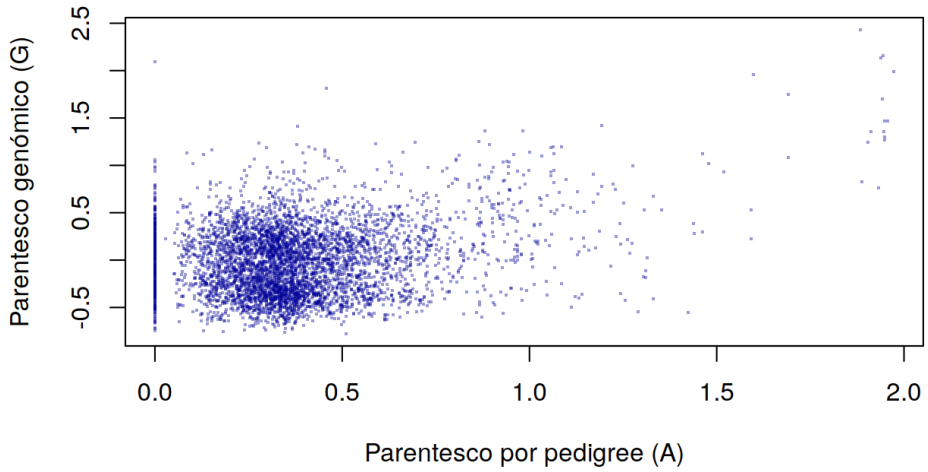
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.34    1.77    2.00    2.00    2.23    2.88
```

```r
summary(diag(A))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.56    1.98    1.99    1.98    2.00    2.00
```

La diagonal de **A** está cerca de 2, como corresponde a líneas endogámicas ($1 + F$ con $F \approx 1$). La diagonal de **G** también tiene media 2, pero varía mucho más entre líneas (de 1.3 a 2.9): mide la endogamia realizada respecto a la población genotipada, mientras que el pedigree solo da su valor esperado. Las diferencias de escala entre matrices no afectan las predicciones, porque se absorben en la varianza genética, pero sí la interpretación de σ_g^2. La relación entre ambas se aprecia en la figura:

```r
up <- upper.tri(A)
set.seed(1); s <- sample(which(up), 5000)
plot(A[s], G[s], pch = ".", cex = 2, col = rgb(0, 0, 0.6, 0.4),
     xlab = "Parentesco por pedigree (A)", ylab = "Parentesco genómico (G)")
```



Parentesco por pedigree frente a parentesco genómico (elementos fuera de la diagonal).

```r
cor(A[up], G[up])
```

```
## [1] 0.2469
```

La correlación moderada refleja que el pedigree predice el parentesco esperado mientras que los marcadores miden el parentesco realizado, que varía alrededor del esperado por la segregación mendeliana.

7.3 GBLUP en un ambiente

El modelo GBLUP para el ambiente E1 es $\mathbf{y} = \mathbf{1}\mu + \mathbf{g} + \mathbf{e}$, $\mathbf{g} \sim N\left(\mathbf{0}, \sigma_g^2 \mathbf{G}\right)$. Se ajusta con el motor general y con la función rápida `gblup()`, que usa la descomposición espectral de $\mathbf{G}$ y reduce REML a una optimización unidimensional.

```
G <- G + diag(1e-4, nrow(G))      # pequeña regularización numérica
d1 <- data.frame(gen = ids, y = Y[, 1])
t_general <- system.time(f1 <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1,
trace = FALSE))
y1 <- setNames(Y[, 1], ids)
t_rapido <- system.time(g1 <- gblup(y1, G))
summary(f1)$varcomp
```

```
##                    estimate std.error z.ratio bound
## vm(gen, G)!sigma2   0.2942   0.05462   5.387     P
## R!units!sigma2      0.5551   0.04607  12.049     P
```

```
g1$varcomp
```

```
## sigma2_g sigma2_e       h2
##   0.2942   0.5551   0.3464
```

```
c(general = t_general[3], espectral = t_rapido[3])
```

```
##   general.elapsed espectral.elapsed
##             0.231             0.061
```

```
c(logLik_general = f1$loglik, logLik_espectral = g1$loglik)
```

```
##   logLik_general logLik_espectral
##           -794.5           -794.5
```

```
cor(f1$blups[[1]]$blup, g1$gebv[f1$blups[[1]]$level])
```

```
## [1] 1
```

Ambas implementaciones dan la misma verosimilitud y los mismos GEBV. La heredabilidad genómica estimada es 0.35. La confiabilidad de los GEBV de las líneas fenotipadas se obtiene directamente:

```
summary(f1$blups[[1]]$reliability)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.603   0.717   0.754   0.763   0.809   0.964
```

7.4 Predicción de líneas no fenotipadas

La utilidad de la selección genómica es predecir candidatos sin fenotipo. Si se elimina el fenotipo de 120 líneas, `vm()` las predice porque están en $\mathbf{G}$; basta con quitar sus registros de los datos:

```
set.seed(2024)
test <- sample(ids, 120)
f_tr <- mlreml(y ~ 1, random = ~ vm(gen, G), data = subset(d1, !gen %in% test),
```

```
trace = FALSE)
b <- f_tr$blups[[1]]
pred <- b$blup[match(test, b$level)]
c(correlacion = cor(pred, Y[test, 1]),
  confiabilidad_media_test = mean(b$reliability[match(test, b$level)]))

##              correlacion confiabilidad_media_test
##                   0.5457                   0.6374
```

7.5 Validación cruzada: pedigree, genómica y ambos

La función `cv_gblup()` repite particiones aleatorias en k grupos. Se compara la capacidad predictiva del pedigree ($\mathbf{A}$), de los marcadores ($\mathbf{G}$) y de un modelo con ambas matrices, que en MLREML se escribe con dos términos aleatorios sobre el mismo factor.

```
Ar <- A + diag(1e-4, nrow(A))
cvA <- cv_gblup(y1, Ar, folds = 5, reps = 4, seed = 3)
cvG <- cv_gblup(y1, G, folds = 5, reps = 4, seed = 3)
data.frame(matriz = c("A (pedigree)", "G (marcadores)"),
           r_media = c(mean(cvA$cor), mean(cvG$cor)),
           r_de = c(sd(cvA$cor), sd(cvG$cor)))

##           matriz r_media   r_de
## 1   A (pedigree)  0.4157 0.0615
## 2 G (marcadores)  0.4909 0.0551

f_AG <- mlreml(y ~ 1, random = ~ vm(gen, Ar) + vm(gen, G), data = d1, trace =
FALSE)
summary(f_AG)$varcomp

##                     estimate std.error z.ratio bound
## vm(gen, Ar)!sigma2   0.1127   0.04147   2.717     P
## vm(gen, G)!sigma2    0.2351   0.05213   4.510     P
## R!units!sigma2       0.4519   0.05102   8.856     P
```

Con la misma partición en el modelo de dos núcleos:

```
set.seed(3); fold <- sample(rep(1:5, length.out = 599))
r_AG <- sapply(1:5, function(k) {
  tr <- d1[fold != k, ]
  f <- mlreml(y ~ 1, random = ~ vm(gen, Ar) + vm(gen, G), data = tr, trace =
FALSE, pev = FALSE)
  te <- ids[fold == k]
  p <- f$blups[[1]]$blup[match(te, f$blups[[1]]$level)] +
f$blups[[2]]$blup[match(te, f$blups[[2]]$level)]
  cor(p, Y[te, 1])
})
mean(r_AG)

## [1] 0.5161
```

7.6 GBLUP multiambiental

Con los cuatro ambientes se ajusta un modelo MET genómico en el que el efecto genético de cada línea en cada ambiente tiene covarianza $\boldsymbol{\Sigma}_E \otimes \mathbf{G}$ (Burgueño et al., 2012), con varianzas residuales propias de cada ambiente:

```r
dl <- data.frame(gen = rep(ids, 4), env = factor(rep(paste0("E", 1:4), each =
599)), y = as.vector(Y))
t_met <- system.time(
  fm <- mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~
diag(env):units,
               data = dl, trace = FALSE, maxit = 100))
t_met[3]
```

```
## elapsed
##   62.54
```

```r
summary(fm)
```

```
## MLREML fit (AI-REML)
## Observations: 2396   Fixed-effect columns: 4   Iterations: 15 (converged)
## REML log-likelihood: -3054.2139   AIC: 6136.43   BIC: 6217.35
##
## Variance parameters:
##                                 estimate std.error z.ratio bound
## us(env):vm(gen, G)!env_E1:E1  0.29972   0.05433    5.52     P
## us(env):vm(gen, G)!env_E2:E1 -0.07022   0.04270   -1.64     P
## us(env):vm(gen, G)!env_E2:E2  0.57991   0.07460    7.77     P
## us(env):vm(gen, G)!env_E3:E1 -0.10855   0.04109   -2.64     P
## us(env):vm(gen, G)!env_E3:E2  0.52893   0.06206    8.52     P
## us(env):vm(gen, G)!env_E3:E3  0.49306   0.06739    7.32     P
## us(env):vm(gen, G)!env_E4:E1 -0.12528   0.03693   -3.39     P
## us(env):vm(gen, G)!env_E4:E2  0.34978   0.04934    7.09     P
## us(env):vm(gen, G)!env_E4:E3  0.35240   0.04828    7.30     P
## us(env):vm(gen, G)!env_E4:E4  0.34639   0.05632    6.15     P
## R!diag(env):units!env_E1      0.55463   0.04568   12.14     P
## R!diag(env):units!env_E2      0.41498   0.03492   11.88     P
## R!diag(env):units!env_E3      0.50242   0.03863   13.01     P
## R!diag(env):units!env_E4      0.56067   0.04425   12.67     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```r
round(vc_matrix(fm, "us(env):vm(gen, G)")$cor, 2)
```

```
##       E1    E2    E3    E4
## E1  1.00 -0.17 -0.28 -0.39
## E2 -0.17  1.00  0.99  0.78
## E3 -0.28  0.99  1.00  0.85
## E4 -0.39  0.78  0.85  1.00
```

Las correlaciones genéticas entre ambientes son más extremas que las fenotípicas: E2, E3 y E4 están fuertemente correlacionados entre sí, mientras que E1 tiene correlación genética negativa con los demás. Esta estructura es la que el modelo MET aprovecha para predecir una línea en un ambiente con sus datos de los otros ambientes (esquema CV2 de Burgueño et al., 2012).

```r
set.seed(5)
oculto <- sample(ids, 120)
dtr <- dl[!(dl$env == "E1" & dl$gen %in% oculto), ]
fm_tr <- mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~
diag(env):units,
                data = dtr, trace = FALSE, maxit = 100, pev = FALSE)
b <- fm_tr$blups[[1]]
```

```r
p_met <- b$blup[match(paste0("E1:", oculto), b$level)]
y_e1 <- y1; y_e1[oculto] <- NA
p_uni <- gblup(y_e1, G)$gebv[oculto]
c(univariado = cor(p_uni, Y[oculto, 1]), multiambiental = cor(p_met, Y[oculto,
1]))
```

```
##     univariado multiambiental
##         0.3651         0.3438
```

En este caso el modelo multiambiental no mejora al univariado (0.344 frente a 0.365). En el esquema CV2 el modelo MET aprovecha los registros de las mismas líneas en los otros ambientes, pero E1 tiene correlaciones genéticas negativas y débiles con E2–E4, de modo que esos registros aportan poca información sobre E1 y el costo de estimar más parámetros no se compensa. Cuando los ambientes están positivamente correlacionados, la ganancia del modelo MET en CV2 suele ser grande (Burgueño et al., 2012); el resultado ilustra que el beneficio depende de la estructura de $\boldsymbol{\Sigma}_E$.

7.7 Notas prácticas

- `gblup()` es cómoda para un solo efecto genético y un residuo homogéneo, y es la base de `cv_gblup()`; en 599 líneas tarda una fracción de segundo. Con varios miles de individuos su costo lo domina la descomposición espectral (Capítulo 15).
- `mlreml()` con `vm()` es necesario para modelos con más términos, residuos heterogéneos, varios ambientes o varios caracteres.
- Una matriz $\mathbf{G}$ con más individuos que marcadores es singular; MLREML no la invierte, pero conviene sumar una constante pequeña a la diagonal para estabilidad numérica, como se hizo arriba.

8 Ejemplo 5: modelo animal con pedigree en ganado lechero

8.1 Los datos

El paquete `pedigreemm` (Vazquez et al., 2010) incluye `milk`, con 3,397 registros de lactancia de 1,359 vacas Holstein en 57 hatos, y `pedCowsR`, el pedigree de esas vacas y sus antepasados (6,547 animales). Cada registro tiene la producción de leche, grasa y proteína a 305 días y el número de lactancia (1 a 5).

```r
library(pedigreemm)
data(milk)
milk$animal <- as.character(milk$id)
milk$leche <- milk$milk / 1000     # toneladas
milk$grasa <- milk$fat / 100       # cientos de kg
milk$lact <- factor(milk$lact)
table(table(milk$animal))
```

```
##
##   1   2   3   4   5
## 337 405 314 207  96
```

8.2 Preparación del pedigree

`pedigree_prepare()` recodifica los padres desconocidos, añade como fundadores a los padres que no aparecen como individuos, detecta errores y ordena el pedigree de modo que los padres precedan a su descendencia.

```r
p <- pedCowsR
ped <- data.frame(id = p@label, sire = p@label[p@sire], dam = p@label[p@dam])
pp <- pedigree_prepare(ped)
head(pp)
```

```
##   id sire  dam sire_i dam_i gen
## 1  1 <NA> <NA>      0     0   0
## 2  2 <NA> <NA>      0     0   0
## 3  3 <NA> <NA>      0     0   0
## 4  4 <NA> <NA>      0     0   0
## 5  5 <NA> <NA>      0     0   0
## 6  6 <NA> <NA>      0     0   0
```

```r
table(generacion = pp$gen)
```

```
## generacion
##    0    1    2    3    4    5    6    7
## 2420 1208  777  612  757  562  173   38
```

```r
c(animales = nrow(pp), con_padre = sum(pp$sire_i > 0), con_madre = sum(pp$dam_i
> 0))
```

```
##  animales con_padre con_madre
##      6547      2645      3454
```

```r
t_A <- system.time(A <- Amatrix(pp))
Fi <- inbreeding_coef(pp)
t_A[3]
```

```
## elapsed
##   0.869
```

```r
summary(Fi[Fi > 0])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.00049 0.00781 0.01562 0.03744 0.03125 0.25000
```

```r
sum(Fi > 0)
```

```
## [1] 31
```

```r
all.equal(unname(diag(A) - 1), unname(Fi))
```

```
## [1] TRUE
```

```r
Ai <- Ainverse(pp)
c(dim = nrow(Ai), no_ceros = length(Ai@x), densidad = length(Ai@x) /
nrow(Ai)^2)
```

```
##       dim  no_ceros  densidad
## 6.547e+03 1.460e+04 3.405e-04
```

La matriz **A** densa de 6,547 animales ocupa unos 340 MB. Su inversa, construida directamente con las reglas de Henderson, es muy dispersa. Los coeficientes de consanguinidad calculados por el algoritmo de Meuwissen y Luo coinciden con la diagonal de **A**.

8.3 Modelo animal para la primera lactancia

El modelo animal (Henderson, 1984) para la producción de leche en la primera lactancia es

$$y_{ij} = \mu + h_j + a_i + e_{ij}, \qquad \mathbf{a} \sim N(\mathbf{0}, \sigma_a^2 \mathbf{A}), \qquad \mathbf{h} \sim N(\mathbf{0}, \sigma_h^2 \mathbf{I}),$$

donde h_j es el efecto del hato, aquí aleatorio porque varios hatos tienen pocas vacas.

```
m1 <- subset(milk, lact == 1)
t_m1 <- system.time(
  a1 <- mlreml(leche ~ 1, random = ~ id(herd) + vm(animal, A), data = m1, trace
= FALSE))
t_m1[3]
```

```
## elapsed
##   2.198
```

```
summary(a1)
```

```
## MLREML fit (AI-REML)
## Observations: 1314   Fixed-effect columns: 1   Iterations: 6 (converged)
## REML log-likelihood: -3605.4877   AIC: 7216.98   BIC: 7232.52
##
## Variance parameters:
##                       estimate std.error z.ratio bound
## id(herd)!sigma2         5.6462    1.3650    4.14     P
## vm(animal, A)!sigma2    0.6750    0.8654    0.78     P
## R!units!sigma2         12.3830    0.9616   12.88     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
vpredict(a1, h2 ~ V2 / (V1 + V2 + V3))
```

```
##    estimate std.error
## h2  0.03609   0.04629
```

La heredabilidad estimada es baja y su error estándar grande, porque cada vaca tiene un solo registro y el pedigree relaciona a pocas vacas fenotipadas entre sí. Los valores genéticos (EBV) se predicen para los 6,547 animales del pedigree, incluidos los toros y antepasados sin registros:

```
ebv <- coef(a1, "random", "vm(animal, A)")
nrow(ebv)
```

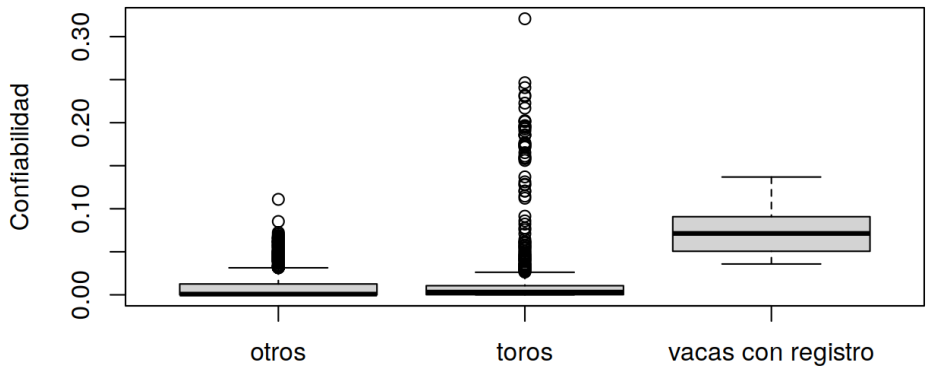
```
## [1] 6547
```

```
toros <- unique(pp$sire[pp$sire_i > 0])
ebv_toros <- ebv[ebv$level %in% toros, ]
head(ebv_toros[order(-ebv_toros$blup), ], 8)
```

```
##      level   blup    pev     se reliability
## 4742  3749 0.7434 0.5434 0.7372     0.19497
## 5020  1630 0.5761 0.5125 0.7159     0.24076
## 4603  2896 0.5031 0.5393 0.7343     0.20114
## 4743  3754 0.4531 0.5426 0.7366     0.19616
## 5069  2833 0.4063 0.5496 0.7413     0.18586
## 3953  2863 0.2970 0.5449 0.7382     0.19282
## 3655  1424 0.2847 0.6334 0.7959     0.06166
## 4432  1679 0.2505 0.5194 0.7207     0.23050
```

```
grupo <- ifelse(ebv$level %in% m1$animal, "vacas con registro",
                ifelse(ebv$level %in% toros, "toros", "otros"))
```

```r
boxplot(reliability ~ grupo, data = cbind(ebv, grupo), ylab = "Confiabilidad",
xlab = "")
```



Confiabilidad de los EBV de las vacas con registro, de los toros y del resto de los animales del pedigree.

Los toros, aunque no tienen registros propios, tienen confiabilidades mayores que las vacas con un solo registro, porque su EBV acumula la información de todas sus hijas. Con una heredabilidad tan baja, sin embargo, todas las confiabilidades son modestas.

8.4 Modelo de repetibilidad con todas las lactancias

Con varias lactancias por vaca se separa el valor genético a_i del ambiente permanente p_i, común a todos los registros de una vaca pero no heredable:

$$y_{ijk} = \mu + \ell_k + h_j + a_i + p_i + e_{ijk}, \qquad \mathbf{p} \sim N\left(\mathbf{0}, \sigma_p^2 \mathbf{I}\right).$$

En MLREML el ambiente permanente es simplemente un segundo término con la misma variable de animal y estructura identidad:

```r
t_rep <- system.time(
  a2 <- mlreml(leche ~ lact, random = ~ id(herd) + vm(animal, A) + id(animal),
               data = milk, trace = FALSE))
t_rep[3]
```

```
## elapsed
##   24.52
```

```r
summary(a2)$varcomp
```

```
##                       estimate std.error z.ratio bound
## id(herd)!sigma2         4.4460    1.0408   4.272     P
## vm(animal, A)!sigma2    0.7982    0.7329   1.089     P
## id(animal)!sigma2       4.7206    0.7970   5.923     P
## R!units!sigma2         10.3955    0.3236  32.121     P
```

```r
vpredict(a2, h2 ~ V2 / (V1 + V2 + V3 + V4))
```

```
##    estimate std.error
## h2   0.0392   0.03597
```

```
vpredict(a2, repetibilidad ~ (V2 + V3) / (V1 + V2 + V3 + V4))

##                estimate std.error
## repetibilidad   0.2711   0.02212

wald(a2)

##          term df wald.inc   F.inc ddf.inc     p.inc wald.con   F.con ddf.con
## 1 (Intercept)  1   6258.1 6201.56   52.88 1.702e-56   6258.1 6201.56   52.88
## 2        lact  4    157.8   39.39 2425.07 5.312e-32    157.8   39.39 2425.07
##       p.con   p.chisq
## 1 1.702e-56 0.000e+00
## 2 5.312e-32 4.409e-33
```

La repetibilidad, la correlación entre dos lactancias de la misma vaca, es mucho mayor que la heredabilidad: gran parte de la semejanza entre lactancias se debe al ambiente permanente. Con más registros por animal la heredabilidad se estima con más precisión que en el análisis de una sola lactancia.

8.5 Efecto materno

El pedigree permite identificar a la madre de cada vaca. Un efecto materno de ambiente permanente (el ambiente que la madre proporciona y que comparten las hermanas maternas) se añade como un término aleatorio con el factor madre:

```
m1$madre <- pp$dam[match(m1$animal, pp$id)]
m1m <- subset(m1, !is.na(madre))
a3 <- mlreml(leche ~ 1, random = ~ id(herd) + vm(animal, A) + id(madre), data =
m1m, trace = FALSE)
a3b <- mlreml(leche ~ 1, random = ~ id(herd) + vm(animal, A), data = m1m, trace
= FALSE)
summary(a3)$varcomp

##                        estimate std.error   z.ratio bound
## id(herd)!sigma2      6.464e+00     1.669 3.872e+00     P
## vm(animal, A)!sigma2 1.457e+00     1.284 1.135e+00     P
## id(madre)!sigma2       1.723e-07     2.568 6.709e-08     B
## R!units!sigma2       1.100e+01     2.676 4.110e+00     P

lrt(a3b, a3)

##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  1   -1.341e-07     0.5 4957 4959 4972 4978
```

La varianza materna queda en la frontera (código `B`): no hay evidencia de un efecto de ambiente materno en la producción de leche de estas vacas. El estadístico de razón de verosimilitudes es cero dentro de la precisión numérica.

Un efecto genético materno correlacionado con el efecto genético directo se ajusta con el término `str()`, presentado en la Sección 8.7.

8.6 Análisis bivariado de leche y grasa

Para estimar la correlación genética entre producción de leche y de grasa en la primera lactancia se ajusta un modelo animal bivariado con matrices de covarianza no estructuradas para los efectos de hato, genéticos y residuales:

$$\text{Var}\begin{pmatrix}\mathbf{a}_L \\ \mathbf{a}_G\end{pmatrix} = \begin{pmatrix}\sigma^2_{a_L} & \sigma_{a_{LG}} \\ \sigma_{a_{LG}} & \sigma^2_{a_G}\end{pmatrix} \otimes \mathbf{A}.$$

```
rm(a2, a3, a3b, Ai, ebv); invisible(gc())
t_biv <- system.time(
  bv <- mlreml(cbind(leche, grasa) ~ trait,
               random = ~ us(trait):id(herd) + us(trait):vm(animal, A),
               residual = ~ us(trait):units, data = m1, trace = FALSE))
t_biv[3]

## elapsed
##   36.59

summary(bv)$varcomp

##                                            estimate std.error z.ratio bound
## us(trait):id(herd)!trait_leche:leche         5.5995    1.3545  4.1339     P
## us(trait):id(herd)!trait_grasa:leche         1.6135    0.4267  3.7812     P
## us(trait):id(herd)!trait_grasa:grasa         0.6589    0.1601  4.1150     P
## us(trait):vm(animal, A)!trait_leche:leche    0.6726    0.8661  0.7765     P
## us(trait):vm(animal, A)!trait_grasa:leche    0.1022    0.2785  0.3669     P
## us(trait):vm(animal, A)!trait_grasa:grasa    0.2399    0.1358  1.7667     P
## R!us(trait):units!trait_leche:leche         12.3885    0.9624 12.8725     P
## R!us(trait):units!trait_grasa:leche          3.1642    0.3041 10.4068     P
## R!us(trait):units!trait_grasa:grasa          1.4939    0.1396 10.7042     P

vpredict(bv, rg ~ V5 / sqrt(V4 * V6))

##    estimate std.error
## rg   0.2544     0.538

vpredict(bv, re ~ V8 / sqrt(V7 * V9))

##    estimate std.error
## re   0.7355   0.03364

vpredict(bv, h2_leche ~ V4 / (V1 + V4 + V7))

##          estimate std.error
## h2_leche  0.03604   0.04644

vpredict(bv, h2_grasa ~ V6 / (V3 + V6 + V9))

##          estimate std.error
## h2_grasa   0.1003   0.05661

round(vc_matrix(bv, "us(trait):vm(animal, A)")$cor, 3)

##       leche grasa
## leche 1.000 0.254
## grasa 0.254 1.000

round(vc_matrix(bv, "residual")$cor, 3)

##       leche grasa
## leche 1.000 0.736
## grasa 0.736 1.000
```

Los parámetros se nombran `V1` a `V9` en el orden de la tabla: covarianzas de hato (V1–V3), genéticas (V4–V6) y residuales (V7–V9). La correlación residual entre leche y grasa es alta y bien estimada; la correlación genética tiene un error estándar grande, como era de esperar con solo 1314 vacas.

8.7 Efectos genéticos directos y maternos

En caracteres que se expresan temprano en la vida (peso al nacer y al destete, tamaño de camada, supervivencia) y en algunos caracteres de la progenie de la madre, el fenotipo de un animal depende de sus propios genes y de la habilidad materna de su madre, que también es en parte genética. Los efectos genéticos directo y materno suelen estar correlacionados negativamente (Willham, 1963; Meyer, 1992). El modelo (2.17) se ajusta con el término `str(animal, dam, K = A)` (Sección 2.11), normalmente junto con un efecto materno de ambiente permanente `id(dam)`. Como los registros de leche de la Sección 8.1 incluyen pocas madres con registros propios (ver el final de esta sección), la estimación se ilustra primero con una población simulada cuyos parámetros verdaderos se conocen.

La siguiente función simula seis generaciones de 600 animales a partir de 300 fundadores, con 25 padres por generación elegidos entre los machos de las dos últimas generaciones; valores genéticos directos y maternos con $\sigma^2_d = 0.40$, $\sigma^2_m = 0.20$ y $\sigma_{dm} = -0.10$ (correlación genética −0.35); una varianza materna de ambiente permanente de 0.10; una varianza residual de 0.50; y 30 grupos contemporáneos, tratados como fijos. Todo animal nacido después de los fundadores tiene un registro, de modo que la mayoría de las madres tiene registros propios.

```
sim_maternal <- function(seed = 2026, nf = 300, ngen = 6, noff = 600,
                         S = matrix(c(0.40, -0.10, -0.10, 0.20), 2), s2pe =
0.10, s2e = 0.50) {
  set.seed(seed)
  ped <- data.frame(id = paste0("F", 1:nf), sire = NA, dam = NA,
                    sex = rep(c("M", "F"), length.out = nf), gen = 0)
  for (g in 1:ngen) {                                  # padres y madres de las
dos últimas generaciones
    cand <- ped[ped$gen >= g - 2, ]
    sires <- sample(cand$id[cand$sex == "M"], 25)
    ped <- rbind(ped, data.frame(id = paste0("G", g, "_", 1:noff),
                                 sire = sample(sires, noff, TRUE),
                                 dam = sample(cand$id[cand$sex == "F"], noff,
TRUE),
                                 sex = sample(c("M", "F"), noff, TRUE), gen =
g))
  }
  A <- Amatrix(ped[, 1:3])
  U <- t(chol(A)) %*% matrix(rnorm(2 * nrow(A)), ncol = 2) %*% chol(S)   #
(directo, materno)
  rownames(U) <- rownames(A)
  d <- ped[ped$gen > 0, ]
  d$cg <- factor(paste(d$gen, sample(1:5, nrow(d), TRUE), sep = "_"))   #
grupos contemporáneos
  pe <- setNames(rnorm(nrow(A), sd = sqrt(s2pe)), rownames(A))
  d$y <- 10 + rnorm(nlevels(d$cg))[d$cg] + U[d$id, 1] + U[d$dam, 2] + pe[d$dam]
+
         rnorm(nrow(d), sd = sqrt(s2e))
  list(data = d, ped = ped, A = A, U = U)
}
```

```
sm <- sim_maternal()
dsm <- sm$data; Asim <- sm$A
c(registros = nrow(dsm), animales = nrow(sm$A), madres =
length(unique(dsm$dam)),
  madres_con_registro = sum(unique(dsm$dam) %in% dsm$id))
```

```
##              registros               animales                  madres
madres_con_registro
##                   3600                  3900                  1407
1257
```

El modelo se ajusta con el motor denso, con la matriz **A**, y con el motor disperso, con su inversa:

```
t_d <- system.time(
  dm_d <- mlreml(y ~ cg, random = ~ str(id, dam, K = Asim) + id(dam), data =
dsm,
                 trace = FALSE, engine = "denso"))
Aisim <- Ainverse(sm$ped)
t_s <- system.time(
  dm_s <- mlreml(y ~ cg, random = ~ str(id, dam, K = Aisim) + id(dam), data =
dsm,
                 trace = FALSE, engine = "disperso"))
c(denso = t_d[3], disperso = t_s[3], dif_logLik = dm_d$loglik - dm_s$loglik)
```

```
##     denso.elapsed    disperso.elapsed dif_logLik
##         3.352e+01         1.592e+00        -5.401e-09
```

```
cbind(verdadero = c(0.40, -0.10, 0.20, 0.10, 0.50), summary(dm_d)$varcomp[,
c("estimate", "std.error")])
```

```
##                                 verdadero estimate std.error
## str(id, dam, K = Asim)!id:id          0.4  0.40027   0.06655
## str(id, dam, K = Asim)!dam:id        -0.1 -0.12404   0.04039
## str(id, dam, K = Asim)!dam:dam        0.2  0.27205   0.04846
## id(dam)!sigma2                        0.1  0.04608   0.02658
## R!units!sigma2                        0.5  0.51821   0.03935
```

Ambos motores alcanzan el mismo máximo, y el motor disperso es mucho más rápido incluso con 3,600 registros. Todas las estimaciones están a unos dos errores estándar o menos de los valores verdaderos. Los efectos maternos se estiman con menos precisión que los directos, como es bien sabido, y la varianza materna de ambiente permanente está parcialmente confundida con la varianza genética materna.

```
vc_matrix(dm_d, "str(id, dam, K = Asim)")$cor
```

```
##          id     dam
## id   1.0000 -0.3759
## dam -0.3759  1.0000
```

```
vpredict(dm_d, h2_directa ~ V1 / (V1 + V2 + V3 + V4 + V5))
```

```
##           estimate std.error
## h2_directa   0.3598   0.05505
```

```
vpredict(dm_d, h2_total ~ (V1 + 1.5 * V2 + 0.5 * V3) / (V1 + V2 + V3 + V4 +
V5))
```

```
##          estimate std.error
## h2_total   0.3148   0.03447
```

`vc_matrix()` devuelve $\widehat{\boldsymbol{\Sigma}}$ y su matriz de correlación. La heredabilidad total de Willham (1972), que determina la respuesta a la selección fenotípica, es menor que la heredabilidad directa porque la covarianza directa–materna es negativa.

```
ebv <- coef(dm_d, "random", "str(id, dam, K = Asim)")
ebv[ebv$level %in% c("G6_1", "G6_2"), ]
```

```
##      effect level    blup    pev     se reliability
## 3330     id  G6_1 -0.3304 0.2288 0.4784      0.4305
## 3681     id  G6_2 -0.1133 0.2282 0.4777      0.4381
## 7230    dam  G6_1  0.2735 0.2311 0.4807      0.1539
## 7581    dam  G6_2  0.1242 0.2306 0.4802      0.1646
```

```
c(r_directo = cor(ebv$blup[ebv$effect == "id"], sm$U[ebv$level[ebv$effect ==
"id"], 1]),
  r_materno = cor(ebv$blup[ebv$effect == "dam"], sm$U[ebv$level[ebv$effect ==
"dam"], 2]))
```

```
##   r_directo r_materno
##      0.6712     0.5159
```

```
dm_0 <- mlreml(y ~ cg, random = ~ str(id, dam, K = Asim, cov = "diag") +
id(dam), data = dsm,
               trace = FALSE)
lrt(dm_0, dm_d, boundary = FALSE)
```

```
##   df LR.statistic  p.value  AIC0  AIC1  BIC0  BIC1
## 1  1        14.69 0.000127 10047 10034 10072 10065
```

Cada animal recibe un valor genético directo (`effect = "id"`) y uno materno (`effect = "dam"`), incluidos los machos, cuyo valor materno se expresa en sus hijas. Las correlaciones con los valores verdaderos simulados muestran que los valores maternos se predicen con menor exactitud. El modelo con `cov = "diag"` tiene la covarianza directa–materna fijada en cero; como una covarianza de cero no está en la frontera del espacio paramétrico, la prueba de razón de verosimilitudes usa `boundary = FALSE`, y rechaza $\sigma_{dm} = 0$.

Aplicación a los datos de leche. De los 1,314 registros de primera lactancia de la Sección 8.3, 907 tienen madre conocida, pero solo 14 madres tienen un registro propio de primera lactancia. Sin registros de las madres, el efecto genético materno solo se identifica a través de los parentescos entre medias hermanas maternas y sus parientes, y la covarianza directa–materna no es estimable: con `str(animal, dam, K = A)` la correlación estimada tiende a −1 y el algoritmo no converge en 50 iteraciones. El modelo sin la covarianza converge en pocas iteraciones, con la varianza genética materna en la frontera:

```
c(registros = nrow(m1), con_madre = sum(!is.na(m1$madre)),
  madres_con_registro = sum(unique(na.omit(m1$madre)) %in% m1$animal))
```

```
##             registros             con_madre madres_con_registro
##                  1314                   907                    14
```

```
a5 <- mlreml(leche ~ 1, random = ~ id(herd) + str(animal, madre, K = A, cov =
"diag"),
             data = m1, trace = FALSE)
summary(a5)$varcomp
```

```
##                                              estimate std.error   z.ratio
bound
## id(herd)!sigma2                             5.646e+00    1.3654 4.135e+00
P
## str(animal, madre, K = A, cov = "diag")!animal 6.751e-01    0.8682 7.775e-01
P
## str(animal, madre, K = A, cov = "diag")!madre    6.103e-08    0.9978 6.117e-
08     B
## R!units!sigma2                              1.238e+01    1.1259 1.100e+01
P

lrt(a1, a5)

##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  1   -1.944e-07     0.5 7217 7219 7233 7240
```

Al igual que con el efecto de ambiente materno de la Sección 8.5, estos datos no contienen evidencia de una varianza genética materna para la producción de leche en primera lactancia: la log-verosimilitud es la misma que la del modelo de la Sección 8.3.

8.8 Conjuntos de datos grandes: el motor disperso

Los ejemplos de este capítulo tienen unos pocos miles de registros, un tamaño en el que el motor denso todavía es práctico. El motor disperso (Sección 2.12) resuelve en cambio las ecuaciones del modelo mixto, y su ventaja crece rápidamente con el tamaño de los datos. El modelo de repetibilidad de la Sección 8.4 se ajusta de nuevo con ambos motores; el motor disperso recibe la inversa de $\mathbf{A}$ construida por `Ainverse()`, que lleva el atributo `INVERSE = TRUE`:

```
Ai <- Ainverse(pp)
t_dense <- system.time(
  a2d <- mlreml(leche ~ lact, random = ~ id(herd) + vm(animal, A) + id(animal),
                data = milk, trace = FALSE, engine = "denso"))
t_sparse <- system.time(
  a2s <- mlreml(leche ~ lact, random = ~ id(herd) + vm(animal, Ai) +
id(animal),
                data = milk, trace = FALSE, engine = "disperso"))
c(denso = t_dense[3], disperso = t_sparse[3])

##  denso.elapsed disperso.elapsed
##         25.720          0.526

summary(a2s)

## MLREML fit (AI-REML, sparse engine)
## Observations: 3397   Fixed-effect columns: 5   Iterations: 7 (converged)
## REML log-likelihood: -9400.8394   AIC: 18809.68   BIC: 18834.20
##
## Variance parameters:
##                        estimate std.error z.ratio bound
## id(herd)!sigma2          4.4460    1.0410    4.27     P
## vm(animal, Ai)!sigma2    0.7982    0.7329    1.09     P
## id(animal)!sigma2        4.7206    0.7970    5.92     P
## R!units!sigma2          10.3960    0.3236   32.12     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

La salida es la de la Sección 8.4, y el motor disperso necesita bastante menos de un segundo en lugar de casi medio minuto. Las diferencias entre los dos motores son del orden de la tolerancia de convergencia, porque las iteraciones siguen trayectorias numéricas ligeramente distintas:

```
e_d <- coef(a2d, "random", "vm(animal, A)"); e_s <- coef(a2s, "random",
"vm(animal, Ai)")
c(logLik = a2d$loglik - a2s$loglik, theta = max(abs(a2d$theta - a2s$theta)),
  ebv = max(abs(e_d$blup - e_s$blup)), pev = max(abs(e_d$pev - e_s$pev)),
  se_beta = max(abs(sqrt(diag(vcov(a2d))) - sqrt(diag(vcov(a2s))))))

##    logLik     theta       ebv       pev   se_beta
## 2.910e-10 1.573e-08 1.576e-08 1.891e-08 3.790e-11

a2s$sparse[c("N", "nnzL")]

## $N
## [1] 7968
##
## $nnzL
## [1] 106055
```

Las ecuaciones del modelo mixto de este modelo tienen 7,968 incógnitas: 5 efectos fijos, 57 hatos, los valores genéticos de los 6,547 animales del pedigree y los efectos de ambiente permanente de las 1,359 vacas con registros. El modelo bivariado de la Sección 8.6, con matrices de covarianza no estructuradas para los efectos de hato, genéticos y residuales y registros con un solo carácter observado, también se ajusta con el motor disperso:

```
t_bs <- system.time(
  bvs <- mlreml(cbind(leche, grasa) ~ trait,
                random = ~ us(trait):id(herd) + us(trait):vm(animal, Ai),
                residual = ~ us(trait):units, data = m1, trace = FALSE, engine
= "sparse"))
t_bs[3]

## elapsed
##   1.655

c(logLik_denso = bv$loglik, diferencia = bv$loglik - bvs$loglik)

## logLik_denso   diferencia
##   -5.445e+03    1.237e-08

vpredict(bvs, rg ~ V5 / sqrt(V4 * V6))

##    estimate std.error
## rg   0.2544     0.538
```

El ajuste disperso tarda menos de dos segundos (37.6 s con el motor denso en la Sección 8.6) y alcanza la misma log-verosimilitud. La Sección 15.2 muestra el tamaño de los conjuntos de datos que maneja el motor disperso, con datos simulados de hasta dos millones de registros.

9 Ejemplo 6: single-step GBLUP con individuos genotipados y no genotipados

9.1 Planteamiento

En la práctica casi nunca se genotipan todos los individuos con pedigree y fenotipo. El *single-step* GBLUP (Legarra et al., 2009; Christensen y Lund, 2010) usa en un solo análisis el pedigree de todos los individuos y los marcadores de los genotipados, mediante la matriz $\mathbf{H}$ de la ecuación (2.16). Los individuos no genotipados se benefician de los marcadores de sus parientes genotipados.

Para ilustrarlo con datos reales usamos de nuevo las 599 líneas de trigo de `BGLR`, que tienen pedigree y marcadores, y simulamos la situación habitual: solo 360 líneas (60 %) se consideran genotipadas. Esto permite comparar las predicciones de pedigree (A-BLUP) y single-step (H-BLUP) con los fenotipos reales de las líneas no genotipadas.

```
data(wheat)
ids <- paste0("L", 1:599)
Aw <- wheat.A; dimnames(Aw) <- list(ids, ids)
Mw <- 2 * wheat.X; rownames(Mw) <- ids
yw <- setNames(wheat.Y[, 1], ids)
set.seed(7)
genotipadas <- sample(ids, 360)
Gg <- Gmatrix(Mw[genotipadas, ], method = "VanRaden")
```

9.2 Alineación de G con el pedigree y construcción de H

Antes de combinar las matrices, $\mathbf{G}$ se alinea con $\mathbf{A}_{22}$ (el bloque de pedigree de los genotipados) y se mezcla con un 5 % de $\mathbf{A}_{22}$:

```
A22 <- Aw[genotipadas, genotipadas]
Gal <- align_G(Gg, A22)
c(alfa = attr(Gal, "alpha"), beta = attr(Gal, "beta"))

##   alfa   beta
## 0.3799 0.7989

rbind(G_original = c(diag = mean(diag(Gg)), todos = mean(Gg)),
      G_alineada = c(mean(diag(Gal)), mean(Gal)),
      A22 = c(mean(diag(A22)), mean(A22)))

##              diag     todos
## G_original 2.000 3.550e-17
## G_alineada 1.978 3.799e-01
## A22        1.978 3.799e-01

H <- Hmatrix(Aw, Gg, blend = 0.05, align = TRUE)
range(eigen(H, symmetric = TRUE, only.values = TRUE)$values)

## [1] 9.786e-04 2.197e+02
```

Después de la alineación, las medias de la diagonal y de todos los elementos de $\mathbf{G}$ coinciden con las de $\mathbf{A}_{22}$. La matriz $\mathbf{H}$ resultante es definida positiva. Si no lo fuera, `bend()` la corregiría.

La inversa $\mathbf{H}^{-1}$, que es la que usan los programas basados en ecuaciones del modelo mixto, se obtiene con `inverse = TRUE`; la verificación numérica muestra que es la inversa de $\mathbf{H}$:

```
Hi <- Hmatrix(Aw, Gg, blend = 0.05, align = TRUE, inverse = TRUE)
max(abs(Hi %*% H - diag(599)))

## [1] 1.669e-11
```

9.3 Comparación por validación

Se eliminan los fenotipos de 120 líneas elegidas al azar (genotipadas y no genotipadas), se predicen con A-BLUP y con H-BLUP y se calcula la correlación con los fenotipos reales por separado para los dos grupos. Se repite cinco veces.

```
Ar <- Aw + diag(1e-3, 599); Hr <- H + diag(1e-3, 599)
res <- do.call(rbind, lapply(1:5, function(r) {
  set.seed(100 + r)
  test <- sample(ids, 120)
  ytr <- yw; ytr[test] <- NA
  pA <- gblup(ytr, Ar)$gebv
  pH <- gblup(ytr, Hr)$gebv
  ng <- setdiff(test, genotipadas); gg <- intersect(test, genotipadas)
  data.frame(rep = r, A_no_genot = cor(pA[ng], yw[ng]), H_no_genot =
cor(pH[ng], yw[ng]),
             A_genot = cor(pA[gg], yw[gg]), H_genot = cor(pH[gg], yw[gg]))
}))
round(res, 3)

##   rep A_no_genot H_no_genot A_genot H_genot
## 1   1      0.424      0.402   0.349   0.412
## 2   2      0.362      0.315   0.226   0.465
## 3   3      0.341      0.353   0.488   0.500
## 4   4      0.344      0.283   0.461   0.415
## 5   5      0.584      0.636   0.387   0.537

round(colMeans(res[, -1]), 3)

## A_no_genot H_no_genot    A_genot    H_genot
##      0.411      0.398      0.382      0.466
```

La ganancia de H-BLUP sobre el pedigree se concentra en las líneas genotipadas, cuyo parentesco realizado se conoce. En las líneas no genotipadas, H-BLUP y A-BLUP tienen en promedio una capacidad predictiva similar, con diferencias en ambos sentidos entre repeticiones. En teoría las no genotipadas pueden beneficiarse porque sus covarianzas con las genotipadas se actualizan a través de $\mathbf{A}_{12}\mathbf{A}_{22}^{-1}$, pero ese beneficio depende de qué tan emparentadas estén con las genotipadas; en estas líneas de trigo, con un pedigree que ya describe bien el parentesco, no se observa. Es un resultado común en la literatura: la ventaja del single-step para los no genotipados suele ser pequeña.

9.4 Single-step con el motor general

`gblup()` se usó arriba por velocidad, pero $\mathbf{H}$ puede usarse en cualquier modelo de `mlreml()` a través de `vm()`, por ejemplo con varianzas residuales heterogéneas o en un modelo multiambiental:

```
dss <- data.frame(gen = ids, y = yw)
fss <- mlreml(y ~ 1, random = ~ vm(gen, Hr), data = dss, trace = FALSE)
summary(fss)$varcomp
```

```
##                     estimate std.error z.ratio bound
## vm(gen, Hr)!sigma2   0.3648   0.06446   5.659     P
## R!units!sigma2       0.5186   0.05096  10.176     P
```

```
vpredict(fss, h2 ~ V1 / (V1 + V2))
```

```
##    estimate std.error
## h2   0.4129   0.05974
```

9.5 Recomendaciones

- Verificar que los identificadores de $\mathbf{G}$ estén en el pedigree y que el pedigree esté completo hacia atrás.
- Alinear $\mathbf{G}$ con $\mathbf{A}_{22}$ (`align = TRUE`) y mezclar con una fracción pequeña de $\mathbf{A}_{22}$ (5–10 %).
- En poblaciones con pedigree poco profundo, los parámetros τ y ω de $\mathbf{H}^{-1}$ pueden ajustarse para reducir el sesgo de las predicciones (Misztal et al., 2014).

10 Ejemplo 7: medidas repetidas y regresión aleatoria

Muchos caracteres se miden varias veces en el mismo individuo: peso a distintas edades, producción de leche a lo largo de la lactancia, altura de planta durante el ciclo, rendimiento en cortes sucesivos. Tratar esas observaciones como independientes subestima los errores estándar y desperdicia la información sobre la trayectoria de cada individuo. MLREML ofrece dos enfoques complementarios: modelar la correlación de los residuos dentro de individuo (estructuras residuales) y modelar trayectorias individuales con coeficientes aleatorios (regresión aleatoria).

10.1 Correlación residual: crecimiento de niños

Los datos `Orthodont` (Potthoff y Roy, 1964; en `nlme`) miden la distancia entre la hipófisis y la fisura pterigomaxilar en 27 niños y niñas a los 8, 10, 12 y 14 años. Se comparan varias estructuras para la matriz de covarianza de los cuatro registros de cada niño.

```
library(nlme)
ort <- as.data.frame(Orthodont)
ort$Subject <- factor(as.character(ort$Subject))
ort$edad <- ort$age
o1 <- mlreml(distance ~ age * Sex, random = ~ Subject, data = ort, trace =
FALSE)
o2 <- mlreml(distance ~ age * Sex, residual = ~ ar1(edad):id(Subject), data =
ort, trace = FALSE)
o3 <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
ar1(edad):id(Subject),
             data = ort, trace = FALSE)
o4 <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
diag(Sex):units,
             data = ort, trace = FALSE)
o5 <- mlreml(distance ~ age * Sex, residual = ~ us(edad):id(Subject), data =
ort, trace = FALSE)
data.frame(modelo = c("intercepto aleatorio (simetría compuesta)", "AR1",
                      "intercepto + AR1", "intercepto + residual por sexo", "no
estructurada"),
           parametros = sapply(list(o1, o2, o3, o4, o5), function(f)
sum(f$free)),
```

```
          logLik = sapply(list(o1, o2, o3, o4, o5), `[[`, "loglik"),
          AIC = sapply(list(o1, o2, o3, o4, o5), `[[`, "AIC"))
```

```
##                                      modelo parametros logLik   AIC
## 1 intercepto aleatorio (simetría compuesta)          2 -216.9 437.8
## 2                                       AR1          2 -222.3 448.6
## 3                            intercepto + AR1          3 -216.9 439.8
## 4              intercepto + residual por sexo          3 -207.6 421.2
## 5                               no estructurada         10 -212.3 444.5
```

En `ar1(edad)` las edades son numéricas, de modo que MLREML usa las distancias reales entre mediciones: la correlación entre dos registros separados por Δ años es ρ^{Δ}. Así, la misma estructura sirve para mediciones a intervalos irregulares, como haría `expc()`. El modelo con varianza residual distinta por sexo es el preferido por el AIC: las niñas tienen una variabilidad dentro de individuo mucho menor que los niños.

```
summary(o4)
```

```
## MLREML fit (AI-REML)
## Observations: 108   Fixed-effect columns: 4   Iterations: 7 (converged)
## REML log-likelihood: -207.6102   AIC: 421.22   BIC: 429.15
##
## Variance parameters:
##                                estimate std.error z.ratio bound
## Subject!sigma2                   3.4135    1.0870    3.14     P
## R!diag(Sex):units!Sex_Male       2.7883    0.5711    4.88     P
## R!diag(Sex):units!Sex_Female     0.6104    0.1528    3.99     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

10.2 Regresión aleatoria lineal: privación de sueño

Los datos `sleepstudy` (Belenky et al., 2003; en `lme4`) registran el tiempo de reacción promedio de 18 personas durante 10 días de privación de sueño. Cada persona tiene su propia recta, con intercepto y pendiente aleatorios correlacionados:

$$y_{ij} = (\beta_0 + b_{0i}) + (\beta_1 + b_{1i})\, t_j + e_{ij}, \qquad \begin{pmatrix} b_{0i} \\ b_{1i} \end{pmatrix} \sim N\left(\mathbf{0}, \begin{pmatrix} \sigma_0^2 & \sigma_{01} \\ \sigma_{01} & \sigma_1^2 \end{pmatrix}\right).$$

En MLREML la base `pol(Days, 1)` genera las columnas $(1, t)$ y la estructura `us` su matriz de covarianza; el término completo es `us(pol(Days, 1)):id(Subject)`.

```
data(sleepstudy, package = "lme4")
rr <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy,
             trace = FALSE)
summary(rr)
```

```
## MLREML fit (AI-REML)
## Observations: 180   Fixed-effect columns: 2   Iterations: 4 (converged)
## REML log-likelihood: -871.8141   AIC: 1751.63   BIC: 1764.36
##
## Variance parameters:
##                                                 estimate std.error z.ratio
bound
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_0:0  612.090    288.80    2.12
```

```
P
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_1:0    9.604     46.68    0.21
P
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_1:1   35.072     14.78    2.37
P
## R!units!sigma2                                  654.940     77.19    8.49
P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
vpredict(rr, cor_int_pend ~ V2 / sqrt(V1 * V3))
```

```
##              estimate std.error
## cor_int_pend  0.06555     0.325
```

Los BLUP de los coeficientes de cada persona permiten reconstruir sus rectas. Los niveles de un término siguen el orden de sus componentes, con el último variando más rápido. En `us(pol(Days, 1)):id(Subject)` el primer componente es la base polinomial y el segundo la persona, de modo que los primeros 18 BLUP son los interceptos de las 18 personas y los 18 siguientes sus pendientes. La matriz de coeficientes se forma por columnas:

```
b <- coef(rr, "random")[[1]]
head(b$level, 3)
```

```
## [1] "0:308" "0:309" "0:310"
```

```
subj <- levels(factor(sleepstudy$Subject))
```

```
B <- matrix(b$blup, ncol = 2, dimnames = list(subj, c("intercepto",
"pendiente")))
head(B)
```

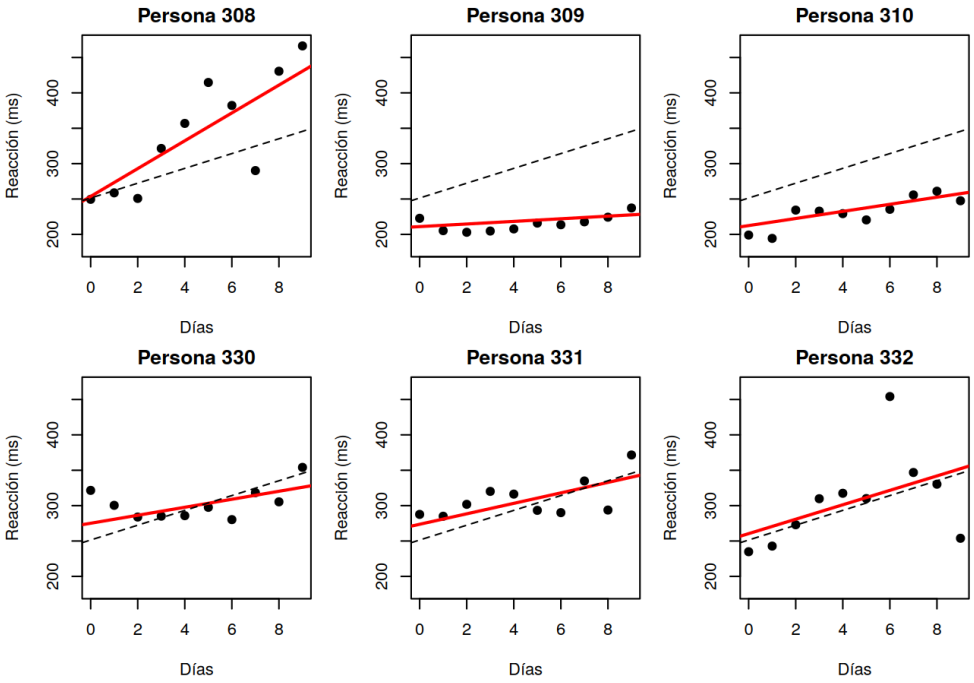
```
##     intercepto pendiente
## 308      2.259    9.1990
## 309    -40.399   -8.6197
## 310    -38.960   -5.4489
## 330     23.691   -4.8143
## 331     22.260   -3.0699
## 332      9.040   -0.2722
```

```
head(lme4::ranef(lme4::lmer(Reaction ~ Days + (Days | Subject),
sleepstudy))$Subject)
```

```
##     (Intercept)    Days
## 308       2.259  9.1990
## 309     -40.399 -8.6197
## 310     -38.960 -5.4489
## 330      23.691 -4.8144
## 331      22.260 -3.0699
## 332       9.040 -0.2722
```

```
op <- par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
for (s in subj[1:6]) {
  ds <- subset(sleepstudy, Subject == s)
  plot(ds$Days, ds$Reaction, pch = 19, ylim = c(180, 470), main =
paste("Persona", s),
       xlab = "Días", ylab = "Reacción (ms)")
  abline(rr$beta[1] + B[s, 1], rr$beta[2] + B[s, 2], col = "red", lwd = 2)
```

```
  abline(rr$beta, lty = 2)
}
```



Tiempo de reacción observado (puntos), rectas individuales predichas (rojo) y recta media (discontinua).

```
par(op)
```

10.3 Regresión aleatoria con polinomios de Legendre: curvas de crecimiento

En animal breeding las trayectorias se modelan con polinomios de Legendre (Kirkpatrick et al., 1990; Schaeffer, 2004), que son ortogonales en $[-1,1]$ y dan matrices de covarianza mejor condicionadas que los polinomios crudos. Los datos `ChickWeight` (Crowder y Hand, 1990) contienen el peso de 50 pollos alimentados con cuatro dietas, medido desde el nacimiento hasta el día 21.

```
cw <- as.data.frame(ChickWeight)
cw$Chick <- factor(as.character(cw$Chick))
ck1 <- mlreml(weight ~ Diet + Diet:Time, random = ~ us(leg(Time, 1)):id(Chick),
              residual = ~ diag(Time):units, data = cw, trace = FALSE)
ck2 <- mlreml(weight ~ Diet + Diet:Time, random = ~ us(leg(Time, 2)):id(Chick),
              residual = ~ diag(Time):units, data = cw, trace = FALSE)
lrt(ck1, ck2, boundary = FALSE)

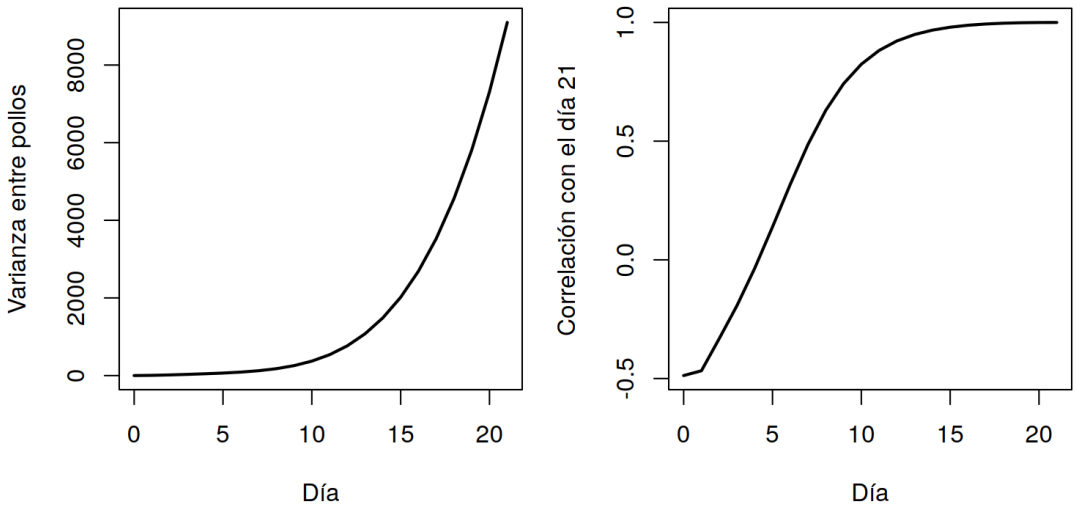
##   df LR.statistic   p.value AIC0 AIC1 BIC0 BIC1
## 1  3        445.3 3.436e-96 4538 4099 4603 4177

summary(ck2)$varcomp[1:6, ]

##                                                 estimate std.error z.ratio bound
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_0:0   1665.1    367.06   4.536     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_1:0   1542.1    340.91   4.523     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_1:1   1526.5    325.35   4.692     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:0    442.1    105.07   4.208     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:1    484.7    104.72   4.629     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:2    175.1     36.42   4.808     P
```

El modelo cuadrático es claramente mejor. La función de covarianza entre pesos a dos edades t y t' es $\boldsymbol{\phi}(t)'\mathbf{K}\boldsymbol{\phi}(t')$, donde $\boldsymbol{\phi}(t)$ es el vector de polinomios de Legendre evaluados en t y $\mathbf{K}$ la matriz de covarianza de los coeficientes. Con ella se obtiene la varianza entre pollos en cada edad y la correlación entre edades:

```
Kc <- vc_matrix(ck2, "us(leg(Time, 2)):id(Chick)")$cov
tt <- 0:21
Phi <- MLREML:::.legendre(c(tt, range(cw$Time)), 2)[seq_along(tt), ]
Cfun <- Phi %*% Kc %*% t(Phi)
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
plot(tt, diag(Cfun), type = "l", lwd = 2, xlab = "Día", ylab = "Varianza entre
pollos")
plot(tt, cov2cor(Cfun)[, 22], type = "l", lwd = 2, xlab = "Día", ylab =
"Correlación con el día 21")
```



Varianza entre pollos estimada por la regresión aleatoria (izquierda) y correlación del peso a cada edad con el peso al día 21 (derecha).

```
par(op)
```

Con un pedigree, el mismo término con `vm(animal, A)` en lugar de `id(Chick)` da la función de covarianza genética aditiva, de la que se obtienen heredabilidades a cada edad y correlaciones genéticas entre edades. Un modelo completo de regresión aleatoria en animal breeding tiene típicamente un término genético y otro de ambiente permanente:

```
mlreml(peso ~ grupo + leg(edad, 3),
       random = ~ us(leg(edad, 2)):vm(animal, A) + us(leg(edad, 2)):id(animal),
       residual = ~ diag(clase_edad):units, data = registros)
```

11 Ejemplo 8: análisis dialélico

11.1 Los datos

El conjunto `hayman.tobacco` (Hayman, 1954; en `agridat`) es un dialelo completo de 8 variedades de tabaco (*Nicotiana rustica*): las 64 combinaciones de progenitor femenino y masculino, incluidas las autofecundaciones y las cruzas recíprocas, evaluadas en 2 bloques en cada uno de 2 años. La respuesta es el número de días a floración.

```
data(hayman.tobacco)
dt <- hayman.tobacco
dt$anio <- factor(dt$year)
str(dt)
```

```
## 'data.frame':    256 obs. of  6 variables:
##  $ year  : int  1951 1951 1951 1951 1951 1951 1951 1951 1951 1951 ...
##  $ block : Factor w/ 2 levels "B1","B2": 1 1 1 1 1 1 1 1 2 2 ...
##  $ male  : Factor w/ 8 levels "G1","G2","G3",..: 1 2 3 4 5 6 7 8 1 2 ...
##  $ female: Factor w/ 8 levels "G1","G2","G3",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ day   : num  27.6 13.6 24.6 31.8 15 18.2 17.4 15.2 30.2 14.2 ...
##  $ anio  : Factor w/ 2 levels "1951","1952": 1 1 1 1 1 1 1 1 1 1 ...
```

```
with(dt, table(female, male))[1:4, 1:4]
```

```
##       male
## female G1 G2 G3 G4
##     G1  4  4  4  4
##     G2  4  4  4  4
##     G3  4  4  4  4
##     G4  4  4  4  4
```

11.2 El modelo

En un dialelo, el valor de la cruza entre la hembra i y el macho j se descompone en (Griffing, 1956; Cockerham y Weir, 1977):

$$y_{ijk} = \mu + \text{ambiente}_k + g_i + g_j + s_{ij} + m_i + r_{ij} + e_{ijk},$$

donde g es la aptitud combinatoria general (ACG) de cada progenitor, la misma actúe como hembra o como macho; $s_{ij} = s_{ji}$ es la aptitud combinatoria específica (ACE) del par, independiente de la dirección de la cruza; m_i es el efecto materno general de la hembra; y r_{ij} es el efecto recíproco específico de la combinación ordenada. Con progenitores aleatorios, cada efecto tiene su varianza.

La dificultad técnica es la ACG: cada observación tiene **dos** efectos del mismo factor. En MLREML esto se resuelve con la base de pertenencia múltiple `mm(female, male)`, cuya matriz de diseño tiene un 1 en la columna de cada progenitor (y un 2 en las autofecundaciones). La ACE usa un factor con la cruza sin orden y los efectos recíprocos un factor con la cruza ordenada.

```
dt$cruza <- factor(with(dt, paste(pmin(as.character(female),
as.character(male)),
                                  pmax(as.character(female),
as.character(male)), sep = "x")))
dt$cruza_ord <- factor(paste(dt$female, dt$male, sep = ">"))
c(cruzas_sin_orden = nlevels(dt$cruza), cruzas_ordenadas =
nlevels(dt$cruza_ord))
```

```
## cruzas_sin_orden cruzas_ordenadas
##               36               64
```

```
di <- mlreml(day ~ anio + anio:block,
             random = ~ mm(female, male) + cruza + female + cruza_ord,
             data = dt, trace = FALSE)
summary(di)
```

```
## MLREML fit (AI-REML)
## Observations: 256   Fixed-effect columns: 4   Iterations: 7 (converged)
## REML log-likelihood: -676.2058   AIC: 1362.41   BIC: 1380.06
##
## Variance parameters:
##                          estimate std.error z.ratio bound
## mm(female, male)!sigma2   9.0600    5.1950    1.74     P
## cruza!sigma2              3.8401    1.6140    2.38     P
## female!sigma2             0.4936    0.6835    0.72     P
## cruza_ord!sigma2          0.7941    0.9461    0.84     P
## R!units!sigma2            8.3574    0.8597    9.72     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

11.3 Interpretación

La varianza de ACG es la mayor de las varianzas genéticas, lo que indica que la mayor parte de la variación genética en días a floración es aditiva y se transmite de forma predecible a la descendencia. La varianza de ACE refleja efectos no aditivos (dominancia y epistasis) específicos de cada combinación. Los efectos maternos y recíprocos son pequeños.

Bajo los supuestos habituales (progenitores no emparentados, sin epistasis), $\sigma^2_A = 4\sigma^2_{ACG}$ en una población diploide no endogámica y $2\sigma^2_{ACG}$ para progenitores completamente endogámicos, que es el caso de las variedades de tabaco; la proporción de la varianza genética que es aditiva se estima con `vpredict()`:

```
vpredict(di, prop_ACG ~ 2 * V1 / (2 * V1 + V2 + V3 + V4))

##          estimate std.error
## prop_ACG   0.7794    0.1137

vpredict(di, Baker ~ 2 * V1 / (2 * V1 + V2))

##       estimate std.error
## Baker   0.8251    0.1033
```

La razón de Baker (1978), $2\sigma^2_{ACG}/(2\sigma^2_{ACG} + \sigma^2_{ACE})$, cercana a uno indica que la predicción de cruzas a partir de la ACG de los progenitores será precisa.

11.4 Aptitudes combinatorias predichas

```
acg <- coef(di, "random", "mm(female, male)")
acg[order(acg$blup), c("level", "blup", "se", "reliability")]

##   level    blup   se reliability
## 7    G7 -3.9663 1.29      0.8163
## 8    G8 -2.0061 1.29      0.8163
## 2    G2 -1.6694 1.29      0.8163
## 5    G5 -1.3773 1.29      0.8163
## 6    G6  0.2339 1.29      0.8163
## 4    G4  0.5217 1.29      0.8163
## 1    G1  3.7705 1.29      0.8163
## 3    G3  4.4929 1.29      0.8163
```

Como la respuesta es días a floración, los progenitores con ACG negativa transmiten precocidad. El valor predicho de una cruza no evaluada $i \times j$ se obtiene sumando la media, las dos ACG y, si se dispone de ellos, la ACE y los efectos maternos:

```
ace <- coef(di, "random", "cruza")
g <- setNames(acg$blup, acg$level); s <- setNames(ace$blup, ace$level)
mu <- mean(di$fixed_fitted)
pred_cruza <- function(p1, p2) {
  k <- paste(min(p1, p2), max(p1, p2), sep = "x")
  unname(mu + g[p1] + g[p2] + ifelse(k %in% names(s), s[k], 0))
}
c(G1xG3 = pred_cruza("G1", "G3"), G7xG8 = pred_cruza("G7", "G8"))

## G1xG3 G7xG8
## 27.85 10.95
```

11.5 Comparación de modelos

¿Son necesarios los efectos maternos y recíprocos? Se comparan modelos anidados con la razón de verosimilitudes:

```
di0 <- mlreml(day ~ anio + anio:block, random = ~ mm(female, male) + cruza,
              data = dt, trace = FALSE)
lrt(di0, di, boundary = FALSE)

##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  2        3.974  0.1371 1362 1362 1373 1380
```

Con dos varianzas probadas a la vez en la frontera, la distribución nula es una mezcla de χ^2_0, χ^2_1 y χ^2_2; la prueba con χ^2_2 que se muestra es conservadora.

La misma sintaxis sirve para diseños de apareamiento como el Carolina Norte I y II, el dialelo sin recíprocos (método 4 de Griffing) o las cruzas de prueba (*top-crosses*), cambiando qué términos se incluyen. Si los progenitores tienen pedigree o marcadores, `mm()` no se combina directamente con `vm()` en esta versión. La solución es construir la matriz de covarianza de la ACG a nivel de cruza: si $\mathbf{Z}_{mm}$ es la matriz de pertenencia múltiple de las cruzas ordenadas y $\mathbf{K}$ la matriz de relaciones de los progenitores, $\mathrm{Var}(g_i + g_j)$ entre cruzas es $\sigma^2_{ACG}\,\mathbf{Z}_{mm}\mathbf{K}\mathbf{Z}_{mm}'$, y el término se escribe `vm(cruza_ord, Kc)` con `Kc` igual a esa matriz:

```
cr <- levels(dt$cruza_ord)
p1 <- sub(">.*", "", cr); p2 <- sub(".*>", "", cr)
Zmm <- outer(p1, rownames(K), "==") + outer(p2, rownames(K), "==")
Kc <- Zmm %*% K %*% t(Zmm); dimnames(Kc) <- list(cr, cr)
mlreml(day ~ anio + anio:block, random = ~ vm(cruza_ord, Kc) + cruza, data =
dt)
```

12 Ejemplo 9: ensayo no replicado de generación temprana

12.1 Los datos

En las primeras etapas de un programa de mejoramiento se evalúan cientos de líneas nuevas con una sola parcela cada una, intercaladas con variedades testigo repetidas que sirven para estimar el error y la variación espacial. Cullis, Smith y Coombes (2006) mostraron que el análisis espacial es especialmente valioso en estos ensayos, porque sin repeticiones no hay otra forma de separar el mérito genético de la fertilidad de la parcela. Los datos `cullis.earlygen` (en `agridat`) provienen de ese trabajo: 670 parcelas de trigo en 67 filas por 10 columnas, con 532 genotipos. La mayoría aparece una sola vez; los testigos se repiten. Para cada parcela se registró también una calificación de malezas.

```
data(cullis.earlygen)
eg <- cullis.earlygen
c(parcelas = nrow(eg), genotipos = nlevels(eg$gen), filas = max(eg$row),
columnas = max(eg$col))

##  parcelas genotipos     filas  columnas
##       670       532        67        10

rep_gen <- table(eg$gen)
table(repeticiones = rep_gen)

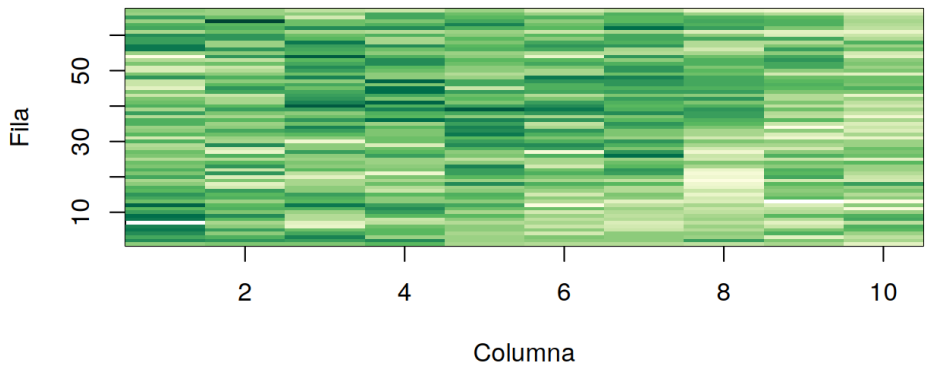
## repeticiones
##   1   3   4   5 121
## 525   1   4   1   1

head(sort(rep_gen, decreasing = TRUE), 6)

##
##    Kite   Banks  Hartog   K1056 Rosella  Vulcan
##     121       5       4       4       4       4

eg$testigo <- factor(ifelse(rep_gen[as.character(eg$gen)] > 1, "testigo",
"linea"))

Ym <- tapply(eg$yield, list(eg$col, eg$row), mean)
image(1:10, 1:67, Ym, col = hcl.colors(30, "YlGn", rev = TRUE), xlab =
"Columna", ylab = "Fila")
```



Rendimiento observado en las 670 parcelas (67 filas por 10 columnas).

12.2 Modelos

Se comparan tres modelos con los mismos efectos fijos (media, efecto de malezas como covariable y diferencia media entre testigos y líneas nuevas) y efectos genéticos aleatorios:

1. residuos independientes;
2. residuo autorregresivo separable AR1 × AR1;
3. residuo AR1 × AR1 más un *nugget* independiente.

```
e1 <- mlreml(yield ~ weed + testigo, random = ~ gen, data = eg, trace = FALSE)
e2 <- mlreml(yield ~ weed + testigo, random = ~ gen, residual = ~
ar1(row):ar1(col),
             data = eg, trace = FALSE)
```

```
e3 <- mlreml(yield ~ weed + testigo, random = ~ gen + units, residual = ~
ar1(row):ar1(col),
             data = eg, trace = FALSE)
data.frame(modelo = c("independiente", "AR1 x AR1", "AR1 x AR1 + nugget"),
           parametros = sapply(list(e1, e2, e3), function(f) sum(f$free)),
           logLik = sapply(list(e1, e2, e3), `[[`, "loglik"),
           AIC = sapply(list(e1, e2, e3), `[[`, "AIC"))
```

```
##               modelo parametros logLik  AIC
## 1      independiente          2  -4897 9798
## 2          AR1 x AR1          4  -4840 9688
## 3 AR1 x AR1 + nugget          5  -4833 9676
```

```
summary(e3)
```

```
## MLREML fit (AI-REML)
## Observations: 668   Fixed-effect columns: 3   Iterations: 7 (converged)
## REML log-likelihood: -4832.9955   AIC: 9675.99   BIC: 9698.49
##
## Variance parameters:
##                               estimate std.error z.ratio bound
## gen!sigma2                   7.237e+04 1.046e+04    6.92     P
## units!sigma2                 3.274e+04 7.974e+03    4.11     P
## R!ar1(row):ar1(col)!sigma2  5.704e+04 1.123e+04    5.08     P
## R!ar1(row):ar1(col)!row_cor 8.561e-01 3.998e-02   21.41     P
## R!ar1(row):ar1(col)!col_cor 4.027e-01 1.137e-01    3.54     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
wald(e3)
```

```
##          term df  wald.inc     F.inc ddf.inc     p.inc  wald.con     F.con
ddf.con
## 1 (Intercept)  1 3.521e+03 3.946e+03   10.93 2.480e-15 3.521e+03 3.946e+03
10.93
## 2        weed  1 8.288e+01 7.938e+01  496.09 9.735e-18 8.289e+01 7.940e+01
495.97
## 3       testigo  1 1.743e-02 1.737e-02  164.50 8.953e-01 1.743e-02 1.737e-02
164.50
##       p.con   p.chisq
## 1 2.480e-15 0.000e+00
## 2 9.693e-18 8.656e-20
## 3 8.953e-01 8.950e-01
```

Los modelos espaciales mejoran claramente la verosimilitud respecto al modelo de residuos independientes. El efecto de las malezas es significativo y negativo, un ejemplo de variación extraña (*extraneous*) que conviene modelar con covariables cuando se ha medido.

12.3 Consecuencias para la selección

El objetivo de un ensayo de generación temprana es seleccionar la fracción superior de las líneas. La pregunta práctica es si el modelo cambia qué líneas se seleccionan. Se comparan los BLUP de las líneas nuevas bajo el modelo independiente y bajo el modelo espacial preferido por el AIC:

```
b1 <- coef(e1, "random", "gen"); bb <- coef(eb, "random", "gen")
lineas <- names(rep_gen)[rep_gen == 1]
```

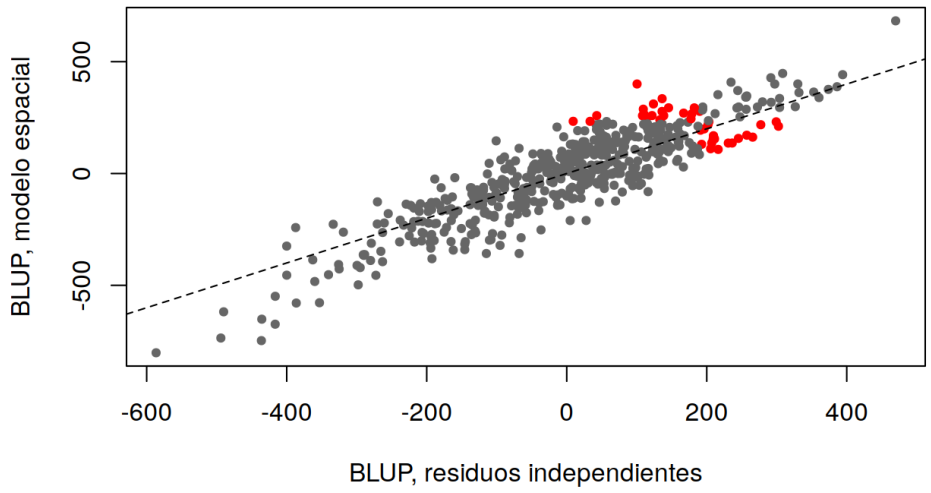
```r
u1 <- setNames(b1$blup, b1$level)[lineas]
ub <- setNames(bb$blup, bb$level)[lineas]
cor(u1, ub, method = "spearman")
```

```
## [1] 0.8613
```

```r
k <- round(0.1 * length(lineas))
sel1 <- names(sort(u1, decreasing = TRUE))[1:k]
selb <- names(sort(ub, decreasing = TRUE))[1:k]
c(seleccionadas = k, coincidencias = length(intersect(sel1, selb)))
```

```
## seleccionadas coincidencias
##            52            32
```

```r
solo_una <- setdiff(union(sel1, selb), intersect(sel1, selb))
plot(u1, ub, pch = 20, col = ifelse(lineas %in% solo_una, "red", "grey40"),
     xlab = "BLUP, residuos independientes", ylab = "BLUP, modelo espacial")
abline(0, 1, lty = 2)
```



BLUP de las líneas nuevas con residuos independientes frente al modelo espacial. Los puntos rojos son las líneas seleccionadas (10 % superior) solo por uno de los dos modelos.

Una parte importante de las líneas seleccionadas cambia según el modelo. Como el modelo espacial describe mejor los datos, sus predicciones son más confiables, lo que se refleja también en la confiabilidad media de los BLUP y en la heredabilidad generalizada:

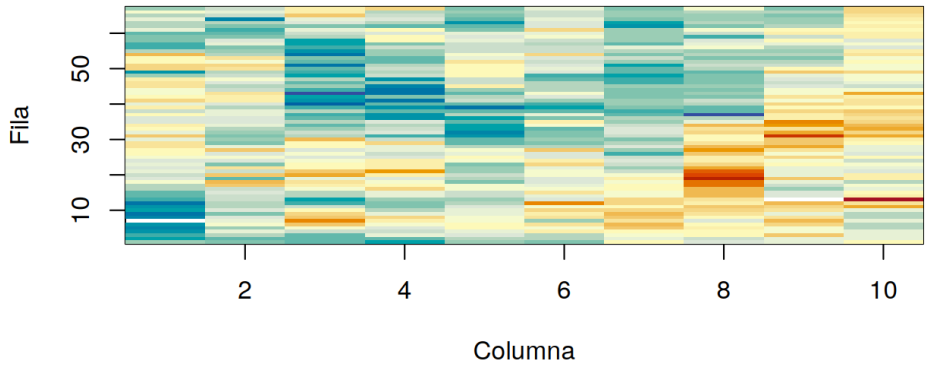
```r
H2c <- function(f) {
  pr <- predict(f, "gen", sed = TRUE)
  1 - pr$avsed^2 / (2 * f$theta[1])
}
data.frame(modelo = c("independiente", "espacial"),
           confiabilidad_media = c(mean(b1$reliability), mean(bb$reliability)),
           H2_Cullis = c(H2c(e1), H2c(eb)))
```

```
##          modelo confiabilidad_media H2_Cullis
## 1 independiente              0.4025    0.4035
## 2      espacial              0.5767    0.5779
```

12.4 Tendencia espacial y testigos

La variación espacial estimada puede visualizarse con los residuos del modelo después de quitar los efectos fijos y genéticos. Los testigos repetidos, distribuidos por todo el ensayo, son los que permiten estimarla.

```
gefe <- bb$blup[match(as.character(eb$data$gen), bb$level)]
esp <- eb$y - eb$fixed_fitted - gefe
Em <- tapply(esp, list(eb$data$col, eb$data$row), mean)
image(1:10, 1:67, Em, col = hcl.colors(30, "RdYlBu"), xlab = "Columna", ylab =
"Fila")
```



Componente espacial estimado (rendimiento menos efectos fijos y genéticos).

Este ejemplo muestra el patrón de trabajo que Cullis et al. (2006) recomiendan para ensayos no replicados: modelar la variación espacial y los factores extraños medidos, tratar los genotipos como aleatorios y basar la selección en los BLUP del modelo con mejor ajuste.

13 Uso avanzado y solución de problemas

Este capítulo reúne técnicas que no corresponden a un tipo de experimento en particular: cómo inspeccionar la convergencia, cómo usar valores iniciales y parámetros fijos, cómo obtener intervalos de confianza basados en la verosimilitud, qué hacer con estimaciones en la frontera y con matrices mal condicionadas, y cómo reducir el consumo de memoria. Termina con una tabla de problemas frecuentes y sus soluciones.

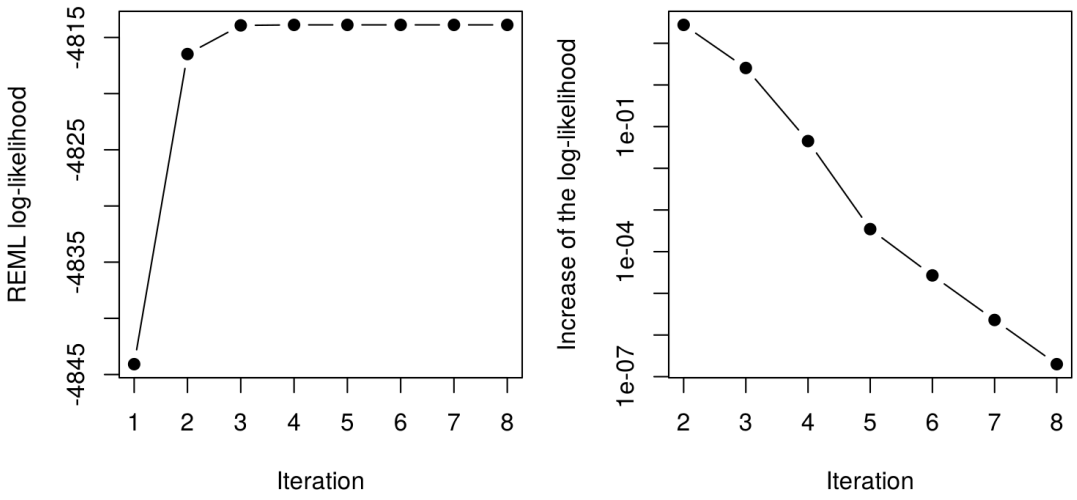
13.1 Historia de iteraciones

El elemento `history` del objeto ajustado guarda la log-verosimilitud de cada iteración y el factor de amortiguamiento λ usado. Un λ igual a cero indica un paso AI puro; valores positivos indican que el paso puro no era válido o no mejoraba la verosimilitud y hubo que acortarlo.

```
d <- agridat::besag.met
fh <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
             residual = ~ diag(county):units, data = d, trace = FALSE, maxit =
100)
head(fh$history, 12)
```

```
##   iteration loglik lambda
## 1         1  -4844      0
## 2         2  -4816      0
## 3         3  -4814      0
## 4         4  -4814      0
## 5         5  -4814      0
## 6         6  -4814      0
## 7         7  -4814      0
## 8         8  -4814      0
```

```
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
plot(fh$history$iteration, fh$history$loglik, type = "b", pch = 19,
     xlab = "Iteración", ylab = "log-verosimilitud REML")
inc <- diff(fh$history$loglik)
plot(seq_along(inc) + 1, pmax(inc, 1e-8), log = "y", type = "b", pch = 19,
     xlab = "Iteración", ylab = "Aumento de la log-verosimilitud")
```



Historia de la log-verosimilitud REML (izquierda) y aumento de la verosimilitud por iteración en escala logarítmica (derecha) para el modelo FA(1).

```
par(op)
```

Cerca del óptimo el algoritmo AI converge de forma casi cuadrática: el aumento de la verosimilitud cae varios órdenes de magnitud por iteración. Aunque tres varianzas específicas de este modelo están en la frontera, el tratamiento de la frontera por conjunto activo (Sección 2.4.1) evita que bloqueen el paso conjunto, y el modelo converge en 8 iteraciones sin amortiguamiento. Una caída lenta y regular de los aumentos indica una verosimilitud plana en alguna dirección.

13.2 Valores iniciales y reinicio

Cuando un modelo complejo no converge, conviene reiniciarlo desde los valores finales del intento anterior o desde las estimaciones de un modelo más simple. Los valores iniciales se dan como un vector con nombres:

```
ini <- fh$theta                                  # estimaciones del FA(1)
fh2 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
              residual = ~ diag(county):units, data = d, trace = FALSE, start =
ini)
c(iteraciones = fh2$iterations, logLik = fh2$loglik, convergio = fh2$converged)
```

```
## iteraciones      logLik   convergio
##           1       -4814           1
```

Al iniciar en el óptimo, el algoritmo confirma la convergencia en una o dos iteraciones. Esta es también la forma de verificar que un ajuste ha convergido de verdad.

Para usar un modelo simple como punto de partida de uno complejo, se copian los parámetros comunes. Por ejemplo, las varianzas de un modelo diagonal sirven como varianzas específicas iniciales de un FA:

```
fd <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
diag(county):id(gen),
             residual = ~ diag(county):units, data = d, trace = FALSE)
vg <- fd$theta[grep("diag\\(county\\):id\\(gen\\)", names(fd$theta))]
names(vg) <- sub("diag\\(county\\):id\\(gen\\)!county_", "fa(county,
1):id(gen)!county_var_", names(vg))
head(vg, 3)
```

```
## fa(county, 1):id(gen)!county_var_C1 fa(county, 1):id(gen)!county_var_C2
##                               57.41                               19.29
## fa(county, 1):id(gen)!county_var_C3
##                               94.35
```

Ese vector puede pasarse a `start` (solo para los parámetros que se quieran inicializar; el resto toma los valores por defecto).

13.3 Parámetros fijos y verosimilitud perfil

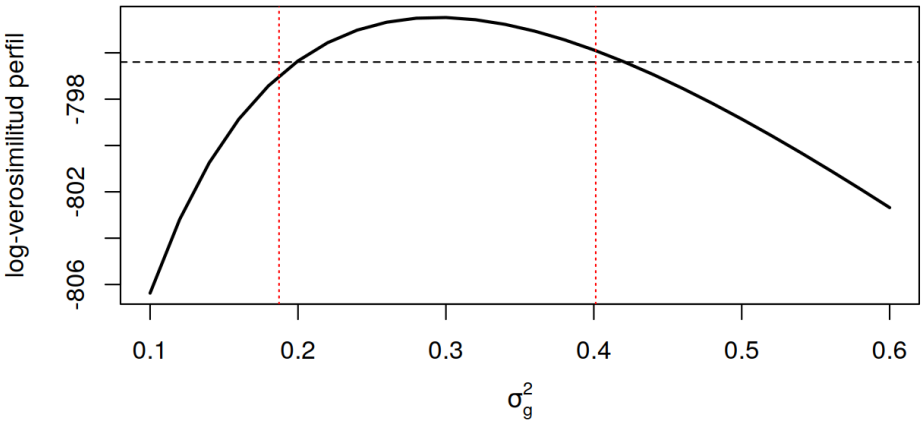
Con `fix` se mantienen parámetros en sus valores iniciales. Una aplicación importante es la **verosimilitud perfil**: se fija un parámetro en una rejilla de valores, se estiman los demás y se grafica la verosimilitud máxima resultante. El intervalo de confianza al 95 % basado en la verosimilitud contiene los valores cuya log-verosimilitud perfil está a menos de $\chi^2_{1,0.95}/2 = 1.92$ unidades del máximo. A diferencia del intervalo de Wald (estimación ± 1.96 errores estándar), no es necesariamente simétrico y nunca incluye valores negativos de una varianza.

Usamos el modelo GBLUP del Capítulo 7 para la varianza genética del ambiente E1:

```
f0 <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, trace = FALSE)
nm <- "vm(gen, G)!sigma2"
grid <- seq(0.10, 0.60, by = 0.02)
prof <- sapply(grid, function(s) {
  mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, start = setNames(s, nm), fix
= nm,
         trace = FALSE, pev = FALSE)$loglik
})
lim <- range(grid[prof >= max(prof) - qchisq(0.95, 1) / 2])
est <- f0$theta[nm]; se <- f0$varcomp[nm, "std.error"]
rbind(perfil = lim, wald = est + c(-1, 1) * 1.96 * se)
```

```
##          [,1]   [,2]
## perfil 0.2000 0.4200
## wald   0.1872 0.4013
```

```
plot(grid, prof, type = "l", lwd = 2, xlab = expression(sigma[g]^2), ylab =
"log-verosimilitud perfil")
abline(h = max(prof) - qchisq(0.95, 1) / 2, lty = 2)
abline(v = est + c(-1, 1) * 1.96 * se, lty = 3, col = "red")
```



Log-verosimilitud perfil de la varianza genética (GBLUP, ambiente E1). La línea horizontal marca el umbral del intervalo al 95 %; las verticales, el intervalo de Wald.

El mismo procedimiento sirve para cualquier parámetro, incluidas correlaciones genéticas y parámetros autorregresivos, y es la forma recomendada de obtener intervalos para parámetros cercanos a la frontera.

13.4 Componentes de varianza conocidos

Si los componentes de varianza se conocen (por ejemplo, estimados en un conjunto de datos grande o publicados), se pueden fijar todos y obtener solo los BLUE y BLUP. Así se predicen nuevas generaciones con parámetros estables y sin reestimarlos cada vez:

```
vc_conocidos <- c("vm(gen, G)!sigma2" = 0.30, "R!units!sigma2" = 0.55)
fk <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, start = vc_conocidos,
             fix = names(vc_conocidos), trace = FALSE)
summary(fk)$varcomp

##                    estimate std.error z.ratio bound
## vm(gen, G)!sigma2     0.30        NA      NA     F
## R!units!sigma2        0.55        NA      NA     F

cor(fk$blups[[1]]$blup, f0$blups[[1]]$blup)

## [1] 1
```

13.5 Efectos fijos confundidos

Las columnas de la matriz de efectos fijos que son combinación lineal de otras se eliminan automáticamente, como los efectos *aliased* de ASReml. Por ejemplo, si por error se incluye dos veces el mismo factor con otro nombre, o un factor que es función de otro (aquí, una agrupación de las repeticiones en dos campos), las columnas redundantes se descartan:

```
ja <- agridat::john.alpha
ja$campo <- factor(ifelse(ja$rep == "R1", "A", "B"))      # función de rep
fa_alias <- mlreml(yield ~ rep + campo, random = ~ gen, data = ja, trace =
FALSE)
c(columnas_posibles = ncol(model.matrix(~ rep + campo, ja)),
columnas_estimables = fa_alias$p)
```

```
##   columnas_posibles columnas_estimables
##                   4                   3
```

```
colnames(fa_alias$X)
```

```
## [1] "(Intercept)" "repR2"       "repR3"
```

13.6 Parámetros en la frontera

Una varianza estimada en su límite inferior (código `B`) significa que la verosimilitud es máxima en el borde del espacio de parámetros. Las causas habituales son que el efecto realmente es pequeño, que el diseño no permite separarlo de otros efectos, o que el modelo está sobreparametrizado. En esos casos:

1. el error estándar reportado no es válido;
2. la prueba de razón de verosimilitudes debe usar la mezcla de distribuciones χ^2 (`lrt(..., boundary = TRUE)`);
3. conviene considerar eliminar el término, o reemplazar una estructura compleja (`us`) por una más parsimoniosa (`fa`, `diag`).

Una varianza en su límite cuyo score es negativo se mantiene en el límite y se elimina del sistema AI (conjunto activo), de modo que los demás parámetros convergen a su óptimo dada la frontera; una matriz no estructurada que converge a una matriz singular (correlaciones de ± 1) detiene el algoritmo con una advertencia cuando los pasos se vuelven despreciables.

En modelos factor analíticos, las varianzas específicas en cero (casos Heywood) son frecuentes y no impiden interpretar el modelo.

13.7 Matrices de relación mal condicionadas

El motor denso de MLREML no invierte las matrices de relación, de modo que una matriz singular es aceptable mientras **V** sea definida positiva; el motor disperso necesita la inversa y, por lo tanto, una matriz definida positiva. Sin embargo, una matriz con valores propios negativos (por ejemplo, una matriz **G** calculada con datos faltantes imputados, o una **H** mal construida) no es una matriz de covarianza válida. La función `bend()` la corrige reemplazando los valores propios pequeños o negativos:

```
set.seed(1)
Kbad <- cor(matrix(rnorm(30 * 20), 20))          # 30 x 30 con rango 20
Kbad[1, 2] <- Kbad[2, 1] <- 0.99                  # perturbación que la hace
indefinida
range(eigen(Kbad, only.values = TRUE)$values)
```

```
## [1] -0.7215  4.4667
```

```
Kok <- bend(Kbad, tol = 1e-4)
range(eigen(Kok, only.values = TRUE)$values)
```

```
## [1] 0.0004467 4.4667035
```

```
max(abs(Kok - Kbad))
```

```
## [1] 0.284
```

13.8 Escala de las variables

El algoritmo es invariante a la escala en teoría, pero en la práctica la convergencia es más robusta cuando las respuestas tienen varianzas del orden de la unidad. En modelos multivariados con caracteres en escalas muy distintas (por ejemplo, kilogramos de leche y porcentaje de proteína) conviene reescalar cada carácter, como se hizo en el Capítulo 8 al expresar la leche en toneladas y la grasa en cientos de kilogramos. Las covarianzas en la escala original se recuperan multiplicando por los factores de escala.

13.9 Memoria y tiempo

- `engine = "sparse"` (Sección 2.12) evita por completo las matrices $n \times n$; con más de 5,000 registros se elige automáticamente cuando el modelo es admitido.
- `pev = FALSE` omite el cálculo de las varianzas del error de predicción, que es la parte más costosa después del ajuste cuando un término tiene muchos niveles (por ejemplo, todos los animales del pedigree).
- `store = FALSE` no guarda las matrices $n \times n$ que necesita `predict()` y el ajuste de Kenward–Roger de `wald()` (con el motor disperso, el factor de Cholesky); reduce mucho el tamaño del objeto ajustado.
- Dentro de un análisis largo (validación cruzada, simulaciones) conviene eliminar los objetos ajustados que ya no se usan y llamar a `gc()`.

13.10 Problemas frecuentes

Problemas frecuentes y soluciones.

Mensaje o síntoma	Causa probable	Solución
`levels of 'x' missing from the vm() matrix`	Hay niveles del factor que no están en los nombres de fila de la matriz.	Revisar identificadores (espacios, mayúsculas, prefijos); usar `setdiff()` para encontrarlos.
`the matrix given to vm() must have rownames`	La matriz de relaciones no tiene nombres.	Asignar `dimnames(K) <- list(ids, ids)`.
`term ... has more than one variance-type component`	Dos estructuras de varianza en un término, por ejemplo `us(env):diag(gen)`.	Dejar solo una estructura de varianza; la otra debe ser de correlación (`id`, `vm`, `ar1`...).
`AI-REML did not converge`	Verosimilitud plana, modelo sobreparametrizado o malos valores iniciales.	Aumentar `maxit`, reiniciar con `start = fit$theta`, simplificar el modelo.
`no improving step found`	El algoritmo no encuentra un paso que aumente la verosimilitud, normalmente en la frontera.	Revisar parámetros con código `B`; probar otros valores iniciales.
`AI-REML stopped because the steps became negligible although the score is not zero`	Una matriz no estructurada (`us`, `str()`) converge a una matriz singular.	Revisar las correlaciones (± 1); usar `fa`, `diag` o `str(..., cov = "diag")`.
Varianza con código `B` y error estándar enorme	Parámetro en la frontera.	Interpretar como cero; usar LRT con mezcla; considerar eliminar el término.
Correlación genética igual	La matriz `us` está en la frontera del	Normal con pocos datos;

Mensaje o síntoma	Causa probable	Solución
a ± 1	cono semidefinido.	considerar un modelo `fa`.
Error en `chol()` o memoria insuficiente	n demasiado grande para matrices densas.	Usar `engine = "sparse"` (Sección 2.12) o ver las estrategias del Capítulo 15.
`the sparse engine does not support this model (...)`	El modelo tiene una estructura sin implementación dispersa (`cs`, `ar1`, `expc`, `sexp`, `fa`) o un residuo no admitido.	Usar `engine = "dense"`, o una estructura admitida.
`the relationship matrix is not positive definite; use bend() or give its inverse`	Motor disperso con una matriz de relación singular.	Corregir (bending) o combinar la matriz (Sección 13.7), dar su inversa, o usar el motor denso.
`REML likelihoods are only comparable with identical fixed effects`	Se compararon modelos con efectos fijos distintos en `lrt()`.	Comparar solo modelos con la misma parte fija; usar Wald para los efectos fijos.

14 Estudio de simulación: propiedades de los estimadores

La comparación con otros programas (Capítulo 15) verifica que MLREML calcula correctamente la verosimilitud y su máximo. Una pregunta distinta es qué tan buenos son los estimadores REML, sus errores estándar y las pruebas en muestras del tamaño que se usa en la práctica. Este capítulo responde con dos simulaciones pequeñas que el lector puede repetir y modificar.

14.1 Diseño de bloques completos con genotipos aleatorios

Se simulan 200 experimentos con 40 genotipos en 3 bloques completos, con $\sigma^2_g = 1$, $\sigma^2_e = 2$ y efectos de bloque fijos. La heredabilidad a nivel de media de genotipo es $H^2 = \sigma^2_g / \left(\sigma^2_g + \sigma^2_e/3\right) = 0.6$.

```
sim_rcbd <- function(seed, ngen = 40, nblk = 3, sg2 = 1, se2 = 2) {
  set.seed(seed)
  dd <- expand.grid(gen = factor(1:ngen), blk = factor(1:nblk))
  dd$y <- c(0, 0.5, -0.5)[dd$blk] + rnorm(ngen, sd = sqrt(sg2))[dd$gen] +
    rnorm(nrow(dd), sd = sqrt(se2))
  f <- mlreml(y ~ blk, random = ~ gen, data = dd, trace = FALSE, pev = FALSE,
store = FALSE)
  h <- vpredict(f, H2 ~ V1 / (V1 + V2 / 3))
  c(sg2 = f$theta[[1]], se2 = f$theta[[2]], se_sg2 = f$varcomp$std.error[1],
    H2 = h$estimate, se_H2 = h$std.error, bound = f$varcomp$bound[1] == "B",
    conv = f$converged)
}
res1 <- as.data.frame(t(sapply(1:200, sim_rcbd)))
c(convergieron = mean(res1$conv), en_frontera = mean(res1$bound))

## convergieron  en_frontera
##            1            0
```

```
verdad <- c(sg2 = 1, se2 = 2, H2 = 0.6)
tab1 <- data.frame(parametro = names(verdad), verdadero = verdad,
                   media = colMeans(res1[names(verdad)]),
                   sesgo_relativo_pct = 100 * (colMeans(res1[names(verdad)]) -
verdad) / verdad,
                   de_empirica = sapply(res1[names(verdad)], sd))
tab1$se_medio <- c(mean(res1$se_sg2), NA, mean(res1$se_H2))
knitr::kable(tab1, digits = 3, row.names = FALSE,
             caption = "Simulación de 200 experimentos de bloques completos:
media, sesgo y variabilidad de los estimadores REML.")
```

Simulación de 200 experimentos de bloques completos: media, sesgo y variabilidad de los estimadores REML.

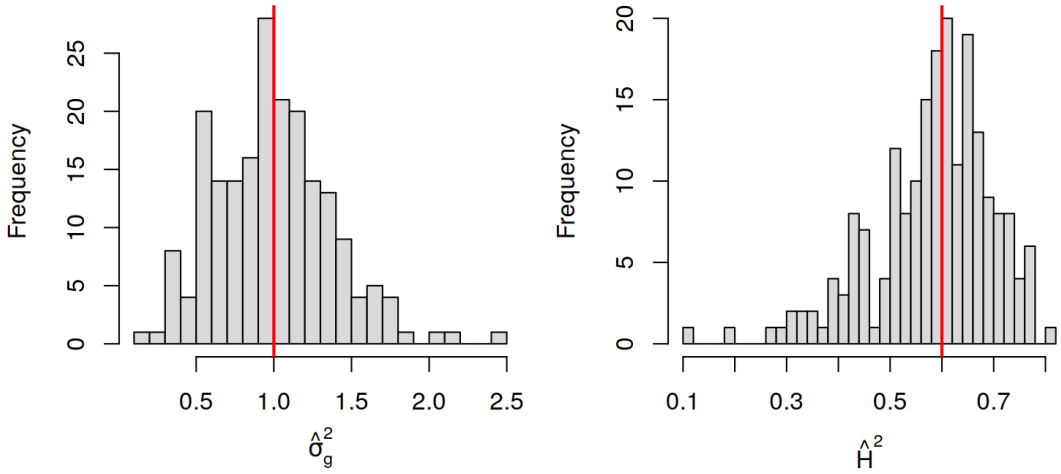
parametro	verdadero	media	sesgo_relativo_pct	de_empirica	se_medio
sg2	1.0	0.997	-0.292	0.378	0.392
se2	2.0	1.990	-0.511	0.326	NA
H2	0.6	0.582	-3.061	0.116	0.116

```
cob <- function(est, se, v) mean(est - 1.96 * se <= v & v <= est + 1.96 * se)
c(cobertura_sg2 = cob(res1$sg2, res1$se_sg2, 1), cobertura_H2 = cob(res1$H2,
res1$se_H2, 0.6))

## cobertura_sg2  cobertura_H2
##          0.95          0.94
```

Los estimadores REML tienen un sesgo pequeño y el error estándar medio obtenido de la matriz de información promedio se parece a la desviación estándar empírica de las estimaciones, lo que indica que la aproximación asintótica es razonable con 40 genotipos. La cobertura de los intervalos de Wald está cerca del 95 % nominal. La distribución de $\hat{\sigma}_g^2$ es asimétrica a la derecha, como corresponde a un estimador de varianza:

```
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
hist(res1$sg2, breaks = 25, col = "grey85", main = "", xlab =
expression(hat(sigma)[g]^2))
abline(v = 1, col = "red", lwd = 2)
hist(res1$H2, breaks = 25, col = "grey85", main = "", xlab =
expression(hat(H)^2))
abline(v = 0.6, col = "red", lwd = 2)
```



Distribución de las estimaciones REML de la varianza genética (izquierda) y de la heredabilidad (derecha) en 200 experimentos simulados. Las líneas verticales marcan los valores verdaderos.

```
par(op)
```

14.2 Tamaño de la prueba de razón de verosimilitudes en la frontera

Para comprobar la corrección por frontera de `lrt()`, se simulan 200 experimentos **sin** varianza genética ($\sigma^2_g = 0$) y se registra la proporción de rechazos de H_0: $\sigma^2_g = 0$ al 5 %, con la χ^2_1 ingenua y con la mezcla 50:50.

```
sim_null <- function(seed) {
  set.seed(seed)
  dd <- expand.grid(gen = factor(1:40), blk = factor(1:3))
  dd$y <- rnorm(nrow(dd), sd = sqrt(2))
  f1 <- mlreml(y ~ blk, random = ~ gen, data = dd, trace = FALSE, pev = FALSE,
store = FALSE)
  f0 <- mlreml(y ~ blk, data = dd, trace = FALSE, pev = FALSE, store = FALSE)
  c(mezcla = lrt(f0, f1, boundary = TRUE)$p.value, ingenua = lrt(f0, f1,
boundary = FALSE)$p.value,
    frontera = f1$varcomp$bound[1] == "B")
}
pv <- as.data.frame(t(sapply(1:200, sim_null)))
colMeans(pv[c("mezcla", "ingenua")] < 0.05)

##  mezcla ingenua
##   0.020   0.015

mean(pv$frontera)

## [1] 0.49
```

En 49 % de las simulaciones la varianza genética se estima en la frontera, cerca de la mitad, como predice la teoría asintótica. En esas simulaciones el estadístico es cero, de modo que ambas versiones de la prueba resultan conservadoras con este tamaño de muestra: la χ^2_1 ingenua rechazó en 1.5 % de los casos y la mezcla 50:50 en 2 %, más cerca del 5 % nominal. La mezcla es la aproximación asintótica correcta; con pocos niveles del factor aleatorio la prueba sigue siendo algo conservadora, de modo que un valor p significativo es una evidencia confiable de varianza genética.

14.3 Modelo espacial

La segunda simulación evalúa el modelo AR1 × AR1 en una cuadrícula de 16 filas por 12 columnas con 48 genotipos en 4 repeticiones, con $\rho_r = 0.7$, $\rho_c = 0.4$, varianza espacial 1, varianza genética 0.5 y *nugget* 0.2.

```
sim_sp <- function(seed, rr = 0.7, rc = 0.4) {
  set.seed(seed)
  dd <- expand.grid(row = 1:16, col = 1:12)
  dd$rep <- factor((dd$col - 1) %/% 3 + 1)
  dd$gen <- factor(unlist(lapply(1:4, function(r) sample(1:48))))
  Rr <- rr^abs(outer(1:16, 1:16, "-")); Rc <- rc^abs(outer(1:12, 1:12, "-"))
  xi <- drop(t(chol(kronecker(Rc, Rr))) %*% rnorm(192))       # filas varían
más rápido
  dd$y <- rnorm(48, sd = sqrt(0.5))[dd$gen] + xi + rnorm(192, sd = sqrt(0.2))
  f <- mlreml(y ~ rep, random = ~ gen + units, residual = ~ ar1(row):ar1(col),
              data = dd, trace = FALSE, pev = FALSE, store = FALSE)
  c(f$theta, conv = f$converged)
}
res3 <- as.data.frame(t(sapply(1:40, sim_sp)))
names(res3) <- c("sg2", "nugget", "s2_espacial", "rho_fila", "rho_col", "conv")
verdad3 <- c(sg2 = 0.5, nugget = 0.2, s2_espacial = 1, rho_fila = 0.7, rho_col
= 0.4)
data.frame(verdadero = verdad3, mediana = sapply(res3[names(verdad3)], median),
           media = colMeans(res3[names(verdad3)]), de =
sapply(res3[names(verdad3)], sd))
```

```
##            verdadero mediana   media      de
## sg2              0.5  0.5232  0.5091 0.12644
## nugget           0.2  0.1568  0.1723 0.11259
## s2_espacial      1.0  1.0209  1.0445 0.28329
## rho_fila         0.7  0.6573  0.6636 0.09491
## rho_col          0.4  0.3723  0.4015 0.14507
## [1] 0.95
```

```
mean(res3$conv)
```

```
## [1] 0.95
```

Convergió el 95 % de los ajustes. Los parámetros autorregresivos y la varianza genética se recuperan bien, con un sesgo pequeño. La separación entre el *nugget* y la varianza espacial es más difícil, como es bien conocido en geoestadística: con una cuadrícula de este tamaño ambos componentes tienen una variabilidad de muestreo considerable, y su suma se estima con mucha más precisión que cada uno por separado.

```
c(media_suma = mean(res3$nugget + res3$s2_espacial), de_suma = sd(res3$nugget +
res3$s2_espacial))
```

```
## media_suma   de_suma
##      1.217      0.290
```

Estas simulaciones son pequeñas a propósito, para que se puedan ejecutar en pocos minutos. Para un estudio formal conviene aumentar el número de repeticiones a 1,000 o más y explorar los tamaños de experimento y los valores de los parámetros que interesen.

15 Validación numérica, rendimiento y limitaciones

15.1 Comparación con otros programas

Un programa de estimación debe dar los mismos resultados que el software establecido cuando ambos ajustan el mismo modelo. La siguiente tabla compara la log-verosimilitud REML de MLREML con la de `lme4` y `nlme` en todos los modelos de esta guía que esos paquetes pueden ajustar, y la función `gblup()` con el motor general. Las pruebas automáticas del paquete (`tests/testthat`) repiten estas comparaciones cada vez que el paquete se verifica.

```
library(lme4); library(nlme)
v <- list()
d <- agridat::john.alpha
f <- mlreml(yield ~ rep, random = ~ gen + rep:block, data = d, trace = FALSE)
v[[1]] <- c("Alfa-látice, dos varianzas", "lme4", f$loglik,
            logLik(lmer(yield ~ rep + (1 | gen) + (1 | rep:block), data = d)))
f <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy, trace = FALSE)
v[[2]] <- c("Regresión aleatoria (us)", "lme4", f$loglik,
            logLik(lmer(Reaction ~ Days + (Days | Subject), sleepstudy)))
f <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
diag(Sex):units, data = ort,
            trace = FALSE)
v[[3]] <- c("Residuo heterogéneo (diag)", "nlme::lme", f$loglik,
            logLik(lme(distance ~ age * Sex, random = ~ 1 | Subject,
                       weights = varIdent(form = ~ 1 | Sex), data = ort)))
f <- mlreml(distance ~ age * Sex, residual = ~ ar1(age):id(Subject), data =
ort, trace = FALSE)
v[[4]] <- c("Residuo autorregresivo (ar1)", "nlme::gls", f$loglik,
            logLik(gls(distance ~ age * Sex, correlation = corExp(form = ~ age
| Subject), data = ort)))
f <- mlreml(distance ~ age * Sex, residual = ~ us(age):id(Subject), data = ort,
trace = FALSE)
v[[5]] <- c("Residuo no estructurado (us)", "nlme::gls", f$loglik,
            logLik(gls(distance ~ age * Sex, correlation = corSymm(form = ~ 1 |
Subject),
                       weights = varIdent(form = ~ 1 | age), data = ort)))
v[[6]] <- c("GBLUP (vm)", "gblup() espectral", f1$loglik, g1$loglik)
tab <- as.data.frame(do.call(rbind, v))
names(tab) <- c("modelo", "referencia", "logLik_MLREML", "logLik_referencia")
tab$logLik_MLREML <- as.numeric(tab$logLik_MLREML)
tab$logLik_referencia <- as.numeric(tab$logLik_referencia)
tab$diferencia <- signif(tab$logLik_MLREML - tab$logLik_referencia, 2)
knitr::kable(tab, digits = 4, caption = "Log-verosimilitud REML de MLREML y de
otros programas.")
```

Log-verosimilitud REML de MLREML y de otros programas.

modelo	referencia	logLik_MLREML	logLik_referencia	diferencia
Alfa-látice, dos varianzas	lme4	-46.6	-46.6	0
Regresión aleatoria (us)	lme4	-871.8	-871.8	0
Residuo heterogéneo (diag)	nlme::lme	-207.6	-207.6	0
Residuo autorregresivo	nlme::gls	-222.3	-222.3	0

modelo	referencia	logLik_MLREML	logLik_referencia	diferencia
(ar1)				
Residuo no estructurado (us)	nlme::gls	-212.3	-212.3	0
GBLUP (vm)	gblup() espectral	-794.5	-794.5	0

Las diferencias son del orden de la tolerancia de convergencia. Los componentes de varianza, los efectos fijos, los BLUP y los SED también coinciden (Capítulos 4 y 10). Para las estructuras que ningún paquete de R de uso general ajusta (AR1 × AR1 con *nugget*, factor analítico, términos genéticos con estructura no estructurada entre ambientes), la prueba automática verifica que el punto encontrado es un máximo: perturbar cualquier parámetro reduce la verosimilitud.

Las herramientas de pedigree se comparan con `pedigreemm`:

```
Fp <- pedigreemm::inbreeding(pedCowsR)
c(max_dif_consanguinidad = max(abs(Fp - Fi[p@label])))

## max_dif_consanguinidad
##                      0

s <- sample(p@label, 300)
A <- Amatrix(pp)
Ap <- as.matrix(pedigreemm::getA(pedCowsR))[s, s]
c(max_dif_A = max(abs(Ap - A[s, s])))

## max_dif_A
##  1.11e-16

rm(A); invisible(gc())
```

Comparaciones adicionales. El siguiente código verifica otras capacidades: (1) la correlación exponencial con distancias menores que uno frente a `nlme::gls()`, con la que MLREML alcanza el mismo máximo; (2) los estadísticos F de Kenward–Roger y los grados de libertad del denominador de las hipótesis que prueba `wald()` frente a los de `pbkrtest`; y (3) el motor disperso frente al motor denso.

```
## 1. correlación exponencial con distancias menores que uno
set.seed(7)
dx <- data.frame(t = rep(c(0, 0.3, 0.7, 1.2, 2), 20), id = factor(rep(1:20,
each = 5)))
dx$y <- 0.5 * dx$t + unlist(lapply(1:20, function(i) cumsum(rnorm(5)) / 2))
fx <- mlreml(y ~ t, residual = ~ expc(t):id(id), data = dx, trace = FALSE)
gx <- gls(y ~ t, correlation = corExp(form = ~ t | id), data = dx)
c(MLREML = fx$loglik, nlme = as.numeric(logLik(gx)))

## MLREML   nlme
## -84.21 -84.21

c(rho_MLREML = unname(fx$theta[2]), rho_nlme = unname(exp(-1 /
coef(gx$modelStruct$corStruct, FALSE))))

## rho_MLREML   rho_nlme
##     0.7511     0.7511
```

```r
## 2. Pruebas F de Kenward-Roger frente a pbkrtest
kr_row <- function(label, f, l, term) {
  j <- which(f$assign == match(term, attr(f$terms_fixed, "term.labels")))
  L <- chol(solve(vcov(f)))[j, , drop = FALSE]        # hipótesis incremental
de wald()
  w <- wald(f, ddf = "KR"); w <- w[w$term == term, ]
  pk <- KRmodcomp(l, L)$stats
  data.frame(modelo = label, termino = term, F_MLREML = w$F.inc, F_pbkrtest =
pk$Fstat,
             ddf_MLREML = w$ddf.inc, ddf_pbkrtest = pk$ddf)
}
ja <- agridat::john.alpha
oats <- as.data.frame(nlme::Oats)
oats$nitro <- factor(oats$nitro)
oats$Block <- factor(as.character(oats$Block)); oats$Variety <-
factor(as.character(oats$Variety))
f1 <- mlreml(yield ~ rep + gen, random = ~ rep:block, data = ja, trace = FALSE,
tol = 1e-10)
l1 <- lmer(yield ~ rep + gen + (1 | rep:block), data = ja)
f2 <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy,
             trace = FALSE, tol = 1e-10)
l2 <- lmer(Reaction ~ Days + (Days | Subject), sleepstudy)
f3 <- mlreml(yield ~ Variety * nitro, random = ~ Block + Block:Variety, data =
oats,
             trace = FALSE, tol = 1e-10)
l3 <- lmer(yield ~ Variety * nitro + (1 | Block) + (1 | Block:Variety), oats)
kr <- rbind(kr_row("Alfa-látice", f1, l1, "rep"), kr_row("Alfa-látice", f1, l1,
"gen"),
            kr_row("Regresión aleatoria", f2, l2, "Days"),
            kr_row("Parcelas divididas (Oats)", f3, l3, "Variety"),
kr_row("Parcelas divididas (Oats)", f3, l3, "nitro"),
            kr_row("Parcelas divididas (Oats)", f3, l3, "Variety:nitro"))
knitr::kable(kr, digits = 4, row.names = FALSE,
             caption = "Estadísticos F de Kenward-Roger y grados de libertad
del denominador de MLREML y pbkrtest.")
```

Estadísticos F de Kenward-Roger y grados de libertad del denominador de MLREML y pbkrtest.

modelo	termino	F_MLREML	F_pbkrtest	ddf_MLREML	ddf_pbkrtest
Alfa-látice	rep	9.2124	9.2124	11.52	11.52
Alfa-látice	gen	5.3628	5.3628	35.50	35.50
Regresión aleatoria	Days	45.8530	45.8530	17.00	17.00
Parcelas divididas (Oats)	Variety	1.4853	1.4853	10.00	10.00
Parcelas divididas (Oats)	nitro	37.6856	37.6857	45.00	45.00
Parcelas divididas (Oats)	Variety:nitro	0.3028	0.3028	45.00	45.00

Los estadísticos F y los grados de libertad del denominador coinciden con los de `pbkrtest` en cinco o más dígitos significativos. En el diseño de parcelas divididas el método reproduce los grados de libertad clásicos de los errores de parcela grande (10) y de subparcela (45), y $\lambda = 1$.

```
## 3. motor disperso frente al motor denso
both <- function(label, ...) {
  fd <- mlreml(..., trace = FALSE, engine = "dense")
  fs <- mlreml(..., trace = FALSE, engine = "sparse")
  bd <- do.call(rbind, fd$blups); bs <- do.call(rbind, fs$blups)
  f <- function(x) sprintf("%.1e", x)
  data.frame(modelo = label, logLik = round(fd$loglik, 2), logLik_diff =
f(fd$loglik - fs$loglik),
             theta_reldiff = f(max(abs(fd$theta - fs$theta) /
pmax(abs(fd$theta), 1e-8))),
             blup_diff = f(max(abs(bd$blup - bs$blup))), pev_diff =
f(max(abs(bd$pev - bs$pev))))
}
sp <- rbind(
  both("Alfa-látice (dos varianzas)", yield ~ rep, random = ~ gen + rep:block,
data = ja),
  both("Regresión aleatoria us(leg)", Reaction ~ Days, random = ~ us(leg(Days,
1)):id(Subject),
       data = sleepstudy),
  both("Residuo heterogéneo diag(Sex):units", distance ~ age * Sex, random = ~
Subject,
       residual = ~ diag(Sex):units, data = ort),
  both("Parcelas divididas (dos varianzas)", yield ~ Variety * nitro, random =
~ Block + Block:Variety,
       data = oats))
knitr::kable(sp, row.names = FALSE, align = "lrrrrr",
             caption = "Diferencias entre los motores denso y disperso (log-
verosimilitud, máxima diferencia relativa de los parámetros de varianza,
máximas diferencias absolutas de BLUP y PEV).")
```

Diferencias entre los motores denso y disperso (log-verosimilitud, máxima diferencia relativa de los parámetros de varianza, máximas diferencias absolutas de BLUP y PEV).

modelo	logLik	logLik_diff	theta_reldiff	blup_diff	pev_diff
Alfa-látice (dos varianzas)	-46.6	-1.8e-12	2.8e-15	6.7e-15	8.3e-17
Regresión aleatoria us(leg)	-871.8	3.0e-12	1.3e-14	4.0e-13	3.7e-12
Residuo heterogéneo diag(Sex):units	-207.6	6.2e-12	6.8e-15	1.8e-13	5.9e-15
Parcelas divididas (dos varianzas)	-264.5	-1.3e-12	1.2e-14	6.3e-13	7.1e-13

Los dos motores dan los mismos resultados salvo errores de redondeo. Además, el término directo–materno se verificó frente a una maximización directa de la verosimilitud (2.6), con $\mathbf{V}$ construida a partir de (2.18) (las estimaciones coinciden hasta la precisión del optimizador y los BLUP y PEV hasta 10^{-13}), y las pruebas automáticas del paquete verifican que, con la covarianza fijada en cero, reproduce el modelo con dos términos independientes `vm(animal, A) + vm(dam, A)`.

15.2 Tiempo de cómputo y memoria

El costo del motor denso de MLREML está dominado por las operaciones con la matriz densa $\mathbf{V}$ de $n \times n$. Para medirlo se ajustó un modelo GBLUP con datos simulados de distintos tamaños, en la misma computadora (un núcleo, OpenBLAS):

```
tiempos <- do.call(rbind, lapply(c(500, 1000, 2000, 3000), function(n) {
  set.seed(n)
  Msim <- matrix(rbinom(n * 1000, 2, 0.3), n); rownames(Msim) <- paste0("i",
```

```
1:n)
  Ks <- Gmatrix(Msim) + diag(1e-3, n)
  ds <- data.frame(id = rownames(Msim), y = drop(Msim %*% rnorm(1000, 0, 0.05))
+ rnorm(n))
  t1 <- system.time(fs <- mlreml(y ~ 1, random = ~ vm(id, Ks), data = ds, trace
= FALSE))[3]
  t2 <- system.time(gblup(setNames(ds$y, ds$id), Ks))[3]
  data.frame(n = n, iteraciones = fs$iterations, seg_mlreml = t1,
             seg_por_iteracion = t1 / (fs$iterations + 2), seg_gblup = t2,
             memoria_V_GB = round(3 * 8 * n^2 / 1e9, 2))
}))
knitr::kable(tiempos, digits = 2, row.names = FALSE,
             caption = "Tiempo de ajuste de un modelo GBLUP según el número de
observaciones.")
```

Tiempo de ajuste de un modelo GBLUP según el número de observaciones.

n	iteraciones	seg_mlreml	seg_por_iteracion	seg_gblup	memoria_V_GB
500	4	0.13	0.02	0.04	0.01
1000	3	0.50	0.10	0.23	0.02
2000	4	3.61	0.60	1.37	0.10
3000	3	8.74	1.75	22.64	0.22

El tiempo por iteración crece aproximadamente con n^3 y la memoria con n^2 (la columna de memoria estima tres matrices $n \times n$ en doble precisión: **V**, **P** y una matriz de trabajo). Con 8 GB de memoria, MLREML ajusta con comodidad modelos de unos 6,000 registros; con 16 GB, de unos 9,000. En un modelo con un solo término aleatorio el algoritmo AI converge en muy pocas iteraciones, de modo que `mlreml()` es tan rápido como `gblup()` o más. El costo de `gblup()` está dominado por la descomposición espectral, que es varias veces más cara que una factorización de Cholesky; su ventaja aparece cuando el modelo general necesita muchas iteraciones o cuando la descomposición puede reutilizarse.

El número de parámetros influye linealmente: cada parámetro requiere formar su matriz derivada y una traza, de costo $O(n^2)$. Los modelos con muchos parámetros (`us` con muchos ambientes, `fa` con varios factores) necesitan además más iteraciones.

Motor disperso. Para medir el motor disperso se simularon poblaciones de tipo lechero con diez generaciones superpuestas, toros de inseminación artificial usados en varios hatos, madres dentro de hatos, registros en todos los animales con padres conocidos y grupos contemporáneos definidos por hato y año (efectos fijos, unos 20 registros cada uno). Se ajustaron el modelo animal aditivo `y ~ hy`, `random = ~ vm(id, Ai)`, y el modelo directo–materno con `random = ~ str(id, dam, K = Ai)` en la misma computadora que antes (un núcleo, OpenBLAS, 8 GB de memoria), junto con un modelo sin pedigree para registros repetidos de vacas en hatos, `y ~ year`, `random = ~ herd + cow`, con cuatro registros por vaca. El código se instala con el paquete: `system.file("scale", "sparse_scale.R", package = "MLREML")`.

Motor disperso: tamaño del problema, tiempo de cómputo y memoria (elementos del factor en su almacenamiento supernodal, que incluye los ceros explícitos de los supernodos).

Modelo	Registros	Animales o vacas	Ecuaciones	Elementos del factor (millones)	Iteraciones	Tiempo (s)	Memoria máxima (GB)

Modelo	Registros	Animales o vacas	Ecuaciones	Elementos del factor (millones)	Iteraciones	Tiempo (s)	Memoria máxima (GB)
Modelo animal	23,750	25,000	26,234	1.5	4	3.6	0.34
Modelo animal	47,500	50,000	52,468	3.8	4	12.3	0.65
Modelo animal	95,000	100,000	104,908	10.7	4	20.1	0.77
Modelo animal	237,500	250,000	262,287	49.4	5	125.1	2.36
Directo–materno	23,750	25,000	51,234	4.0	8	20.0	0.54
Directo–materno	47,500	50,000	102,468	11.1	5	48.6	1.19
Directo–materno	95,000	100,000	204,908	34.4	8	134.5	2.15
Registros repetidos	1,000,000	250,000	250,358	3.0	5	26.5	1.43
Registros repetidos	2,000,000	500,000	500,691	6.0	7	63.2	2.56

Las estimaciones fueron cercanas a los valores simulados en todos los casos (por ejemplo, 0.304 y 0.644 para las varianzas aditiva y residual del modelo animal con 100,000 registros, simuladas con 0.30 y 0.60 más un componente materno que el modelo aditivo no incluye; y 2.00, 1.00 y 2.99 para las varianzas de hato, vaca y residual del modelo con un millón de registros, simuladas con 2, 1 y 3). El tiempo y la memoria dependen del número de elementos distintos de cero del factor de Cholesky de las ecuaciones del modelo mixto más que del número de registros. En los modelos animales el pedigree y los grupos contemporáneos vinculan a muchos animales y el factor se rellena: de 100,000 a 250,000 registros su tamaño se multiplicó por 4.6 y el tiempo por 6. Con 8 GB de memoria, los modelos animales más grandes que pudieron ajustarse con esta estructura de población fueron el modelo aditivo con 250,000 registros y el modelo directo–materno con 100,000 registros; los siguientes tamaños probados (500,000 y 250,000 registros) excedieron la memoria. Un modelo cuyos efectos aleatorios están anidados, como los registros repetidos de vacas dentro de hatos, tiene un factor con pocos elementos distintos de cero, y el motor disperso lo ajustó con uno y dos millones de registros en aproximadamente un minuto. Por lo tanto, los modelos animales con millones de registros necesitan una computadora con varias veces más memoria que la usada aquí o, para las evaluaciones nacionales rutinarias, los programas especializados con métodos de solución iterativos (Sección 15.4).

15.3 Estrategias para conjuntos de datos grandes

- **Motor disperso.** Con las estructuras que admite, `engine = "sparse"` analiza todos los registros en un solo paso (Sección 2.12); use `Ainverse()` para los parentescos por pedigree, y recuerde que su costo depende del relleno del factor de Cholesky, que crece con el número de efectos fijos y aleatorios conectados por los registros.
- **Análisis en dos etapas.** Ajustar primero cada ensayo o ambiente por separado y analizar después las medias ajustadas (BLUE) con sus pesos (Smith et al., 2001; Piepho

et al., 2012). Las medias reducen el número de observaciones a una por genotipo y ambiente.

- **Compresión de datos.** Resumir individuos en grupos (por ejemplo, por cuantiles del fenotipo) y ajustar un modelo ponderado a las medias de grupo.
- **Submuestras para estimar varianzas.** Estimar los componentes en una submuestra representativa y fijarlos (`start` y `fix`) para predecir en el conjunto completo con `gblup()`.
- **Más memoria y una BLAS paralela.** Con OpenBLAS o MKL multihilo, el tiempo por iteración se reduce casi en proporción al número de núcleos.

15.4 Limitaciones de la versión 1.0.0

1. **Escala.** El motor denso está limitado a unos pocos miles de registros. En una computadora con 8 GB de memoria el motor disperso ajusta modelos animales con unos cientos de miles de registros y modelos con efectos aleatorios anidados con millones de registros (Sección 15.2), porque su memoria crece con el relleno del factor de Cholesky; no pretende sustituir a los programas especializados (ASReml, BLUPF90, DMU, WOMBAT) para evaluaciones nacionales con decenas de millones de registros, métodos de solución por iteración sobre los datos o modelos genómicos single-step con cientos de miles de animales genotipados. Las estructuras con parámetros de correlación (`cs`, `ar1`, `expc`, `sexp`) y las estructuras factor analíticas solo están disponibles en el motor denso.
2. **Covarianza entre términos.** Dos términos aleatorios solo pueden estar correlacionados mediante `str()`, que requiere que sus factores compartan los niveles de la misma matriz **K** (por ejemplo, los efectos genéticos directo y materno); no están disponibles las correlaciones entre términos con matrices de relación distintas o con bases distintas. Un intercepto y una pendiente se correlacionan dentro de un único término de regresión aleatoria.
3. **Grados de libertad.** Los grados de libertad de Kenward–Roger están disponibles para el motor denso; el motor disperso usa $n - p$, y no se implementa la aproximación de Satterthwaite.
4. **Distribuciones.** Solo se ajustan respuestas normales; no hay modelos lineales generalizados mixtos.
5. **Estructuras.** No hay estructuras específicas de ASReml como `corgh` con varianzas por separado, ARMA, splines cúbicos aleatorios o `at()` para términos condicionales, aunque `diag()` y `us()` cubren la mayoría de los usos de `at()`. La correlación de los modelos espaciales es común a todos los ambientes cuando se usa `diag(env):ar1(row):ar1(col)`.
6. **Errores estándar en la frontera.** Como en ASReml, los errores estándar de parámetros en su límite no son válidos.

Estas limitaciones son las prioridades de desarrollo para versiones futuras, en particular la extensión del motor disperso a las estructuras con parámetros de correlación y a los modelos factor analíticos, y el ajuste de Kenward–Roger para el motor disperso.

16 Resumen de funciones

La siguiente tabla resume todas las funciones exportadas. La documentación completa de cada una, con sus argumentos, valores y ejemplos, está en el manual de referencia que se incluye como apéndice al final de este documento y que también se consulta en R con `?nombre_de_la_funcion`.

Funciones de MLREML.

Función	Propósito
`mlreml()`	Ajusta un modelo lineal mixto por AI-REML (motor denso o disperso).
`summary()`, `print()`	Componentes de varianza, verosimilitud, AIC y BIC; efectos fijos con errores estándar.
`coef()`	Efectos fijos (`type = "fixed"`) o BLUP con PEV, error estándar y confiabilidad (`type = "random"`).
`fitted()`, `residuals()`, `vcov()`, `logLik()`	Métodos estándar de R.
`wald()`	Pruebas de Wald incrementales y condicionales para los términos fijos, con grados de libertad del denominador de Kenward–Roger o residuales.
`predict()`	Medias predichas, errores estándar y matriz de SED para una clasificación.
`vpredict()`	Funciones de los parámetros de varianza con errores estándar por el método delta.
`lrt()`	Prueba de razón de verosimilitudes, con corrección por frontera.
`vc_matrix()`	Matriz de covarianza y de correlación de un componente `diag`, `us` o `fa` o de un término `str()`; cargas y varianzas específicas.
`variogram()`	Variograma muestral de los residuos de un ensayo de campo.
`plot()`	Gráficos de diagnóstico y de convergencia.
`pedigree_prepare()`	Limpia, completa y ordena un pedigree.
`inbreeding_coef()`	Coeficientes de consanguinidad (Meuwissen y Luo, 1992).
`Amatrix()`	Matriz de parentesco aditivo densa (método tabular).
`Ainverse()`	Inversa dispersa de **A** (reglas de Henderson con consanguinidad), utilizable directamente en `vm()` y `str()`.
`snp_qc()`	Control de calidad de marcadores (MAF, tasa de llamado por marcador e individuo).
`Gmatrix()`	Matriz de relaciones genómicas (VanRaden, Yang o escalada).
`align_G()`	Alinea **G** con $\mathbf{A}_{22}$.
`Hmatrix()`	Matriz single-step **H** o su inversa.
`bend()`	Convierte una matriz simétrica en definida positiva.
`gblup()`	GBLUP rápido por rotación espectral.
`cv_gblup()`	Validación cruzada de GBLUP en k grupos.

16.1 Equivalencias de sintaxis con ASReml-R

Para los usuarios de ASReml-R, la tabla siguiente traduce las construcciones más comunes.

Traducción de la sintaxis de ASReml-R.

ASReml-R (versión 4)	MLREML
`asreml(fixed = y ~ rep, random = ~ gen, data = d)`	`mlreml(y ~ rep, random = ~ gen, data = d)`

ASReml-R (versión 4)	MLREML
`random = ~ vm(animal, ainv)`	`random = ~ vm(animal, A)` (matriz **A**) o `random = ~ vm(animal, Ainverse(ped))`
`random = ~ fa(env, 2):id(gen)`	igual
`random = ~ us(env):vm(gen, Ginv)`	`random = ~ us(env):vm(gen, G)`
`residual = ~ dsum(~ units | env)`	`residual = ~ diag(env):units`
`residual = ~ ar1(row):ar1(col)`	igual
`random = ~ str(~ animal + dam, ~ us(2):vm(animal, ainv))`	`random = ~ str(animal, dam, K = A)`
`random = ~ leg(age, 2):id(animal)` con `us()`	`random = ~ us(leg(age, 2)):id(animal)`
`random = ~ and(male)` junto con `female`	`random = ~ mm(female, male)`
`fixed = cbind(y1, y2) ~ trait, residual = ~ id(units):us(trait)`	`cbind(y1, y2) ~ trait, residual = ~ us(trait):units`
`summary(fit)$varcomp`	`summary(fit)$varcomp`
`wald(fit)`	`wald(fit)` (grados de libertad del denominador de Kenward–Roger por defecto)
`predict(fit, classify = "gen")`	`predict(fit, classify = "gen")`
`vpredict(fit, h2 ~ V1/(V1+V2))`	igual
`lrt(fit0, fit1)`	igual

La diferencia más importante es que `vm()` recibe la propia matriz de relaciones, porque el motor denso de MLREML trabaja con la forma marginal del modelo; también puede darse su inversa cuando lleva el atributo `INVERSE = TRUE`, como el resultado de `Ainverse()`. Con el motor denso las matrices pueden ser singulares; el motor disperso las necesita definidas positivas.

16.2 Guía rápida de modelos

Las plantillas siguientes cubren los análisis más comunes. Los nombres de variables son ilustrativos.

```
## Bloques completos, genotipos aleatorios
mlreml(y ~ rep, random = ~ gen, data = d)

## Alfa-látice o bloques incompletos, genotipos fijos
mlreml(y ~ rep + gen, random = ~ rep:bloque, data = d)

## Análisis espacial fila x columna con nugget y tendencia lineal
mlreml(y ~ rep + fila_num, random = ~ gen + units,
       residual = ~ ar1(fila):ar1(columna), data = d)

## Espacial irregular (coordenadas)
mlreml(y ~ 1, random = ~ gen + units, residual = ~ sexp(x, y), data = d)

## MET: varianzas genéticas y residuales por ambiente
mlreml(y ~ env, random = ~ env:rep + diag(env):id(gen),
       residual = ~ diag(env):units, data = d)

## MET: factor analítico de dos factores
mlreml(y ~ env, random = ~ env:rep + fa(env, 2):id(gen),
       residual = ~ diag(env):units, data = d, maxit = 200)
```

```r
## GBLUP y GBLUP multiambiental
mlreml(y ~ 1, random = ~ vm(gen, G), data = d)
mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~ diag(env):units,
data = d)

## Pedigree + marcadores (dos núcleos)
mlreml(y ~ 1, random = ~ vm(gen, A) + vm(gen, G), data = d)

## Single-step
H <- Hmatrix(A, G); mlreml(y ~ 1, random = ~ vm(id, H), data = d)

## Modelo animal, repetibilidad y efecto materno
mlreml(y ~ hato + sexo, random = ~ vm(animal, A), data = d)
mlreml(y ~ hato + parto, random = ~ vm(animal, A) + id(animal), data = d)
mlreml(y ~ hato, random = ~ vm(animal, A) + id(madre), data = d)

## Modelo animal directo-materno con ambiente permanente materno
mlreml(y ~ hato + sexo, random = ~ str(animal, madre, K = A) + id(madre), data
= d)

## Bivariado con correlaciones genética y residual
mlreml(cbind(y1, y2) ~ trait + trait:hato, random = ~ us(trait):vm(animal, A),
       residual = ~ us(trait):units, data = d)

## Medidas repetidas con residuo autorregresivo
mlreml(y ~ trat * tiempo, random = ~ sujeto, residual = ~
ar1(tiempo):id(sujeto), data = d)

## Regresión aleatoria
mlreml(y ~ grupo + edad, random = ~ us(leg(edad, 2)):vm(animal, A) +
       us(leg(edad, 2)):id(animal), residual = ~ diag(clase_edad):units, data =
d)

## Dialelo (ACG, ACE, materno, recíproco)
mlreml(y ~ bloque, random = ~ mm(hembra, macho) + cruza + hembra + cruza_ord,
data = d)

## Componentes de varianza conocidos (solo BLUE y BLUP)
mlreml(y ~ 1, random = ~ vm(gen, G), data = d,
       start = c("vm(gen, G)!sigma2" = 0.3, "R!units!sigma2" = 0.6),
       fix = c("vm(gen, G)!sigma2", "R!units!sigma2"))

## Conjunto de datos grande: motor disperso con la inversa de A
Ai <- Ainverse(ped)
mlreml(y ~ hys + sexo, random = ~ vm(animal, Ai) + id(animal), data = d, engine
= "sparse")
```

16.3 Funciones posteriores al ajuste

```r
summary(fit)                          # componentes de varianza
coef(fit, "random", "vm(gen, G)")     # BLUP, PEV, confiabilidad
wald(fit, ddf = "KR")                 # pruebas de efectos fijos (Kenward-
Roger)
predict(fit, classify = "env:gen")    # medias predichas y SED
vpredict(fit, h2 ~ V1 / (V1 + V2))    # funciones de componentes
vc_matrix(fit, "us(env):vm(gen, G)")  # matrices de covarianza y correlación
lrt(fit0, fit1)                       # razón de verosimilitudes
```

```r
variogram(fit, "fila", "columna")       # variograma muestral
plot(fit)                              # diagnóstico
```

Referencias

Baker, R.J. (1978). Issues in diallel analysis. *Crop Science* 18, 533–536.

Bates, D., Mächler, M., Bolker, B., Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software* 67(1), 1–48.

Belenky, G. et al. (2003). Patterns of performance degradation and restoration during sleep restriction and subsequent recovery. *Journal of Sleep Research* 12, 1–12.

Besag, J., Higdon, D. (1999). Bayesian analysis of agricultural field experiments. *Journal of the Royal Statistical Society B* 61, 691–746.

Burgueño, J., de los Campos, G., Weigel, K., Crossa, J. (2012). Genomic prediction of breeding values when modeling genotype × environment interaction using pedigree and dense molecular markers. *Crop Science* 52, 707–719.

Butler, D.G., Cullis, B.R., Gilmour, A.R., Gogel, B.J., Thompson, R. (2017). *ASReml-R Reference Manual Version 4*. VSN International, Hemel Hempstead.

Chen, Y., Davis, T.A., Hager, W.W., Rajamanickam, S. (2008). Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software* 35(3), 22.

Christensen, O.F., Lund, M.S. (2010). Genomic prediction when some animals are not genotyped. *Genetics Selection Evolution* 42, 2.

Christensen, O.F., Madsen, P., Nielsen, B., Ostersen, T., Su, G. (2012). Single-step methods for genomic evaluation in pigs. *Animal* 6, 1565–1571.

Cockerham, C.C., Weir, B.S. (1977). Quadratic analyses of reciprocal crosses. *Biometrics* 33, 187–203.

Crossa, J. et al. (2010). Prediction of genetic values of quantitative traits in plant breeding using pedigree and molecular markers. *Genetics* 186, 713–724.

Crowder, M.J., Hand, D.J. (1990). *Analysis of Repeated Measures*. Chapman and Hall, Londres.

Cullis, B.R., Smith, A.B., Coombes, N.E. (2006). On the design of early generation variety trials with correlated data. *Journal of Agricultural, Biological and Environmental Statistics* 11, 381–393.

Erisman, A.M., Tinney, W.F. (1975). On computing certain elements of the inverse of a sparse matrix. *Communications of the ACM* 18, 177–179.

Gilmour, A.R., Cullis, B.R., Verbyla, A.P. (1997). Accounting for natural and extraneous variation in the analysis of field experiments. *Journal of Agricultural, Biological and Environmental Statistics* 2, 269–293.

Gilmour, A.R., Gogel, B.J., Cullis, B.R., Welham, S.J., Thompson, R. (2015). *ASReml User Guide Release 4.1*. VSN International, Hemel Hempstead.

Gilmour, A.R., Thompson, R., Cullis, B.R. (1995). Average information REML: an efficient algorithm for variance parameter estimation in linear mixed models. *Biometrics* 51, 1440–1450.

Griffing, B. (1956). Concept of general and specific combining ability in relation to diallel crossing systems. *Australian Journal of Biological Sciences* 9, 463–493.

Halekoh, U., Højsgaard, S. (2014). A Kenward–Roger approximation and parametric bootstrap methods for tests in linear mixed models — the R package pbkrtest. *Journal of Statistical Software* 59(9).

Hayman, B.I. (1954). The analysis of variance of diallel tables. *Biometrics* 10, 235–244.

Henderson, C.R. (1975). Best linear unbiased estimation and prediction under a selection model. *Biometrics* 31, 423–447.

Henderson, C.R. (1976). A simple method for computing the inverse of a numerator relationship matrix used in prediction of breeding values. *Biometrics* 32, 69–83.

Henderson, C.R. (1984). *Applications of Linear Models in Animal Breeding*. University of Guelph.

John, J.A., Williams, E.R. (1995). *Cyclic and Computer Generated Designs*. Chapman and Hall, Londres.

Johnson, D.L., Thompson, R. (1995). Restricted maximum likelihood estimation of variance components for univariate animal models using sparse matrix techniques and average information. *Journal of Dairy Science* 78, 449–456.

Kang, H.M. et al. (2008). Efficient control of population structure in model organism association mapping. *Genetics* 178, 1709–1723.

Kenward, M.G., Roger, J.H. (1997). Small sample inference for fixed effects from restricted maximum likelihood. *Biometrics* 53, 983–997.

Kirkpatrick, M., Lofsvold, D., Bulmer, M. (1990). Analysis of the inheritance, selection and evolution of growth trajectories. *Genetics* 124, 979–993.

Legarra, A., Aguilar, I., Misztal, I. (2009). A relationship matrix including full pedigree and genomic information. *Journal of Dairy Science* 92, 4656–4663.

Lin, L., Yang, C., Meza, J.C., Lu, J., Ying, L., E, W. (2011). SelInv — an algorithm for selected inversion of a sparse symmetric matrix. *ACM Transactions on Mathematical Software* 37(4), 40.

Meuwissen, T.H.E., Hayes, B.J., Goddard, M.E. (2001). Prediction of total genetic value using genome-wide dense marker maps. *Genetics* 157, 1819–1829.

Meuwissen, T.H.E., Luo, Z. (1992). Computing inbreeding coefficients in large populations. *Genetics Selection Evolution* 24, 305–313.

Meyer, K. (1992). Variance components due to direct and maternal effects for growth traits of Australian beef cattle. *Livestock Production Science* 31, 179–204.

Misztal, I., Legarra, A., Aguilar, I. (2014). Using recursion to compute the inverse of the genomic relationship matrix. *Journal of Dairy Science* 97, 3943–3952.

Misztal, I., Perez-Enciso, M. (1993). Sparse matrix inversion for restricted maximum likelihood estimation of variance components by expectation-maximization. *Journal of Dairy Science* 76, 1479–1483.

Patterson, H.D., Thompson, R. (1971). Recovery of inter-block information when block sizes are unequal. *Biometrika* 58, 545–554.

Pérez, P., de los Campos, G. (2014). Genome-wide regression and prediction with the BGLR statistical package. *Genetics* 198, 483–495.

Piepho, H.P., Möhring, J., Schulz-Streeck, T., Ogutu, J.O. (2012). A stage-wise approach for the analysis of multi-environment trials. *Biometrical Journal* 54, 844–860.

Pinheiro, J.C., Bates, D.M. (2000). *Mixed-Effects Models in S and S-PLUS*. Springer, Nueva York.

Potthoff, R.F., Roy, S.N. (1964). A generalized multivariate analysis of variance model useful especially for growth curve problems. *Biometrika* 51, 313–326.

Quaas, R.L. (1976). Computing the diagonal elements and inverse of a large numerator relationship matrix. *Biometrics* 32, 949–953.

Schaeffer, L.R. (2004). Application of random regression models in animal breeding. *Livestock Production Science* 86, 35–45.

Self, S.G., Liang, K.Y. (1987). Asymptotic properties of maximum likelihood estimators and likelihood ratio tests under nonstandard conditions. *Journal of the American Statistical Association* 82, 605–610.

Smith, A.B., Cullis, B.R. (2018). Plant breeding selection tools built on factor analytic mixed models for multi-environment trial data. *Euphytica* 214, 143.

Smith, A.B., Cullis, B.R., Thompson, R. (2001). Analyzing variety by environment data using multiplicative mixed models and adjustments for spatial field trend. *Biometrics* 57, 1138–1147.

Smith, A.B., Cullis, B.R., Thompson, R. (2005). The analysis of crop cultivar breeding and evaluation trials: an overview of current mixed model approaches. *Journal of Agricultural Science* 143, 449–462.

Stram, D.O., Lee, J.W. (1994). Variance components testing in the longitudinal mixed effects model. *Biometrics* 50, 1171–1177.

Takahashi, K., Fagan, J., Chen, M.S. (1973). Formation of a sparse bus impedance matrix and its application to short circuit study. *Proceedings of the 8th Power Industry Computer Applications (PICA) Conference*, Minneapolis, 63–69.

VanRaden, P.M. (2008). Efficient methods to compute genomic predictions. *Journal of Dairy Science* 91, 4414–4423.

Vazquez, A.I., Bates, D.M., Rosa, G.J.M., Gianola, D., Weigel, K.A. (2010). Technical note: an R package for fitting generalized linear mixed models in animal breeding. *Journal of Animal Science* 88, 497–504.

Welham, S., Cullis, B., Gogel, B., Gilmour, A., Thompson, R. (2004). Prediction in linear mixed models. *Australian and New Zealand Journal of Statistics* 46, 325–347.

Willham, R.L. (1963). The covariance between relatives for characters composed of components contributed by related individuals. *Biometrics* 19, 18–27.

Willham, R.L. (1972). The role of maternal effects in animal breeding: III. Biometrical aspects of maternal effects in animals. *Journal of Animal Science* 35, 1288–1293.

Wright, K. (2024). *agridat: Agricultural Datasets*. Paquete de R, CRAN.

Apéndice: manual de referencia de las funciones

Esta sección reproduce la ayuda de cada función del paquete, que está en inglés como es habitual en los paquetes de CRAN. En R se consulta con `?nombre_de_la_funcion`.

16.4 `MLREML-package`: MLREML: linear mixed models by Average-Information REML

16.4.0.1 Description

MLREML fits linear mixed models for plant and animal breeding by restricted maximum likelihood with the Average Information algorithm. The main function is `mlreml`. Two computing engines are available: a dense engine based on the variance matrix of the observations, which supports every variance structure, and a sparse engine based on the mixed model equations, the sparse Cholesky factorisation and the Takahashi sparse inverse, for data sets with hundreds of thousands to millions of records. User guides in English and in Spanish with the theory and worked examples with real data are installed with the package: `system.file("doc", "MLREML-guide-en.pdf", package = "MLREML")` and `system.file("doc", "MLREML-guide-es.pdf", package = "MLREML")`.

16.4.0.2 Details

Main functions:

> `mlreml`: model fitting (including direct-maternal models with the `str()` term and the sparse engine).
>
> `summary.mlreml`, `wald` (with Kenward-Roger denominator degrees of freedom), `predict.mlreml`, `vpredict`, `lrt`, `vc_matrix`, `variogram`: inference.
>
> `pedigree_prepare`, `Amatrix`, `Ainverse`, `inbreeding_coef`, `snp_qc`, `Gmatrix`, `align_G`, `Hmatrix`, `bend`: relationship matrices.
>
> `gblup`, `cv_gblup`: fast GBLUP.

16.4.0.3 Author(s)

Maintainer: Osval A. Montesinos-Lopez maintainer@example.org

16.5 `mlreml`: Fit a linear mixed model by Average-Information REML

16.5.0.1 Description

`mlreml` fits the linear mixed model

$$y = X\beta + \sum_k Z_k\, u_k + e, \quad u_k \sim N(0, G_k), \quad e \sim N(0, R)$$

by restricted maximum likelihood (REML) with the Average Information (AI) algorithm. The covariance matrices G_k and R are specified with an ASReml-like syntax as direct (Kronecker) products of variance structures.

16.5.0.2 Usage

```
mlreml(
  fixed,
  random = NULL,
  residual = NULL,
  data,
  start = NULL,
  fix = NULL,
  maxit = 50,
  tol = 1e-06,
  pev = TRUE,
  trace = TRUE,
  store = TRUE,
  engine = c("auto", "dense", "sparse")
)
```

16.5.0.3 Arguments

Argument	Description
`fixed`	two-sided formula for the fixed effects.
`random`	one-sided formula with the random terms, or `NULL`.
`residual`	one-sided formula for the residual structure.
`data`	data frame.
`start`	optional named vector of starting values (names as in `summary(fit)$varcomp`).
`fix`	character vector with names of variance parameters to be held fixed at their starting values.
`maxit`	maximum number of AI iterations.
`tol`	convergence tolerance for the change of the REML log-likelihood.
`pev`	logical; compute prediction error variances of the BLUPs.
`trace`	logical; print the iteration history.
`store`	logical; keep the matrices needed by `predict.mlreml` and by the Kenward-Roger adjustment of `wald`.
`engine`	`"auto"`, `"dense"` or `"sparse"` (see the section on computing engines).

16.5.0.4 Value

An object of class `"mlreml"`.

16.5.0.5 Specifying random terms and residuals

Each random term is a product of components separated by `:`. Each component is a structure function applied to a factor, to a polynomial basis of a covariate or to a multi-membership factor:

`id(f)` or `f`
independent effects, identity matrix.

vm(f, K)
effects correlated according to a known matrix K (pedigree A, genomic G, single-step H, kinship). K must have rownames that contain all levels of f; it may contain additional levels (individuals without records), whose BLUPs are also returned. K may also be given through its inverse, flagged with the attribute INVERSE = TRUE (as returned by Ainverse); the sparse engine then uses it directly.

diag(f)
independent effects with a different variance for each level of f.

us(f)
unstructured (fully parameterised) covariance matrix.

fa(f, k)
factor analytic structure with k factors, $\Lambda\Lambda' + \Psi$.

cs(f)
compound symmetry correlation.

ar1(f)
first-order autoregressive correlation; if the levels of f are numeric, the actual distances are used, so gaps are allowed.

expc(x)
exponential correlation $\rho^{|x_i - x_j|}$ for an irregularly spaced numeric covariate.

sexp(x, y)
isotropic exponential correlation $\rho^{d_{ij}}$ on two-dimensional coordinates (irregular spatial data).

pol(x, k), leg(x, k)
raw and normalised Legendre polynomial bases of order k (random regression); usually combined with us() or diag().

mm(f1, f2)
multi-membership factor (for example, general combining ability of the two parents in a diallel).

units
the observation units (one level per record).

A term whose components are all correlation structures (id, vm, cs, ar1, expc, sexp) receives a scale variance. At most one variance structure (diag, us, fa) is allowed per term, and it carries the scale.

The residual formula uses the same syntax; the default is ~ units (independent residuals with a common variance) and, for multivariate models, ~ us(trait):units.

16.5.0.6 Correlated effects of several factors (direct-maternal models)

The term str(f1, f2, ..., K = A, cov = "us") defines $m \geq 2$ random effects that share the levels of the matrix K and are correlated with each other: with $u = ({u_1}', \dots, {u_m}')'$ and $Z = [Z_1, \dots, Z_m]$,

$$Var(u) = \Sigma \otimes K,$$

where Σ is an unstructured (cov = "us") or diagonal (cov = "diag") $m \times m$ matrix. The usual application is the direct-maternal animal model, str(animal, dam, K = A), in which Z_1

links each record to the animal and Z_2 to its dam, and Σ contains the direct and maternal genetic variances and their covariance. A missing value of a factor of `str()` (for example an unknown dam) means that the record receives no effect from it; such records are kept. If `K` is omitted the levels are independent ($K = I$).

16.5.0.7 Computing engines

The `"dense"` engine works with the $n \times n$ matrix $V = Var(y)$ and supports every structure; its cost grows with n^3 and its memory with n^2, so it is meant for up to a few thousand records. The `"sparse"` engine works with Henderson's mixed model equations, a sparse Cholesky factorisation (CHOLMOD, through the Matrix package) and the sparse (Takahashi) inverse, so that its cost depends on the number of non-zero elements rather than on n^2; it handles hundreds of thousands to millions of records. It supports random terms built from `id`, `vm`, `diag` and `us` components (with factors, `pol`, `leg` or `mm` bases) and `str()` terms, and the residual structures `units`, `diag(f):units` and, in multivariate models, `us(trait):units`. `engine = "auto"` (default) uses the sparse engine when there are more than 5000 records and the model is supported, and the dense engine otherwise. Both engines give the same estimates, BLUPs and prediction error variances up to numerical precision.

16.5.0.8 Multivariate models

If the left-hand side of `fixed` is `cbind(y1, y2, ...)`, the data are stacked internally, a factor `trait` is created, and `units` identifies the original records, so that `us(trait):units` is the residual covariance matrix between traits measured on the same record. Records with some traits missing are kept for the traits observed.

16.5.0.9 References

Gilmour, A.R., Thompson, R., Cullis, B.R. (1995). Average information REML: an efficient algorithm for variance parameter estimation in linear mixed models. *Biometrics* 51, 1440-1450.

Johnson, D.L., Thompson, R. (1995). Restricted maximum likelihood estimation of variance components for univariate animal models using sparse matrix techniques and average information. *J. Dairy Sci.* 78, 449-456.

Misztal, I., Perez-Enciso, M. (1993). Sparse matrix inversion for restricted maximum likelihood estimation of variance components by expectation-maximization. *J. Dairy Sci.* 76, 1479-1483.

Willham, R.L. (1963). The covariance between relatives for characters composed of components contributed by related individuals. *Biometrics* 19, 18-27.

16.5.0.10 Examples

```
set.seed(1)
d <- data.frame(block = factor(rep(1:4, each = 30)),
                gen = factor(rep(1:30, 4)))
d$y <- 10 + rnorm(4)[d$block] + rnorm(30, sd = 1.5)[d$gen] + rnorm(120)
fit <- mlreml(y ~ block, random = ~ id(gen), data = d)
summary(fit)
```

16.6 `summary.mlreml`: Print and summary methods for mlreml fits

16.6.0.1 Description

Print and summary methods for mlreml fits

16.6.0.2 Usage

```
## S3 method for class 'mlreml'
summary(object, ...)

## S3 method for class 'summary.mlreml'
print(x, ...)

## S3 method for class 'mlreml'
print(x, ...)

## S3 method for class 'mlreml'
fitted(object, ...)

## S3 method for class 'mlreml'
residuals(object, ...)

## S3 method for class 'mlreml'
vcov(object, ...)

## S3 method for class 'mlreml'
logLik(object, ...)
```

16.6.0.3 Arguments

Argument	Description
`object, x`	an object of class `"mlreml"`.
`...`	further arguments (ignored).

16.6.0.4 Value

`summary` returns a list with the variance components (`varcomp`), the fixed effects with standard errors (`coef.fixed`), the log-likelihood and information criteria.

16.7 `coef.mlreml`: Extract fixed and random effects

16.7.0.1 Description

Extract fixed and random effects

16.7.0.2 Usage

```
## S3 method for class 'mlreml'
coef(object, type = c("fixed", "random"), term = NULL, ...)

ranef_mlreml(object, term = NULL)

fixef_mlreml(object)
```

16.7.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`type`	`"fixed"` or `"random"`.
`term`	for random effects, the label of a random term (as in `names(object$blups)`); `NULL` returns all terms.
`...`	ignored.

16.7.0.4 Value

A named vector (fixed effects) or a data frame (BLUPs with prediction error variance, standard error and reliability).

16.8 `wald`: Wald tests for fixed effects

16.8.0.1 Description

Computes incremental and conditional Wald statistics for every term of the fixed formula, with F tests whose denominator degrees of freedom are obtained by the method of Kenward and Roger (1997) or, as a large-sample approximation, by the residual degrees of freedom $n - rank(X)$.

16.8.0.2 Usage

```
wald(object, ddf = c("auto", "KR", "residual"))
```

16.8.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`ddf`	`"auto"` (Kenward-Roger for fits of the dense engine stored with `store = TRUE`, residual degrees of freedom otherwise), `"KR"` or `"residual"`.

16.8.0.4 Details

The incremental statistic of a term tests it adjusted for the terms that precede it in the fixed formula (sequential, "type I" tests). The conditional statistic tests a term adjusted for all other terms that do not contain it, respecting the principle of marginality as ASReml does (equivalent to "type II" tests): the main effect `A` in `y ~ A * B` is adjusted for `B` but not for `A:B`. Both statistics are computed from the Cholesky factor of the inverse of the covariance matrix of the fixed effects, with the columns reordered for the conditional statistic (the term after the terms that do not contain it and before the terms that contain it).

With the Kenward-Roger method, the statistic of each hypothesis $L\beta = 0$ is $F = \lambda\left(L\hat{\beta}\right)'(L\Phi_A L')^{-1}\left(L\hat{\beta}\right)/q$, where Φ_A is the covariance matrix of the fixed effects adjusted for the uncertainty of the estimated variance parameters, and the denominator degrees of freedom m and the scale λ are obtained by matching the moments of F with those of an $F(q, m)$ distribution. The expected information matrix is used; the second derivatives of V are included for structures that are not linear in their parameters (`ar1`, `expc`, `sexp`, `fa`, products of parameterised components). Variance parameters that are fixed or on the boundary of the parameter space are treated as known.

16.8.0.5 Value

A data frame with one row per fixed term: numerator degrees of freedom (`df`); incremental Wald statistic, F statistic, denominator degrees of freedom and p-value (`wald.inc`, `F.inc`, `ddf.inc`, `p.inc`); the same for the conditional test (`wald.con`, `F.con`, `ddf.con`, `p.con`); and the p-value of the conditional Wald statistic referred to a chi-square distribution (`p.chisq`). The attribute `"ddf"` gives the method used.

16.8.0.6 References

Kenward, M.G., Roger, J.H. (1997). Small sample inference for fixed effects from restricted maximum likelihood. *Biometrics* 53, 983-997.

Halekoh, U., Hojsgaard, S. (2014). A Kenward-Roger approximation and parametric bootstrap methods for tests in linear mixed models - the R package pbkrtest. *J. Stat. Softw.* 59(9).

16.8.0.7 Examples

```
set.seed(2)
d <- data.frame(trt = gl(3, 20), blk = gl(4, 5, 60))
d$y <- c(0, 1, 2)[d$trt] + rnorm(4)[d$blk] + rnorm(60)
fit <- mlreml(y ~ trt, random = ~ blk, data = d, trace = FALSE)
wald(fit)
wald(fit, ddf = "residual")
```

16.9 `predict.mlreml`: Predicted means with standard errors and standard errors of differences

16.9.0.1 Description

Forms predictions for the levels of one or more factors (`classify`), averaging the fixed effects over the levels of the other fixed factors (simple averaging) and evaluating covariates at their mean. Random terms whose factors are all contained in `classify` are included as BLUPs; the remaining random terms are ignored (their expectation is zero). The standard errors and the matrix of standard errors of differences (SED) are computed from the joint prediction error variance of fixed and random effects, following Welham et al. (2004).

16.9.0.2 Usage

```
## S3 method for class 'mlreml'
predict(object, classify, sed = TRUE, ...)
```

16.9.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit (fitted with `store = TRUE`).
`classify`	character, for example `"gen"` or `"env:gen"`.
`sed`	logical; return the SED matrix.
`...`	ignored.

16.9.0.4 Value

A list with `pvals` (predicted values and standard errors), `sed` (matrix) and `avsed` (average SED).

16.9.0.5 References

Welham, S., Cullis, B., Gogel, B., Gilmour, A., Thompson, R. (2004). Prediction in linear mixed models. *Aust. N. Z. J. Stat.* 46, 325-347.

16.10 `vpredict`: Functions of variance parameters with delta-method standard errors

16.10.0.1 Description

Evaluates a function of the variance parameters (for example a heritability or a genetic correlation) and its approximate standard error from the inverse of the Average Information matrix, as the `vpredict` function of ASReml-R. Parameters are referred to as `V1`, `V2`, ... in the order of `summary(fit)$varcomp`.

16.10.0.2 Usage

```
vpredict(object, formula)
```

16.10.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`formula`	a formula such as `h2 ~ V2 / (V2 + V3)`.

16.10.0.4 Value

A data frame with the estimate and its standard error.

16.10.0.5 Examples

```
set.seed(3)
d <- data.frame(fam = gl(40, 5))
d$y <- rnorm(40, sd = 1)[d$fam] + rnorm(200, sd = 2)
fit <- mlreml(y ~ 1, random = ~ fam, data = d, trace = FALSE)
vpredict(fit, h2 ~ 4 * V1 / (V1 + V2))
```

16.11 `lrt`: Likelihood ratio test between nested models

16.11.0.1 Description

Compares two models with the same fixed effects that differ in their variance structure. With `boundary = TRUE` the p-value uses a 50:50 mixture of chi-square distributions, appropriate when a single variance parameter is tested on the boundary of the parameter space.

16.11.0.2 Usage

```
lrt(fit0, fit1, boundary = TRUE)
```

16.11.0.3 Arguments

Argument	Description
`fit0`	reduced model.
`fit1`	full model.
`boundary`	logical; use the mixture distribution.

16.11.0.4 Value

A data frame with the statistic, degrees of freedom and p-value.

16.12 `vc_matrix`: Estimated covariance and correlation matrix of a structured component

16.12.0.1 Description

Returns the estimated covariance matrix of a `us`, `diag` or `fa` component (for example the genetic covariance matrix between environments or traits), its correlation matrix and, for `fa`, the loadings and specific variances.

16.12.0.2 Usage

```
vc_matrix(object, term)
```

16.12.0.3 Arguments

Argument	Description

Argument	Description
`object`	an `"mlreml"` fit.
`term`	label of the term (as in `object$varcomp$term`), or `"residual"`.

16.12.0.4 Value

A list with `cov` and `cor` (and `loadings`, `psi` for factor analytic structures).

16.13 `variogram`: Sample variogram of residuals of a field trial

16.13.0.1 Description

Computes the empirical semi-variogram of the residuals by absolute row and column displacement, the standard diagnostic of spatial analyses (Gilmour, Cullis and Verbyla, 1997).

16.13.0.2 Usage

```
variogram(object, row, col, plot = TRUE, resid = NULL)
```

16.13.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`row, col`	names of the numeric row and column variables in the data.
`plot`	logical; draw a perspective plot.
`resid`	optional vector of residuals (default: conditional residuals).

16.13.0.4 Value

A data frame with row lag, column lag, semi-variance and number of pairs.

16.13.0.5 References

Gilmour, A.R., Cullis, B.R., Verbyla, A.P. (1997). Accounting for natural and extraneous variation in the analysis of field experiments. *JABES* 2, 269-293.

16.14 `plot.mlreml`: Diagnostic plots

16.14.0.1 Description

Residuals against fitted values, normal quantile plot and histogram of the residuals, and the history of the REML log-likelihood.

16.14.0.2 Usage

```
## S3 method for class 'mlreml'
plot(x, ...)
```

16.14.0.3 Arguments

Argument	Description
`x`	an `"mlreml"` fit.
`...`	passed to `plot`.

16.15 `pedigree_prepare`: Prepare a pedigree

16.15.0.1 Description

Recodes unknown parents, adds parents that do not appear as individuals (as founders), checks for errors and sorts the pedigree so that parents precede their offspring.

16.15.0.2 Usage

```
pedigree_prepare(ped)
```

16.15.0.3 Arguments

Argument	Description
`ped`	data frame whose first three columns are individual, sire (father) and dam (mother). Unknown parents can be coded as `NA`, `0`, `"0"`, `"-"` or `""`.

16.15.0.4 Value

A data frame with columns `id`, `sire`, `dam` (character, `NA` for unknown), `sire_i`, `dam_i` (integer positions, 0 for unknown) and `gen` (generation number).

16.15.0.5 Examples

```
ped <- data.frame(id = c("A3", "A4", "A5"), sire = c("A1", "A1", "A3"),
                  dam = c("A2", "A2", "A4"))
pedigree_prepare(ped)
```

16.16 `inbreeding_coef`: Inbreeding coefficients (Meuwissen and Luo, 1992)

16.16.0.1 Description

Inbreeding coefficients (Meuwissen and Luo, 1992)

16.16.0.2 Usage

```
inbreeding_coef(ped)
```

16.16.0.3 Arguments

Argument	Description
`ped`	a pedigree as returned by `pedigree_prepare`, or any data frame accepted by it.

16.16.0.4 Value

A named vector of inbreeding coefficients.

16.16.0.5 References

Meuwissen, T.H.E., Luo, Z. (1992). Computing inbreeding coefficients in large populations. *Genet. Sel. Evol.* 24, 305-313.

16.17 `Amatrix`: Numerator (additive) relationship matrix

16.17.0.1 Description

Builds the dense additive relationship matrix A with the tabular method. Suitable for up to roughly 15,000 individuals; for larger pedigrees use the sparse inverse `Ainverse`.

16.17.0.2 Usage

```
Amatrix(ped)
```

16.17.0.3 Arguments

Argument	Description
ped	pedigree (see pedigree_prepare).

16.17.0.4 Value

A symmetric matrix with individual names as dimnames.

16.17.0.5 Examples

```
ped <- data.frame(id = c("A3", "A4", "A5"), sire = c("A1", "A1", "A3"),
                  dam = c("A2", "A2", "A4"))
Amatrix(ped)
```

16.18 Ainverse: Sparse inverse of the additive relationship matrix

16.18.0.1 Description

Henderson's rules with inbreeding (Quaas, 1976), using the inbreeding coefficients of Meuwissen and Luo (1992).

16.18.0.2 Usage

```
Ainverse(ped)
```

16.18.0.3 Arguments

Argument	Description
ped	pedigree (see pedigree_prepare).

16.18.0.4 Value

A sparse symmetric matrix (Matrix package) with the attribute INVERSE = TRUE, so that it can be given directly to vm() or str() in mlreml (the sparse engine uses it as it is; the dense engine inverts it).

16.19 snp_qc: Quality control of a SNP matrix

16.19.0.1 Description

Quality control of a SNP matrix

16.19.0.2 Usage

```
snp_qc(M, maf = 0.05, callrate = 0.9, ind_callrate = 0.9)
```

16.19.0.3 Arguments

Argument	Description
M	matrix of genotypes (individuals in rows), coded 0/1/2 or -1/0/1; missing values as NA.
maf	minimum minor-allele frequency.
callrate	minimum call rate per marker.
ind_callrate	minimum call rate per individual.

16.19.0.4 Value

A list with the filtered matrix (M, coded 0/1/2), allele frequencies and a report of the removed markers and individuals.

16.20 `Gmatrix`: Genomic relationship matrix

16.20.0.1 Description

Genomic relationship matrix

16.20.0.2 Usage

```
Gmatrix(M, method = c("VanRaden", "Yang", "scaled"), p = NULL)
```

16.20.0.3 Arguments

Argument	Description
`M`	matrix of genotypes (individuals in rows) coded 0/1/2 or -1/0/1. Missing values are replaced by the marker mean.
`method`	`"VanRaden"` (VanRaden 2008, method 1), `"Yang"` (standardised markers, VanRaden method 2) or `"scaled"` (centred and scaled markers divided by the number of markers, ZZ'/p, the kernel of GBLUP as ridge regression).
`p`	optional vector of base allele frequencies.

16.20.0.4 Value

A symmetric matrix with the rownames of `M`.

16.20.0.5 References

VanRaden, P.M. (2008). Efficient methods to compute genomic predictions. *J. Dairy Sci.* 91, 4414-4423.

16.21 `align_G`: Align a genomic matrix with the pedigree matrix

16.21.0.1 Description

Rescales G as $\alpha + \beta G$ so that the averages of its diagonal and of all its elements equal those of A22 (Christensen et al., 2012).

16.21.0.2 Usage

```
align_G(G, A22)
```

16.21.0.3 Arguments

Argument	Description
`G`	genomic relationship matrix.
`A22`	pedigree relationships of the genotyped individuals.

16.21.0.4 Value

The aligned matrix, with attributes `alpha` and `beta`.

16.22 `Hmatrix`: Single-step relationship matrix H and its inverse

16.22.0.1 Description

Combines the pedigree matrix A of all individuals with the genomic matrix G of the genotyped individuals (Legarra et al., 2009; Aguilar et al., 2010). G is optionally blended with A22 and aligned to it before combining.

16.22.0.2 Usage

```
Hmatrix(A, G, blend = 0.05, align = TRUE, tau = 1, omega = 1, inverse = FALSE)
```

16.22.0.3 Arguments

Argument	Description
`A`	pedigree relationship matrix of all individuals (dense).
`G`	genomic relationship matrix of the genotyped individuals; its rownames must be in `rownames(A)`.
`blend`	weight of A22 in $(1-w)G + wA_{22}$.
`align`	logical; align G to A22 with `align_G`.
`tau`, `omega`	scaling factors of G^{-1} and A_{22}^{-1} in the inverse.
`inverse`	logical; return H^{-1} instead of H.

16.22.0.4 Value

H (or its inverse) with the dimnames of A.

16.22.0.5 References

Legarra, A., Aguilar, I., Misztal, I. (2009). A relationship matrix including full pedigree and genomic information. *J. Dairy Sci.* 92, 4656-4663.

16.23 `bend`: Make a symmetric matrix positive definite (bending)

16.23.0.1 Description

Make a symmetric matrix positive definite (bending)

16.23.0.2 Usage

```
bend(K, tol = 1e-04)
```

16.23.0.3 Arguments

Argument	Description
`K`	symmetric matrix.
`tol`	minimum eigenvalue after bending (relative to the largest).

16.23.0.4 Value

The bent matrix.

16.24 `gblup`: Fast GBLUP by spectral decomposition

16.24.0.1 Description

Fits $y = X\beta + g + e, g \sim N(0, K\sigma_g^2), e \sim N(0, I\sigma_e^2)$ by exact REML using the eigen-decomposition of the relationship matrix of the phenotyped individuals (Kang et al., 2008), which reduces REML to a one-dimensional optimisation. Predictions (GEBVs) are returned for every individual in `K`, phenotyped or not. The results are identical to `mlreml(y ~ ..., random = ~ vm(id, K))` but much faster.

16.24.0.2 Usage

```
gblup(y, K, X = NULL, id = names(y))
```

16.24.0.3 Arguments

Argument	Description
`y`	numeric vector of phenotypes, named by individual, or with the individual identifiers given in `id`.
`K`	relationship matrix with rownames.
`X`	optional fixed-effects design matrix (default: intercept).
`id`	optional character vector of identifiers for `y`.

16.24.0.4 Value

A list with `varcomp` (σ_g^2, σ_e^2, h^2), `beta`, `gebv` (named vector for all individuals in `K`) and `loglik`.

16.24.0.5 References

Kang, H.M. et al. (2008). Efficient control of population structure in model organism association mapping. *Genetics* 178, 1709-1723.

16.25 `cv_gblup`: K-fold cross-validation of GBLUP

16.25.0.1 Description

Randomly partitions the phenotyped individuals into `folds` groups, predicts each group from the others with `gblup` and reports the Pearson correlation between predicted and observed values.

16.25.0.2 Usage

```
cv_gblup(y, K, folds = 5, reps = 1, seed = 1)
```

16.25.0.3 Arguments

Argument	Description
`y`	named vector of phenotypes.
`K`	relationship matrix.
`folds`	number of folds.
`reps`	number of repetitions of the random partition.
`seed`	random seed.

16.25.0.4 Value

A data frame with one row per repetition and fold.